COLLEGE OF ENGINEERING KIDANGOOR

KOTTAYAM - 686583

MICROPROJECT REPORT ON

MOVIE TICKET BOOKING SYSTEM

Submitted to the APJ Abdul Kalam Technological University, Kerala for the partial fulfillment of the requirements for the award of the degree Bachelor of Technology in Computer Science and Engineering

By

JOSEPH JOSHY	**KGR25CS073**
KAAVYANANDA J NAIR	**KGR25CS076**
DEEKSHITHA JAYARAJ	**KGR25CS047**
AJIN SATHEESH	**KGR25CS019**
ADWAITH P MURALI	**KGR25CS014**
M A INDRAJITH MITHRA	**KGR25CS087**

Guided By

GARGI CHANDRABABU

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

November 2026

COLLEGE OF ENGINEERING KIDANGOOR

KOTTAYAM - 686583



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the microproject work report entitled '**MOVIE TICKET BOOKING SYSTEM**' submitted by **JOSEPH JOSHY (KGR25CS073)**, **KAAVYANANDA J NAIR (KGR25CS076)**, **DEEKSHITHA JAYARAJ (KGR25CS047)**, **AJIN SATHEESH (KGR25CS019)**, **ADWAITH P MURALI (KGR25CS014)** and **M A INDRAJITH MITHRA (KGR25CS087)** of Third Semester, B.Tech in Computer Science & Engineering, has been successfully carried out under our supervision and guidance in partial fulfillment of the requirements of the microproject for PBCST304 – Object Oriented Programming, prescribed by APJ Abdul Kalam Technological University, Kerala. It is further certified that this work is a bonafide record of the microproject carried out by the above students during the academic year 2026–2027.

GUIDE

GARGI CHANDRABABU
Assistant Professor
Dept. of CSE
College of Engineering Kidangoor

HOD

Dr. REKHA KS
Assistant Professor
Dept. of CSE
College of Engineering Kidangoor

DECLARATION

We hereby declare that the microproject report MOVIE TICKET BOOKING SYSTEM submitted for the partial fulfillment of the requirements of the Third Semester Microproject for the subject PBCST304 – Object Oriented Programming, prescribed by APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the guidance of Asst. Prof. GARGI CHANDRABABU.

The submission represents our ideas in our own words. Where ideas or words of others have been included, we have adequately and accurately cited and referred to the original sources.

We also declare that we have adhered to ethical and integrity standards and have not misinterpreted or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a case for disciplinary action by the institution and/or the University. This report has not previously formed the basis for the award of any degree, diploma, or similar title from any other university.

Place: Kidangoor
Date: 3/11/2026

JOSEPH JOSHY
KAAVYANANDA J NAIR
DEEKSHITHA JAYARAJ
AJIN SATHEESH
ADWAITH P MURALI
M A INDRAJITH MITHRA

VISION AND MISSION OF COLLEGE

VISION

To be a leading engineering institution in the region, providing competent professionals, who engage in lifelong learning, driven by social values.

MISSION

To prepare engineering graduates for the development activities of the society and industry, and to prepare them for higher engineering education.

VISION AND MISSION OF DEPARTMENT

VISION

To become a center of excellence in Computer Science and Engineering imparting quality professional education to develop competent professionals with social values who are capable of life long learning.

MISSION

To impart quality technical education to students at undergraduate level through constant knowledge upgradation by maintaining pace with the latest sophisticated innovations , research and development and industry interaction in the field of Computer Science and Engineering with focus on lifelong learning for the well-being of the society.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

PEO1	Have sound knowledge and technical skills required to remain productive in the field of computer science and engineering.
PEO2	Be efficient team leaders, effective communicators and successful entrepreneurs.
PEO3	Resolve technical problems with a positive outlook towards well-being of the society.
PEO4	Function in diverse environments with the ability and competence to solve challenging problems.
PEO5	Pursue lifelong learning and professional development through higher education.

PROGRAM OUTCOMES (POs)

PO 1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO 2	Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO 3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO 4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO 5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO 6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO 8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO 9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO 10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO 11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO 12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES (COs)

Code	Course Outcome
CO1	Explain the process of writing, compiling, and executing basic Java programs, including their structure and components, to demonstrate proficiency.
CO2	Utilize object-oriented programming principles in the design and implementation of Java applications.
CO3	Develop and manage Java packages and interfaces, enhancing code modularity and reusability.
CO4	Implement error handling using Java's exception mechanisms and leverage interfaces for modular applications.
CO5	Develop event-driven Java GUI applications with database connectivity using Swing and JDBC.

CO-PO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3										3
CO2	3	3	3									3
CO3	3	3	3		3							3
CO4	3	3	3		3							3
CO5	3	3	3		3							3

CONTENTS

ABSTRACT	**1**
CHAPTER 1 INTRODUCTION	**2**
CHAPTER 2 AIM AND OBJECTIVES	**3**
CHAPTER 3 REQUIREMENT SPECIFICATION	**4**
CHAPTER 4 DATABASE DESIGN	**5–11**
CHAPTER 5 UML CLASS DIAGRAM	**12–14**
CHAPTER 6 SYSTEM OUTPUT / SCREENSHOTS	**14–19**
CHAPTER 7 CODE LISTING	**20–93**
CHAPTER 8 CONCLUSION	**94**

ABSTRACT

The Movie Ticket Booking System (CineBook) is a comprehensive Java-based desktop software application engineered to automate, streamline, and modernise the end-to-end operations of theatrical multiplexes. Traditional cinema ticketing systems heavily suffer from manual box-office queues, delayed seat availability updates, duplicate reservations, and fragmented accounting ledgers. CineBook eliminates these inefficiencies by providing a fully integrated desktop platform developed strictly according to the core principles of Object-Oriented Programming (OOP) — namely abstraction, encapsulation, inheritance, and dynamic runtime polymorphism — paired with an event-driven graphical interface built using Java Swing and relational SQLite persistence managed via the Java Database Connectivity (JDBC) API.

The system enforces strict role-based access control (RBAC) separating Customers and Administrators. Customers can interactively browse the theatrical catalogue with high-resolution film posters, filter movies by title and genre, view screening schedules across multiple auditoriums, reserve tier-based seating through an interactive 2D graphical grid, execute simulated checkout via Strategy and Adapter payment gateways, and review past admissions with digital printable e-tickets. Administrators are granted exclusive access to an operations dashboard providing real-time theatrical catalogue management, automated showtime scheduling, financial transaction audits, and algorithmic recursive occupancy telemetry.

Overall, the Movie Ticket Booking System demonstrates a complete practical implementation of the KTU S3 CSE Object-Oriented Programming (PBCST304) curriculum. It provides a modular, maintainable, and highly robust application architecture with custom exception hierarchies, double-checked locking Singleton patterns, and thread-safe relational persistence.

Chapter 1
INTRODUCTION

In the contemporary entertainment industry, the operational efficiency of movie theatres and multiplexes hinges on prompt, automated, and secure customer self-service systems. Manual ticketing desks and paper ledger logs frequently suffer from severe bottlenecks, including inaccurate seat allocations, human scheduling errors, untracked financial transactions, and long customer waiting times. The Movie Ticket Booking System is designed to address these challenges by replacing manual procedures with an automated, reliable, and user-centric software solution. Built using pure standard Java SE and Swing, the application communicates seamlessly with an embedded SQLite database using standard JDBC drivers.

The architecture of CineBook is rooted in Object-Oriented Programming (OOP) paradigms. User accounts inherit from a common abstract User entity, branching into Customer and AdminUser roles with dynamic polymorphic greetings and view routing. The system segregates interface contracts using interfaces like Cancellable and Searchable<T>, applies the Strategy design pattern for modular payment handling (Credit Card, UPI, Digital Wallet), and incorporates the structural Adapter pattern to interface legacy banking interfaces with modern checkout pipelines. To preserve data integrity across operational sessions, the system persists users, movies, screens, shows, bookings, transactions, and audit trails to SQLite using parameterized PreparedStatements that actively protect against SQL injection vulnerabilities.

By employing a modular six-member team engineering division, the codebase cleanly decouples domain models, business service engines, GUI Swing views, and persistence layers. This project directly showcases the practical realization of Java syntax, GUI components, layout management, thread-safe memory state coordination, custom checked exception handling, and relational database connectivity.

CHAPTER 2

AIM AND OBJECTIVES

AIM

The primary aim of the Movie Ticket Booking System is to design and develop a secure, robust, modular, and user-friendly Java desktop application that automates movie browsing, auditorium showtime scheduling, 2D interactive seat selection, payment simulation, ticket cancellation, and business reporting using Object-Oriented Programming principles and SQLite JDBC connectivity.

OBJECTIVES

- To replace traditional manual counter ticketing with a computerized Java Swing desktop application.
- To apply foundational OOP principles including data abstraction, encapsulation, inheritance, polymorphism, and interfaces in a production-grade architecture.
- To develop an interactive graphical user interface using Swing components (JFrame, JPanel, CardLayout, GridLayout, JTable) and event-driven delegation listeners.
- To integrate an embedded SQLite database using JDBC for persistent storage of users, movies, screens, shows, bookings, financial transactions, and audit logs.
- To implement role-based authentication separating Customer and Administrator views, featuring a mandatory startup login dialog and session reset upon logout.
- To design an interactive 2D auditorium seat selection grid supporting dynamic tier-based pricing multipliers (Standard, Premium, VIP) and 5-minute checkout holds.
- To implement modular payment processing using the Strategy pattern (Card, UPI, Wallet) and Adapter pattern (Legacy Bank DirectDebit).
- To enforce business invariant safety guards, including non-cancellable past bookings, double-confirmation prevention, and safe movie deletion restrictions.
- To calculate real-time multiplex occupancy and revenue metrics using algorithmic recursion and structured audit logging.

CHAPTER 3

REQUIREMENT SPECIFICATION

3.1 SOFTWARE REQUIREMENTS

- **Operating System:** Microsoft Windows 10 / 11, Linux (Ubuntu / Fedora), or macOS
- **Programming Language:** Java Standard Edition (Java SE 17+ / JDK 26)
- **GUI Framework:** Java Swing & Abstract Window Toolkit (AWT)
- **Database Engine:** SQLite 3 (Embedded Serverless Relational Database)
- **Database Driver:** SQLite JDBC Driver (org.sqlite.JDBC v3.47.2.0)
- **Compilation & Execution:** Standard JDK command-line tools (javac and java)
- **Development Tools:** VS Code, IntelliJ IDEA, Eclipse, or NetBeans

3.2 HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (minimum x64 architecture)
- **Random Access Memory (RAM):** Minimum 4 GB (Recommended: 8 GB or higher)
- **Hard Disk Space:** Minimum 500 MB free disk space for project source, binaries, and SQLite database
- **Display Resolution:** 1280 × 800 resolution or higher (optimised for modern widescreen display)
- **Input Peripherals:** Standard QWERTY Keyboard and Mouse / Touchpad

CHAPTER 4

DATABASE DESIGN

The Movie Ticket Booking System utilizes SQLite through the Java Database Connectivity (JDBC) API. The DatabaseManager singleton coordinates all database interactions, verifying the presence of the org.sqlite.JDBC driver, establishing a thread-safe connection to db/cinebook.db, and executing Data Definition Language (DDL) statements upon startup to instantiate the required schema. Foreign key constraints are actively enforced via SQLite PRAGMA directives to maintain referential integrity across movies, shows, and customer bookings. All queries and Data Manipulation Language (DML) operations utilize parameterized PreparedStatement instances, ensuring SQL injection prevention and efficient pre-compiled query plans.

4.1 USERS TABLE

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	Unique user account identifier (e.g. CUST-01, ADM-01)
username	TEXT	UNIQUE, NOT NULL	Login handle used for authentication (admin, user1, user2)
password	TEXT	NOT NULL	Account authentication password credential
name	TEXT	NOT NULL	Full legal name of the user profile
email	TEXT	NOT NULL	Contact email address for communications
role	TEXT	NOT NULL	Security role discriminator: 'Admin' or 'Customer'
loyalty_points	INTEGER	DEFAULT 0	Accumulated reward points for ticket purchases
access_level	TEXT	OPTIONAL	Permission scope for

			administrators (e.g. SuperAdmin)

4.2 MOVIES TABLE

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	Theatrical movie key (e.g. MOV-01)
title	TEXT	NOT NULL	Official feature film presentation title
genre	TEXT	NOT NULL	Film categorization (Sci-Fi, Action, Romance, etc.)
duration	INTEGER	NOT NULL	Total feature runtime in minutes
language	TEXT	NOT NULL	Audio language track of the screening
certification	TEXT	NOT NULL	Film board classification rating (PG, PG-13, R)
description	TEXT	NOT NULL	Official theatrical plot synopsis

4.3 SCREENS TABLE

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	Auditorium screening hall identifier (e.g. SCR-01)
name	TEXT	NOT NULL	Commercial hall name (Screen 1 IMAX Laser, Dolby Atmos)
seat_rows	INTEGER	NOT NULL	Vertical seating row dimension (e.g. 5 rows: A-E)
seat_cols	INTEGER	NOT NULL	Horizontal seating column dimension (e.g. 8 cols: 1-8)

4.4 SHOWS TABLE

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	Screening showtime record key (e.g. SHW-01)
movie_id	TEXT	FOREIGN KEY	References movies(id)
screen_id	TEXT	FOREIGN KEY	References screens(id)
show_time	TEXT	NOT NULL	Formatted screening timestamp (yyyy-MM-dd HH:mm)
base_price	REAL	NOT NULL	Standard tier admission price in USD

4.5 BOOKINGS TABLE

Column	Data Type	Constraint	Description

id	TEXT	PRIMARY KEY	Unique booking reservation reference (e.g. BKG-XXXXXXXX)
customer_id	TEXT	FOREIGN KEY	References users(id) of the purchasing customer
show_id	TEXT	FOREIGN KEY	References shows(id) of the scheduled screening
seat_ids	TEXT	NOT NULL	Comma-separated list of reserved seats (e.g. 'A1,A2')
total_amount	REAL	NOT NULL	Final payment total including 18% GST
created_at	TEXT	NOT NULL	Timestamp when reservation hold was initiated
expires_at	TEXT	NOT NULL	Timestamp when 5-minute checkout hold expires
status	TEXT	NOT NULL	Booking lifecycle: PENDING, CONFIRMED, CANCELLED
cancellation_reason	TEXT	OPTIONAL	Recorded justification for cancellation or refund

4.6 TRANSACTIONS TABLE

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	Ledger transaction reference key (e.g. TXN-XXXXXXXX)
booking_id	TEXT	FOREIGN KEY	References bookings(id) associated with the payment
payment_method	TEXT	NOT NULL	Channel utilized: Credit Card, UPI, Wallet, Adapter
amount	REAL	NOT NULL	Financial transaction currency amount
timestamp	TEXT	NOT NULL	Execution timestamp of transaction authorization
status	TEXT	NOT NULL	Payment ledger status: SUCCESS, FAILED, REFUNDED
reference	TEXT	NOT NULL	Simulated gateway authorization or refund reference code

4.7 AUDIT_LOGS TABLE

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTO	Sequential event sequence identifier
timestamp	TEXT	NOT NULL	Timestamp of security or business operation
action	TEXT	NOT NULL	Detailed audit description of system

			event

Figure 4.1 below illustrates the relational architecture and cardinalities uniting these entities. Customer accounts book shows, shows host independent seat grids linked to auditorium screens and movie titles, and confirmed bookings generate financial transactions within the persistent SQLite ledger.

FIGURE 4.1: ENTITY RELATIONSHIP DIAGRAM

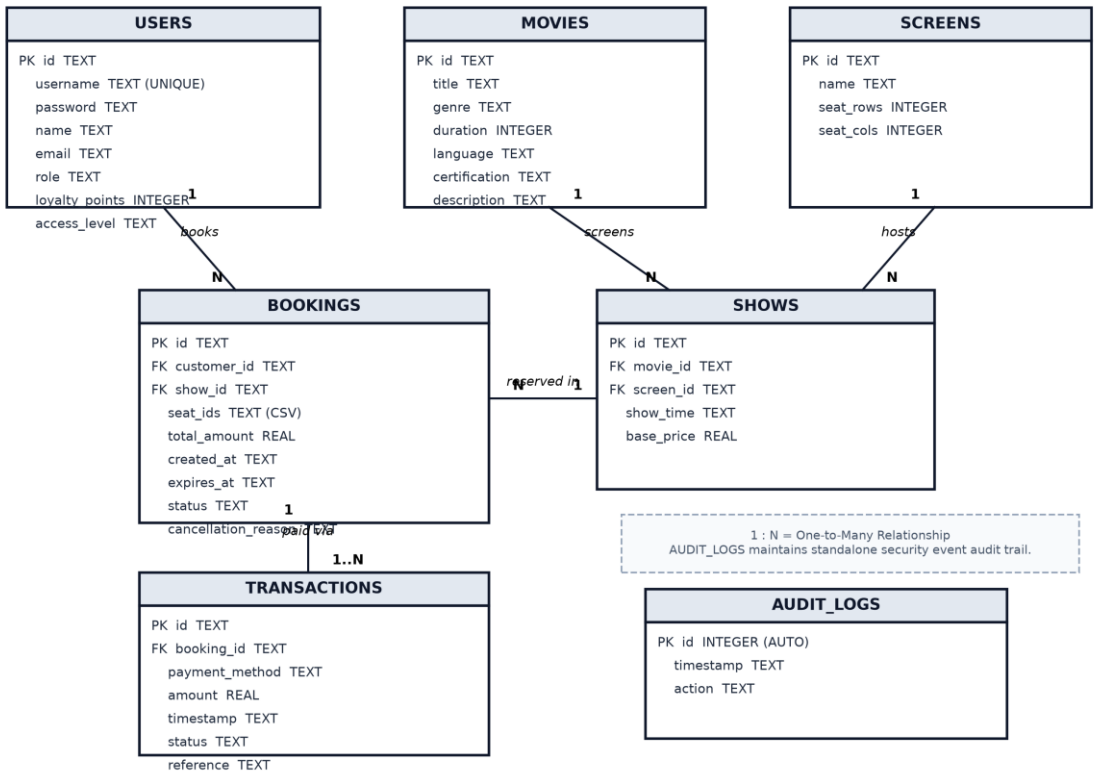


Figure 4.1: Entity relationship diagram of CineBook database schema.

CHAPTER 5

UML CLASS DIAGRAM

The UML class diagrams illustrate the structural organization, inheritance hierarchies, interfaces, and design patterns implemented across the CineBook application. The codebase separates domain state, service controllers, persistence mechanisms, and graphical Swing presentation.

Core Model and Inheritance Hierarchies

The abstract User base class encapsulates shared security attributes (id, username, password, name, email, role) and enforces dynamic runtime polymorphism through the abstract getDashboardGreeting() method. Customer extends User with loyalty points tracking, while AdminUser extends User with administrative privileges. Booking implements the Cancellable interface, which mandates an isCancellable() state predicate (cancellable up to 2 hours before showtime) and a cancel(reason) mutator.

Service, Strategy & Design Patterns

MovieService implements the generic Searchable<Movie> interface for query filtering. PaymentMethod serves as the abstract Strategy base, polymorphically dispatched across MockCardPayment, MockUpiPayment, and MockWalletPayment. The structural Adapter design pattern is implemented via LegacyBankGatewayAdapter, which adapts the incompatible LegacyBankGatewayApi to the system's PaymentGateway interface without modifying legacy code. BookingManager and DatabaseManager are implemented as thread-safe Singletons utilizing volatile instance variables and double-checked locking.

Swing GUI Architecture

MainFrame extends JFrame and manages application routing through a Swing CardLayout. Content views (MoviePanel, MovieDetailsPanel, SeatSelectionPanel, CheckoutPanel, BookingConfirmationPanel, BookingHistoryPanel, and AdminPanel) all extend JPanel. LoginDialog extends JDialog to provide modal authentication on launch.

FIGURE 5.1: UML CLASS DIAGRAM (BACK-END & DOMAIN MODEL)

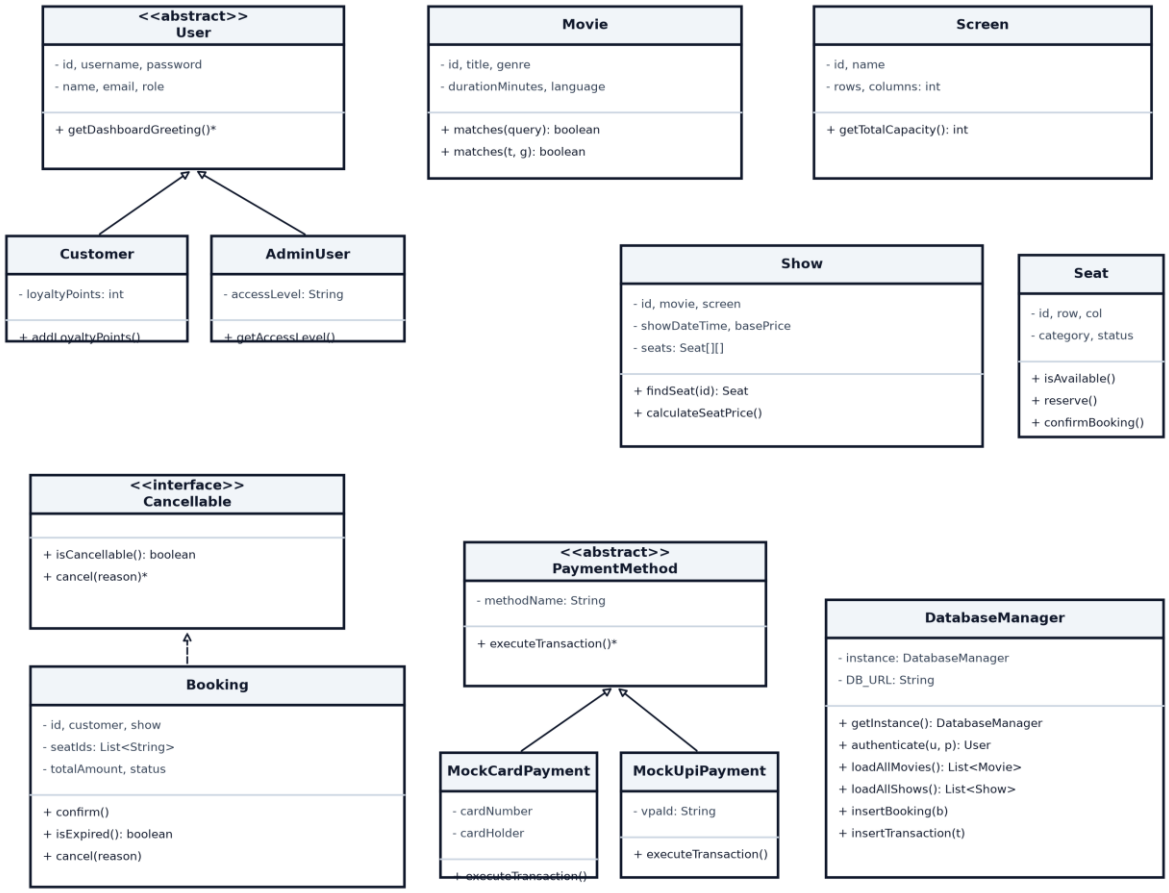


Figure 5.1: UML class diagram of domain model, service, and database classes.

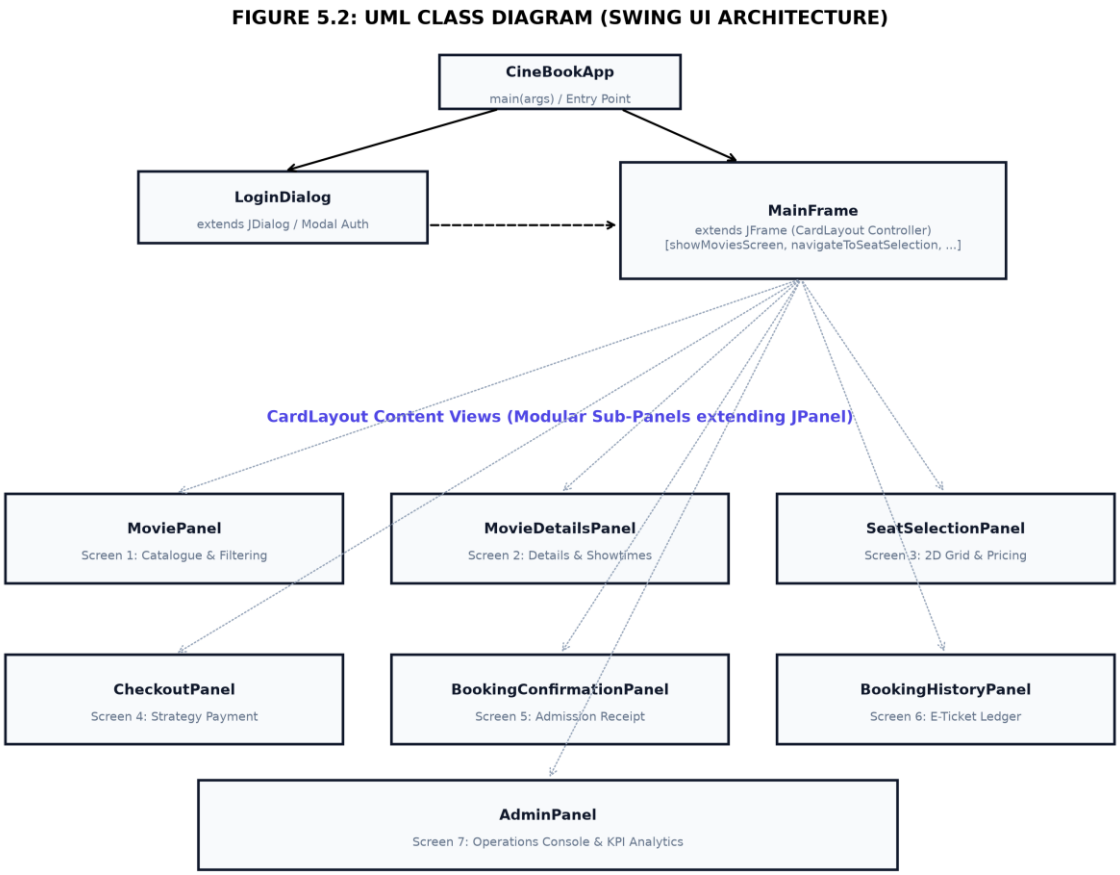


Figure 5.2: UML class diagram of the Swing user interface classes.

Chapter 6

SYSTEM OUTPUT / SCREENSHOTS



Figure 6.1: Account authentication and startup login dialog with credential quick-fill chips.

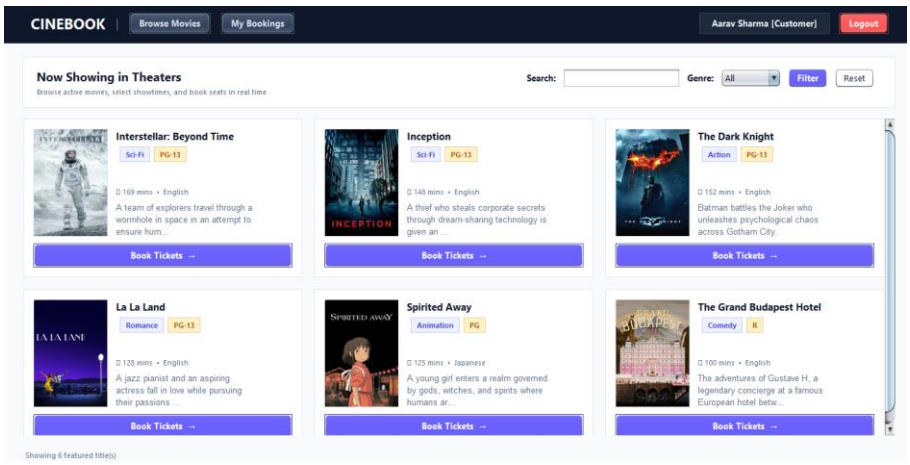


Figure 6.2: Theatrical movie catalogue browsing with live search, genre filter, and high-res posters.

(CONTINUED)

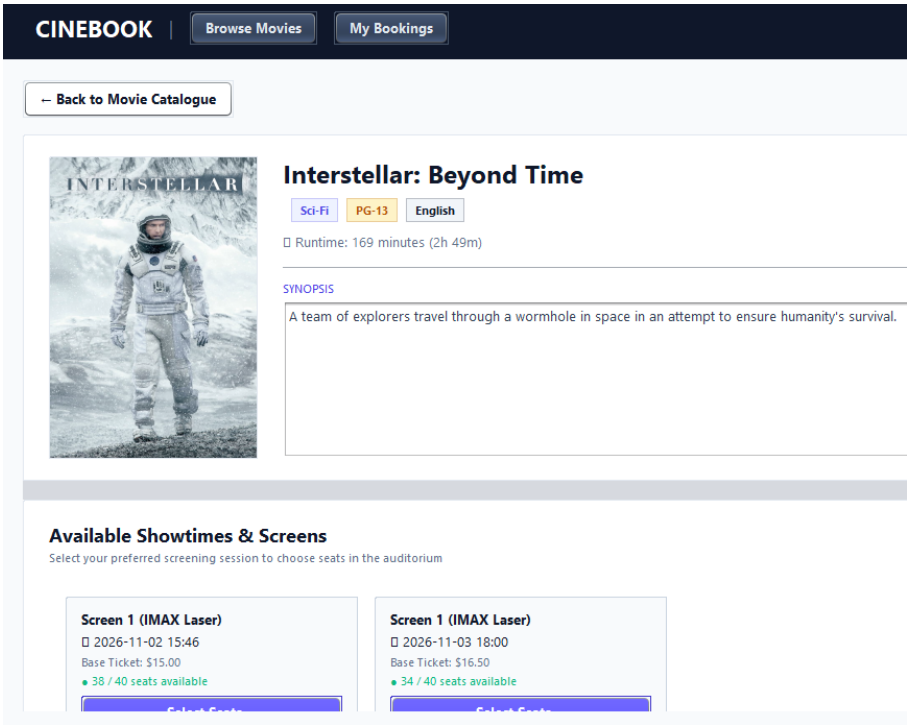


Figure 6.3: Movie presentation details view with synopsis and scheduled auditorium showtime cards.

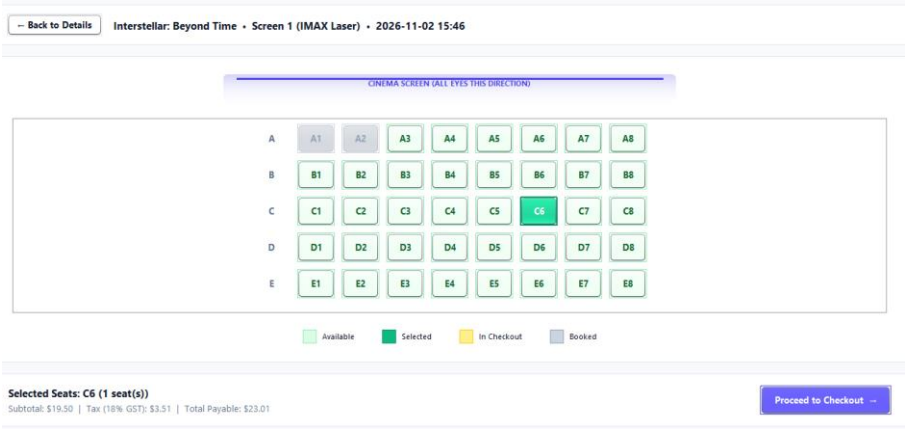


Figure 6.4: Interactive auditorium seating grid featuring glowing cinema screen and tier multiplier badges.

(CONTINUED)

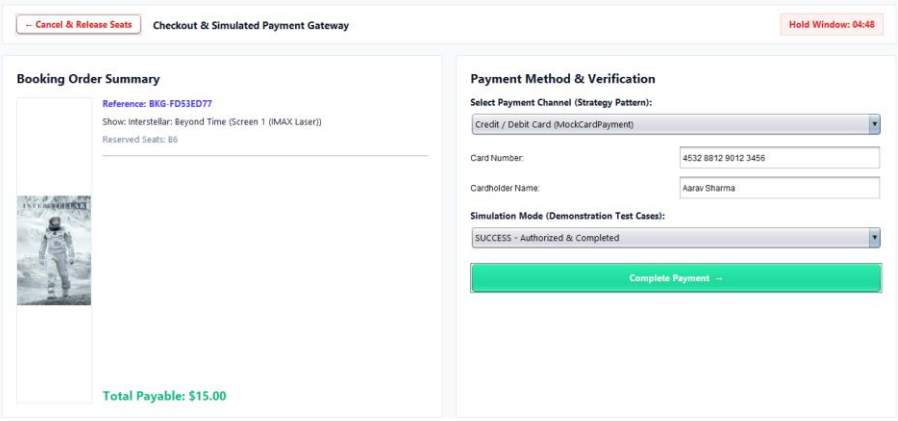


Figure 6.5: Checkout panel with 5-minute countdown hold timer and polymorphic payment simulation.

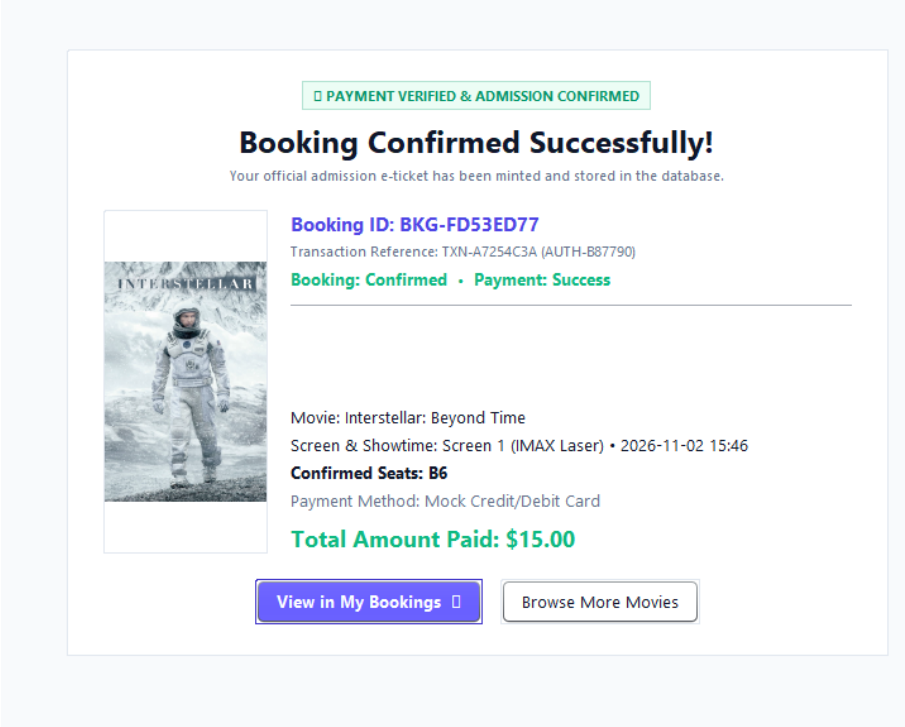


Figure 6.6: Booking confirmation receipt verifying ticket generation and financial authorization.

(CONTINUED)



Figure 6.7: Customer booking history ledger displaying status and cancellation eligibility.

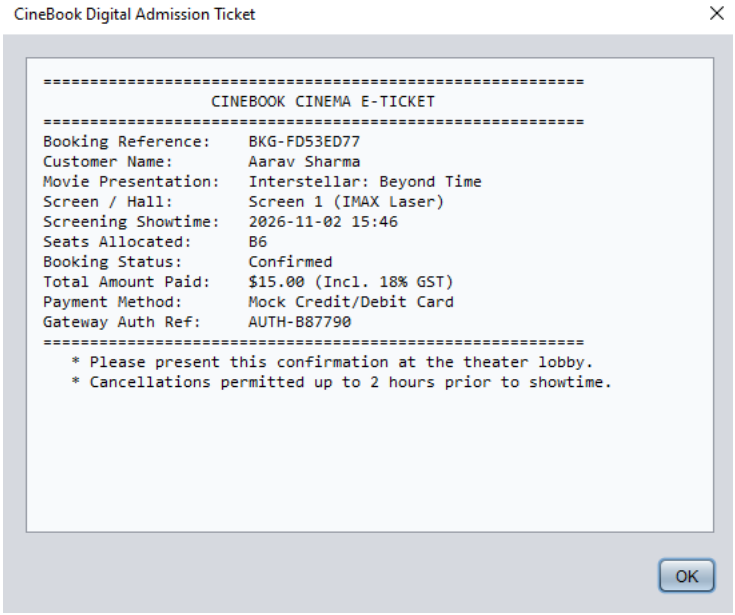


Figure 6.8: Digital cinema e-ticket modal dialog with boarding pass formatting.

(CONTINUED)

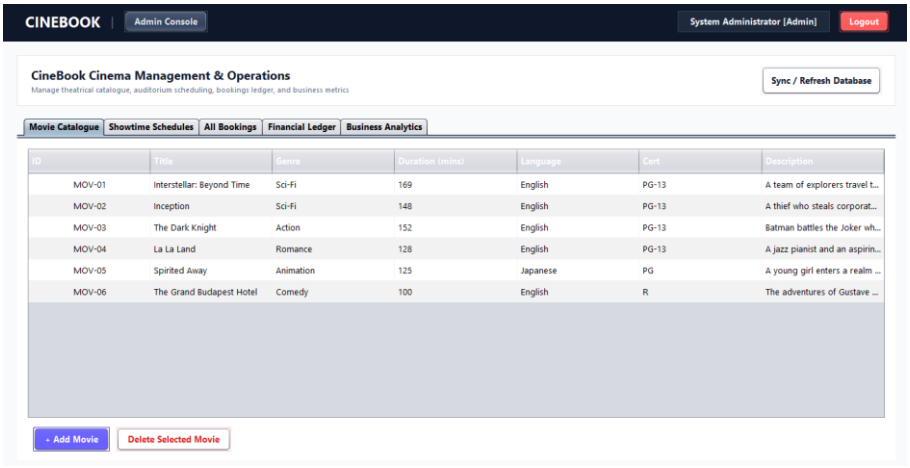


Figure 6.9: Administrator console displaying theatrical catalogue management and scheduling tabs.

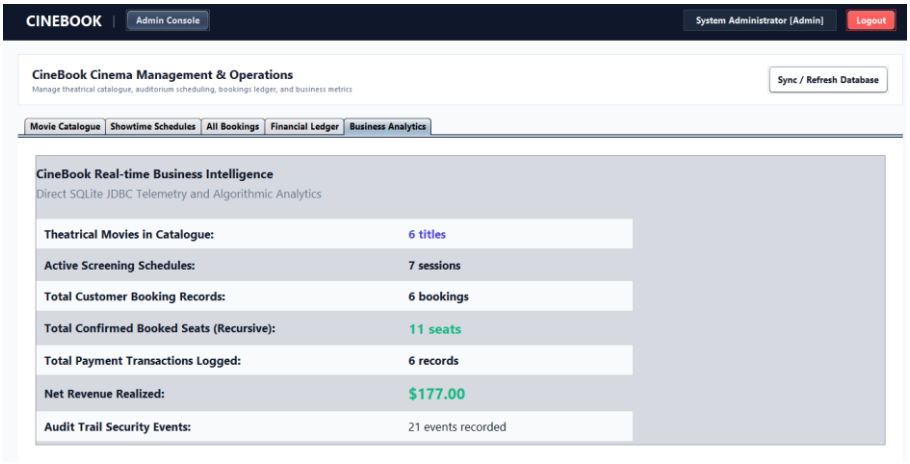


Figure 6.10: Real-time business intelligence dashboard with recursive seat calculation telemetry.

CHAPTER 7
CODE LISTING

Each Java source file of the CineBook project is listed below in full. To optimize readability and pagination for print binding, explanatory inline comments and redundant whitespace have been removed while retaining 100% of the operational logic, data types, method implementations, and architectural structure.

Application Entry Points & Test Suite
CineBookApp.java is the application entry point. It runs an automated 6-step regression verification suite before launching the graphical interface, verifying movie loading, independent seat instances, temporary reservation workflows, double-confirmation prevention, deletion guards, and recursive booked-seat algorithms. MainFrame.java serves as the primary application window coordinating view transitions via CardLayout.

OOP Concepts Implemented in Code
Inheritance: User -> Customer, AdminUser; PaymentMethod -> MockCardPayment, MockUpiPayment, MockWalletPayment. Abstraction: abstract User, abstract PaymentMethod. Interfaces: Cancellable, Searchable<T>, PaymentGateway. Design Patterns: Singleton (BookingManager, DatabaseManager), Strategy (PaymentMethod), Adapter (LegacyBankGatewayAdapter). Custom Exceptions: CineBookException hierarchy. Database: PreparedStatement and ResultSet CRUD operations via JDBC.

File: AdminPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDateTime;

public class AdminPanel extends JPanel {
    private final MainFrame parent;
    private final BookingManager manager;
    private final MovieService movieService;
    private final ShowService showService;

    private DefaultTableModel moviesTableModel;
    private DefaultTableModel showsTableModel;
    private DefaultTableModel bookingsTableModel;
    private DefaultTableModel txnsTableModel;

    private JLabel statsLabel;

    public AdminPanel(MainFrame parent, BookingManager manager, MovieService movieService, ShowService
showService) {
        this.parent = parent;
        this.manager = manager;
        this.movieService = movieService;
```

```
        this.showService = showService;

        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel topBar = new JPanel(new BorderLayout());
        topBar.setBackground(Color.WHITE);
        topBar.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(16, 20, 16, 20)
        ));

        JPanel titleBlock = new JPanel();
        titleBlock.setLayout(new BoxLayout(titleBlock, BoxLayout.Y_AXIS));
        titleBlock.setOpaque(false);

        JLabel title = new JLabel("CineBook Cinema Management & Operations");
        title.setFont(UIConstants.FONT_TITLE);
        title.setForeground(UIConstants.COLOR_TEXT_DARK);

        JLabel subtitle = new JLabel("Manage theatrical catalogue, auditorium scheduling, bookings ledger,
and business metrics");
        subtitle.setFont(UIConstants.FONT_SMALL);
        subtitle.setForeground(UIConstants.COLOR_TEXT_MUTED);

        titleBlock.add(title);
        titleBlock.add(Box.createVerticalStrut(2));
        titleBlock.add(subtitle);

        JButton refreshBtn = new JButton("Sync / Refresh Database");
        refreshBtn.setFont(UIConstants.FONT_BOLD);
        refreshBtn.setBackground(new Color(241, 245, 249));
        refreshBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        refreshBtn.setFocusPainted(false);
        refreshBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        refreshBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        refreshBtn.addActionListener(e -> {
            manager.loadFromDatabase();
            refreshData();
        });

        topBar.add(titleBlock, BorderLayout.WEST);
        topBar.add(refreshBtn, BorderLayout.EAST);
        add(topBar, BorderLayout.NORTH);

        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.setFont(UIConstants.FONT_BOLD);

        tabbedPane.addTab("Movie Catalogue", createMoviesTab());
        tabbedPane.addTab("Showtime Schedules", createShowsTab());
        tabbedPane.addTab("All Bookings", createBookingsTab());
        tabbedPane.addTab("Financial Ledger", createTxnsTab());
        tabbedPane.addTab("Business Analytics", createStatsTab());

        add(tabbedPane, BorderLayout.CENTER);
    }
```

```
    private JPanel createMoviesTab() {
        JPanel panel = new JPanel(new BorderLayout(0, 12));
        panel.setBackground(Color.WHITE);
        panel.setBorder(new EmptyBorder(14, 14, 14, 14));

        String[] cols = {"ID", "Title", "Genre", "Duration (mins)", "Language", "Cert", "Description"};
        moviesTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int row, int col) { return false; }
        };
        JTable table = createStyledTable(moviesTableModel);
        panel.add(new JScrollPane(table), BorderLayout.CENTER);

        JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        actions.setOpaque(false);

        JButton addBtn = new JButton("+ Add Movie");
        addBtn.setFont(UIConstants.FONT_BOLD);
        addBtn.setBackground(UIConstants.COLOR_ACCENT);
        addBtn.setForeground(Color.WHITE);
        addBtn.setFocusPainted(false);
        addBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        addBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        addBtn.addActionListener(e -> promptAddMovie());

        JButton delBtn = new JButton("Delete Selected Movie");
        delBtn.setFont(UIConstants.FONT_BOLD);
        delBtn.setBackground(new Color(254, 242, 242));
        delBtn.setForeground(new Color(220, 38, 38));
        delBtn.setFocusPainted(false);
        delBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        delBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(254, 202, 202), 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        delBtn.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Select a movie to remove.", "Notice",
JOptionPane.INFORMATION_MESSAGE);
                return;
            }
            String mId = (String) moviesTableModel.getValueAt(row, 0);
            try {
                movieService.removeMovie(mId);
                JOptionPane.showMessageDialog(this, "Movie removed successfully from SQLite.", "Success",
JOptionPane.INFORMATION_MESSAGE);
                refreshData();
            } catch (ValidationException ex) {
                JOptionPane.showMessageDialog(this, "Cannot delete movie: " + ex.getMessage(),
"Referential Guard Triggered", JOptionPane.WARNING_MESSAGE);
            }
        });

        actions.add(addBtn);
        actions.add(delBtn);
        panel.add(actions, BorderLayout.SOUTH);
        return panel;
    }

    private void promptAddMovie() {
        JTextField titleField = new JTextField();
        JTextField genreField = new JTextField("Action");
```

```
        JTextField durField = new JTextField("120");
        JTextField langField = new JTextField("English");
        JTextField certField = new JTextField("PG-13");
        JTextArea descArea = new JTextArea(3, 20);

        JPanel form = new JPanel(new GridLayout(6, 2, 6, 8));
        form.add(new JLabel("Title:"));
        form.add(titleField);
        form.add(new JLabel("Genre:"));
        form.add(genreField);
        form.add(new JLabel("Duration (mins):"));
        form.add(durField);
        form.add(new JLabel("Language:"));
        form.add(langField);
        form.add(new JLabel("Certification:"));
        form.add(certField);
        form.add(new JLabel("Description:"));
        form.add(new JScrollPane(descArea));

        int result = JOptionPane.showConfirmDialog(this, form, "Add New Movie Title",
JOptionPane.OK_CANCEL_OPTION);
        if (result == JOptionPane.OK_OPTION) {
            try {
                String id = "MOV-" + String.format("%02d", manager.getMovies().size() + 1);
                int dur = Integer.parseInt(durField.getText().trim());
                Movie m = new Movie(id, titleField.getText().trim(), genreField.getText().trim(),
                        dur, langField.getText().trim(), certField.getText().trim(),
descArea.getText().trim());
                movieService.addMovie(m);
                JOptionPane.showMessageDialog(this, "Movie successfully added and saved to database.",
"Success", JOptionPane.INFORMATION_MESSAGE);
                refreshData();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Error adding movie: " + ex.getMessage(), "Input
Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private JPanel createShowsTab() {
        JPanel panel = new JPanel(new BorderLayout(0, 12));
        panel.setBackground(Color.WHITE);
        panel.setBorder(new EmptyBorder(14, 14, 14, 14));

        String[] cols = {"Show ID", "Movie", "Auditorium Screen", "Screening Time", "Base Price", "Seat
Occupancy"};
        showsTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int row, int col) { return false; }
        };
        JTable table = createStyledTable(showsTableModel);
        panel.add(new JScrollPane(table), BorderLayout.CENTER);

        JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        actions.setOpaque(false);

        JButton addBtn = new JButton("+ Schedule Show");
        addBtn.setFont(UIConstants.FONT_BOLD);
        addBtn.setBackground(UIConstants.COLOR_ACCENT);
        addBtn.setForeground(Color.WHITE);
        addBtn.setFocusPainted(false);
        addBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        addBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
```

```
        addBtn.addActionListener(e -> promptAddShow());

        JButton delBtn = new JButton("Remove Selected Show");
        delBtn.setFont(UIConstants.FONT_BOLD);
        delBtn.setBackground(new Color(254, 242, 242));
        delBtn.setForeground(new Color(220, 38, 38));
        delBtn.setFocusPainted(false);
        delBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        delBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(254, 202, 202), 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        delBtn.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Select a show first.", "Notice",
JOptionPane.INFORMATION_MESSAGE);
                return;
            }
            String sId = (String) showsTableModel.getValueAt(row, 0);
            try {
                showService.removeShow(sId);
                JOptionPane.showMessageDialog(this, "Show deleted successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
                refreshData();
            } catch (ValidationException ex) {
                JOptionPane.showMessageDialog(this, "Cannot delete show: " + ex.getMessage(), "Active
Bookings Exist", JOptionPane.WARNING_MESSAGE);
            }
        });

        actions.add(addBtn);
        actions.add(delBtn);
        panel.add(actions, BorderLayout.SOUTH);
        return panel;
    }

    private void promptAddShow() {
        if (manager.getMovies().isEmpty() || manager.getScreens().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Ensure movies and screens exist before scheduling a
show.", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }

        JComboBox<Movie> movieCombo = new JComboBox<>(manager.getMovies().toArray(new Movie[0]));
        JComboBox<Screen> screenCombo = new JComboBox<>(manager.getScreens().toArray(new Screen[0]));
        JTextField hoursOffsetField = new JTextField("4");
        JTextField priceField = new JTextField("16.00");

        JPanel form = new JPanel(new GridLayout(4, 2, 6, 8));
        form.add(new JLabel("Select Movie:"));
        form.add(movieCombo);
        form.add(new JLabel("Select Auditorium:"));
        form.add(screenCombo);
        form.add(new JLabel("Hours from now:"));
        form.add(hoursOffsetField);
        form.add(new JLabel("Base Price ($):"));
        form.add(priceField);

        int result = JOptionPane.showConfirmDialog(this, form, "Schedule New Showtime",
JOptionPane.OK_CANCEL_OPTION);
        if (result == JOptionPane.OK_OPTION) {
            try {
                Movie m = (Movie) movieCombo.getSelectedItem();
                Screen sc = (Screen) screenCombo.getSelectedItem();
                int hours = Integer.parseInt(hoursOffsetField.getText().trim());
```

```
                double price = Double.parseDouble(priceField.getText().trim());

                String sId = "SHW-" + String.format("%02d", manager.getShows().size() + 1);
                Show newShow = new Show(sId, m, sc, LocalDateTime.now().plusHours(hours), price);
                showService.addShow(newShow);
                JOptionPane.showMessageDialog(this, "Show scheduled successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
                refreshData();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Error scheduling show: " + ex.getMessage(), "Input
Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private JPanel createBookingsTab() {
        JPanel panel = new JPanel(new BorderLayout());
        panel.setBackground(Color.WHITE);
        panel.setBorder(new EmptyBorder(14, 14, 14, 14));

        String[] cols = {"Booking ID", "Customer Name", "Movie", "Auditorium", "Showtime", "Seats", "Total
Paid", "Status"};
        bookingsTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int row, int col) { return false; }
        };
        JTable table = createStyledTable(bookingsTableModel);
        panel.add(new JScrollPane(table), BorderLayout.CENTER);
        return panel;
    }

    private JPanel createTxnsTab() {
        JPanel panel = new JPanel(new BorderLayout());
        panel.setBackground(Color.WHITE);
        panel.setBorder(new EmptyBorder(14, 14, 14, 14));

        String[] cols = {"Transaction ID", "Booking Ref", "Payment Channel", "Amount Paid", "Status",
"Gateway Ref", "Timestamp"};
        txnsTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int row, int col) { return false; }
        };
        JTable table = createStyledTable(txnsTableModel);
        panel.add(new JScrollPane(table), BorderLayout.CENTER);
        return panel;
    }

    private JPanel createStatsTab() {
        JPanel panel = new JPanel(new BorderLayout());
        panel.setBackground(Color.WHITE);
        panel.setBorder(new EmptyBorder(24, 24, 24, 24));

        statsLabel = new JLabel();
        statsLabel.setFont(UIConstants.FONT_REGULAR);
        panel.add(new JScrollPane(statsLabel), BorderLayout.CENTER);
        return panel;
    }

    private JTable createStyledTable(DefaultTableModel model) {
        JTable table = new JTable(model);
        table.setFont(UIConstants.FONT_REGULAR);
        table.setRowHeight(32);
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setSelectionBackground(new Color(238, 242, 255));
        table.setSelectionForeground(UIConstants.COLOR_TEXT_DARK);
        table.setShowGrid(true);
```

```
        table.setGridColor(new Color(241, 245, 249));

        table.getTableHeader().setFont(UIConstants.FONT_BOLD);
        table.getTableHeader().setBackground(UIConstants.COLOR_PRIMARY);
        table.getTableHeader().setForeground(Color.WHITE);
        table.getTableHeader().setPreferredSize(new Dimension(0, 36));

        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(SwingConstants.CENTER);
        table.getColumnModel().getColumn(0).setCellRenderer(centerRenderer);

        return table;
    }

    public void refreshData() {

        moviesTableModel.setRowCount(0);
        for (Movie m : manager.getMovies()) {
            moviesTableModel.addRow(new Object[]{
                    m.getId(), m.getTitle(), m.getGenre(), m.getDurationMinutes(),
                    m.getLanguage(), m.getCertification(), m.getDescription()
            });
        }

        showsTableModel.setRowCount(0);
        for (Show s : manager.getShows()) {
            showsTableModel.addRow(new Object[]{
                    s.getId(), s.getMovie().getTitle(), s.getScreen().getName(),
                    s.getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER),
                    String.format("$%.2f", s.getBasePrice()),
                    s.getAvailableSeatsCount() + " / " + s.getTotalSeatsCount()
            });
        }

        bookingsTableModel.setRowCount(0);
        for (Booking b : manager.getBookings()) {
            bookingsTableModel.addRow(new Object[]{
                    b.getId(), b.getCustomer().getName(), b.getShow().getMovie().getTitle(),
                    b.getShow().getScreen().getName(),
                    b.getShow().getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER),
                    String.join(", ", b.getSeatIds()),
                    String.format("$%.2f", b.getTotalAmount()),
                    b.getBookingStatus().getDisplayName()
            });
        }

        txnsTableModel.setRowCount(0);
        for (PaymentTransaction t : manager.getTransactions()) {
            txnsTableModel.addRow(new Object[]{
                    t.getId(), t.getBookingId(), t.getPaymentMethodName(),
                    String.format("$%.2f", t.getAmount()),
                    t.getPaymentStatus().getDisplayName(),
                    t.getGatewayReference(),
                    t.getTimestamp().format(UIConstants.DATE_TIME_FORMATTER)
            });
        }

        int totalBookedSeats = manager.countTotalBookedSeatsRecursive(manager.getBookings(), 0);
        double revenue = manager.calculateTotalRevenue();

        String statsHtml = String.format(
                "<html><body style='font-family: Segoe UI, sans-serif; font-size: 13px; color: #0F172A;'>"
+
                "<h2 style='color: #0F172A; margin-bottom: 4px;'>CineBook Real-time Business
Intelligence</h2>" +
                "<p style='color: #64748B; margin-top: 0px;'>Direct SQLite JDBC Telemetry and Algorithmic
```

```
Analytics</p>" +
                "<br/>" +
                "<table cellpadding='10' style='border-collapse: collapse; width: 680px;'>" +
                "<tr style='background-color: #F8FAFC;'><td><b>Theatrical Movies in
Catalogue:</b></td><td><b style='color: #4F46E5;'>%d titles</b></td></tr>" +
                "<tr><td><b>Active Screening Schedules:</b></td><td><b>%d sessions</b></td></tr>" +
                "<tr style='background-color: #F8FAFC;'><td><b>Total Customer Booking
Records:</b></td><td><b>%d bookings</b></td></tr>" +
                "<tr><td><b>Total Confirmed Booked Seats (Recursive):</b></td><td><b style='color:
#10B981; font-size: 15px;'>%d seats</b></td></tr>" +
                "<tr style='background-color: #F8FAFC;'><td><b>Total Payment Transactions
Logged:</b></td><td><b>%d records</b></td></tr>" +
                "<tr><td><b>Net Revenue Realized:</b></td><td><b style='color: #10B981; font-size:
17px;'>$%.2f</b></td></tr>" +
                "<tr style='background-color: #F8FAFC;'><td><b>Audit Trail Security Events:</b></td><td>%d
events recorded</td></tr>" +
                "</table>" +
                "</body></html>",
                manager.getMovies().size(),
                manager.getShows().size(),
                manager.getBookings().size(),
                totalBookedSeats,
                manager.getTransactions().size(),
                revenue,
                manager.getAuditLogs().size()
        );

        statsLabel.setText(statsHtml);
    }
}
```

File: AdminUser.java | Package: (default)

```
public class AdminUser extends User {
    private final String accessLevel;

    public AdminUser(String id, String username, String password, String name, String email, String
accessLevel) {
        super(id, username, password, name, email, "Admin");
        this.accessLevel = accessLevel;
    }

    public AdminUser(String id, String name, String email, String accessLevel) {
        super(id, name, email, "Admin");
        this.accessLevel = accessLevel;
    }

    public String getAccessLevel() { return accessLevel; }

    @Override
    public String getDashboardGreeting() {
        return "CineBook Cinema Management Console - Logged in as " + getName() + " [" + accessLevel +
"]";
    }
}
```

File: Booking.java | Package: (default)

```
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Booking implements Cancellable {
    private final String id;
    private final Customer customer;
    private final Show show;
```

```
    private final List<String> seatIds;
    private final double totalAmount;
    private final LocalDateTime createdAt;
    private final LocalDateTime expiresAt;
    private BookingStatus bookingStatus;
    private String cancellationReason;

    public Booking(String id, Customer customer, Show show, List<String> seatIds, double totalAmount, int
expiryMinutes) {
        this.id = id;
        this.customer = customer;
        this.show = show;
        this.seatIds = new ArrayList<>(seatIds);
        this.totalAmount = Math.round(totalAmount * 100.0) / 100.0;
        this.createdAt = LocalDateTime.now();
        this.expiresAt = LocalDateTime.now().plusMinutes(expiryMinutes);
        this.bookingStatus = BookingStatus.PENDING_PAYMENT;
    }

    public String getId() { return id; }
    public Customer getCustomer() { return customer; }
    public Show getShow() { return show; }
    public List<String> getSeatIds() { return Collections.unmodifiableList(seatIds); }
    public double getTotalAmount() { return totalAmount; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getExpiresAt() { return expiresAt; }
    public BookingStatus getBookingStatus() { return bookingStatus; }
    public String getCancellationReason() { return cancellationReason; }

    public boolean isExpired() {
        return bookingStatus == BookingStatus.PENDING_PAYMENT && LocalDateTime.now().isAfter(expiresAt);
    }

    public void confirm() throws InvalidBookingStateException {
        if (bookingStatus == BookingStatus.CONFIRMED) {
            throw new InvalidBookingStateException("Booking " + id + " is already confirmed.");
        }
        if (bookingStatus == BookingStatus.CANCELLED) {
            throw new InvalidBookingStateException("Cannot confirm a cancelled booking.");
        }
        this.bookingStatus = BookingStatus.CONFIRMED;
    }

    public void markPaymentFailed() {
        if (this.bookingStatus == BookingStatus.PENDING_PAYMENT) {
            this.bookingStatus = BookingStatus.PAYMENT_FAILED;
        }
    }

    @Override
    public boolean isCancellable() {
        if (bookingStatus != BookingStatus.CONFIRMED) return false;

        return LocalDateTime.now().isBefore(show.getShowDateTime().minusHours(2));
    }

    @Override
    public void cancel(String reason) throws InvalidBookingStateException {
        if (bookingStatus == BookingStatus.CANCELLED) {
            throw new InvalidBookingStateException("Booking " + id + " is already cancelled.");
        }
        this.bookingStatus = BookingStatus.CANCELLED;
        this.cancellationReason = reason;
    }

    @Override
```

```
    public String toString() {
        return "Booking #" + id + " [" + bookingStatus.getDisplayName() + "] - " +
                show.getMovie().getTitle() + " (" + String.join(", ", seatIds) + ") $" +
String.format("%.2f", totalAmount);
    }
}
```

File: BookingConfirmationPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;

public class BookingConfirmationPanel extends JPanel {
    private final MainFrame parent;

    private JLabel posterLabel;
    private JLabel bookingIdLabel;
    private JLabel txnIdLabel;
    private JLabel movieLabel;
    private JLabel showtimeLabel;
    private JLabel seatsLabel;
    private JLabel amountLabel;
    private JLabel paymentMethodLabel;
    private JLabel statusBadge;

    public BookingConfirmationPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout());
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(24, 28, 24, 28));

        initializeUI();
    }

    private void initializeUI() {
        JPanel card = new JPanel(new BorderLayout(20, 20));
        card.setBackground(Color.WHITE);
        card.setPreferredSize(new Dimension(650, 480));
        card.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(24, 28, 24, 28)
        ));

        JPanel headerPanel = new JPanel();
        headerPanel.setLayout(new BoxLayout(headerPanel, BoxLayout.Y_AXIS));
        headerPanel.setOpaque(false);

        JLabel successPill = UIConstants.createBadge("✓ PAYMENT VERIFIED & ADMISSION CONFIRMED", new
Color(236, 253, 245), new Color(5, 150, 105));
        successPill.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel title = new JLabel("Booking Confirmed Successfully!");
        title.setFont(UIConstants.FONT_HERO);
        title.setForeground(UIConstants.COLOR_TEXT_DARK);
        title.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel subtitle = new JLabel("Your official admission e-ticket has been minted and stored in the
database.");
        subtitle.setFont(UIConstants.FONT_SMALL);
        subtitle.setForeground(UIConstants.COLOR_TEXT_MUTED);
        subtitle.setAlignmentX(Component.CENTER_ALIGNMENT);

        headerPanel.add(successPill);
        headerPanel.add(Box.createVerticalStrut(8));
        headerPanel.add(title);
```

```
        headerPanel.add(Box.createVerticalStrut(4));
        headerPanel.add(subtitle);

        card.add(headerPanel, BorderLayout.NORTH);

        JPanel bodyPanel = new JPanel(new BorderLayout(18, 0));
        bodyPanel.setOpaque(false);

        posterLabel = new JLabel();
        posterLabel.setPreferredSize(new Dimension(130, 190));
        posterLabel.setBorder(BorderFactory.createLineBorder(new Color(226, 232, 240), 1, true));
        bodyPanel.add(posterLabel, BorderLayout.WEST);

        JPanel detailsPanel = new JPanel();
        detailsPanel.setLayout(new BoxLayout(detailsPanel, BoxLayout.Y_AXIS));
        detailsPanel.setOpaque(false);

        bookingIdLabel = new JLabel("Booking ID: ");
        bookingIdLabel.setFont(UIConstants.FONT_HEADING);
        bookingIdLabel.setForeground(UIConstants.COLOR_ACCENT);

        txnIdLabel = new JLabel("Transaction ID: ");
        txnIdLabel.setFont(UIConstants.FONT_SMALL);
        txnIdLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);

        statusBadge = new JLabel("Status: CONFIRMED");
        statusBadge.setFont(UIConstants.FONT_BOLD);
        statusBadge.setForeground(UIConstants.COLOR_SUCCESS);

        movieLabel = new JLabel("Movie: ");
        movieLabel.setFont(UIConstants.FONT_REGULAR);
        movieLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        showtimeLabel = new JLabel("Showtime: ");
        showtimeLabel.setFont(UIConstants.FONT_REGULAR);
        showtimeLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        seatsLabel = new JLabel("Seats: ");
        seatsLabel.setFont(UIConstants.FONT_BOLD);
        seatsLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        paymentMethodLabel = new JLabel("Method: ");
        paymentMethodLabel.setFont(UIConstants.FONT_REGULAR);
        paymentMethodLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);

        amountLabel = new JLabel("Amount Paid: ");
        amountLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        amountLabel.setForeground(UIConstants.COLOR_SUCCESS);

        detailsPanel.add(bookingIdLabel);
        detailsPanel.add(Box.createVerticalStrut(4));
        detailsPanel.add(txnIdLabel);
        detailsPanel.add(Box.createVerticalStrut(6));
        detailsPanel.add(statusBadge);
        detailsPanel.add(Box.createVerticalStrut(10));
        detailsPanel.add(new JSeparator());
        detailsPanel.add(Box.createVerticalStrut(10));
        detailsPanel.add(movieLabel);
        detailsPanel.add(Box.createVerticalStrut(4));
        detailsPanel.add(showtimeLabel);
        detailsPanel.add(Box.createVerticalStrut(4));
        detailsPanel.add(seatsLabel);
        detailsPanel.add(Box.createVerticalStrut(4));
        detailsPanel.add(paymentMethodLabel);
        detailsPanel.add(Box.createVerticalStrut(8));
        detailsPanel.add(amountLabel);
```

```
        bodyPanel.add(detailsPanel, BorderLayout.CENTER);
        card.add(bodyPanel, BorderLayout.CENTER);

        JPanel actionsPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 14, 0));
        actionsPanel.setOpaque(false);

        JButton myBookingsBtn = new JButton("View in My Bookings  📄");
        myBookingsBtn.setFont(UIConstants.FONT_BOLD);
        myBookingsBtn.setBackground(UIConstants.COLOR_ACCENT);
        myBookingsBtn.setForeground(Color.WHITE);
        myBookingsBtn.setFocusPainted(false);
        myBookingsBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        myBookingsBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        myBookingsBtn.addActionListener(e -> parent.showHistoryScreen());

        JButton backMoviesBtn = new JButton("Browse More Movies");
        backMoviesBtn.setFont(UIConstants.FONT_REGULAR);
        backMoviesBtn.setBackground(new Color(241, 245, 249));
        backMoviesBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        backMoviesBtn.setFocusPainted(false);
        backMoviesBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        backMoviesBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        backMoviesBtn.addActionListener(e -> parent.showMoviesScreen());

        actionsPanel.add(myBookingsBtn);
        actionsPanel.add(backMoviesBtn);
        card.add(actionsPanel, BorderLayout.SOUTH);

        JPanel centerWrap = new JPanel(new GridBagLayout());
        centerWrap.setOpaque(false);
        centerWrap.add(card);
        add(centerWrap, BorderLayout.CENTER);
    }

    public void displayConfirmation(Booking booking, PaymentTransaction txn) {
        Movie movie = booking.getShow().getMovie();
        posterLabel.setIcon(UIConstants.getMoviePoster(movie.getId(), 130, 190));

        bookingIdLabel.setText("Booking ID: " + booking.getId());
        txnIdLabel.setText("Transaction Reference: " + txn.getId() + " (" + txn.getGatewayReference() +
")");
        movieLabel.setText("Movie: " + movie.getTitle());
        showtimeLabel.setText("Screen & Showtime: " + booking.getShow().getScreen().getName() + " • " +
                booking.getShow().getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER));
        seatsLabel.setText("Confirmed Seats: " + String.join(", ", booking.getSeatIds()));
        paymentMethodLabel.setText("Payment Method: " + txn.getPaymentMethodName());
        amountLabel.setText(String.format("Total Amount Paid: $%.2f", txn.getAmount()));
        statusBadge.setText("Booking: " + booking.getBookingStatus().getDisplayName() +
                "  •  Payment: " + txn.getPaymentStatus().getDisplayName());
    }
}
```

File: BookingHistoryPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
```

```
import java.util.List;

public class BookingHistoryPanel extends JPanel {
    private final MainFrame parent;
    private final BookingManager manager;
    private final BookingService bookingService;

    private DefaultTableModel tableModel;
    private JTable bookingsTable;
    private JLabel customerStatusLabel;

    public BookingHistoryPanel(MainFrame parent, BookingManager manager, BookingService bookingService) {
        this.parent = parent;
        this.manager = manager;
        this.bookingService = bookingService;

        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel topCard = new JPanel(new BorderLayout());
        topCard.setBackground(Color.WHITE);
        topCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(16, 20, 16, 20)
        ));

        customerStatusLabel = new JLabel("My Booking Records");
        customerStatusLabel.setFont(UIConstants.FONT_TITLE);
        customerStatusLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        JButton refreshBtn = new JButton("Refresh");
        refreshBtn.setFont(UIConstants.FONT_BOLD);
        refreshBtn.setBackground(new Color(241, 245, 249));
        refreshBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        refreshBtn.setFocusPainted(false);
        refreshBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        refreshBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        refreshBtn.addActionListener(e -> refreshHistory());

        topCard.add(customerStatusLabel, BorderLayout.WEST);
        topCard.add(refreshBtn, BorderLayout.EAST);
        add(topCard, BorderLayout.NORTH);

        String[] columns = {"Booking ID", "Movie Title", "Auditorium", "Showtime", "Seats", "Total Paid",
"Status", "Cancellation"};
        tableModel = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int col) { return false; }
        };

        bookingsTable = new JTable(tableModel);
        bookingsTable.setFont(UIConstants.FONT_REGULAR);
        bookingsTable.setRowHeight(34);
        bookingsTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        bookingsTable.setSelectionBackground(new Color(238, 242, 255));
        bookingsTable.setSelectionForeground(UIConstants.COLOR_TEXT_DARK);
        bookingsTable.setShowGrid(true);
```

```
        bookingsTable.setGridColor(new Color(241, 245, 249));

        bookingsTable.getTableHeader().setFont(UIConstants.FONT_BOLD);
        bookingsTable.getTableHeader().setBackground(UIConstants.COLOR_PRIMARY);
        bookingsTable.getTableHeader().setForeground(Color.WHITE);
        bookingsTable.getTableHeader().setPreferredSize(new Dimension(0, 36));

        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(SwingConstants.CENTER);
        bookingsTable.getColumnModel().getColumn(0).setCellRenderer(centerRenderer);
        bookingsTable.getColumnModel().getColumn(4).setCellRenderer(centerRenderer);
        bookingsTable.getColumnModel().getColumn(5).setCellRenderer(centerRenderer);
        bookingsTable.getColumnModel().getColumn(6).setCellRenderer(centerRenderer);
        bookingsTable.getColumnModel().getColumn(7).setCellRenderer(centerRenderer);

        JScrollPane scrollPane = new JScrollPane(bookingsTable);
        scrollPane.setBorder(new LineBorder(UIConstants.COLOR_BORDER, 1, true));
        scrollPane.getViewport().setBackground(Color.WHITE);
        add(scrollPane, BorderLayout.CENTER);

        JPanel bottomCard = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 12));
        bottomCard.setBackground(Color.WHITE);
        bottomCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(10, 16, 10, 16)
        ));

        JButton viewTicketBtn = new JButton("View E-Ticket  📄");
        viewTicketBtn.setFont(UIConstants.FONT_BOLD);
        viewTicketBtn.setBackground(UIConstants.COLOR_ACCENT);
        viewTicketBtn.setForeground(Color.WHITE);
        viewTicketBtn.setFocusPainted(false);
        viewTicketBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        viewTicketBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        viewTicketBtn.addActionListener(e -> viewSelectedTicket());

        JButton cancelBookingBtn = new JButton("Cancel Selected Booking");
        cancelBookingBtn.setFont(UIConstants.FONT_BOLD);
        cancelBookingBtn.setBackground(new Color(254, 242, 242));
        cancelBookingBtn.setForeground(new Color(220, 38, 38));
        cancelBookingBtn.setFocusPainted(false);
        cancelBookingBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        cancelBookingBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(254, 202, 202), 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        cancelBookingBtn.addActionListener(e -> cancelSelectedBooking());

        bottomCard.add(viewTicketBtn);
        bottomCard.add(cancelBookingBtn);
        add(bottomCard, BorderLayout.SOUTH);
    }

    public void refreshHistory() {
        tableModel.setRowCount(0);
        User user = manager.getCurrentUser();
        if (user == null) {
            customerStatusLabel.setText("Please log in to view booking records.");
            return;
        }

        List<Booking> userBookings;
        if ("Admin".equalsIgnoreCase(user.getRole())) {
```

```
            customerStatusLabel.setText("All Cinema Bookings (Administrator View - " + user.getName() +
")");
            userBookings = manager.getBookings();
        } else {
            customerStatusLabel.setText("Bookings for: " + user.getName() + " (" + user.getRole() + ")");
            userBookings = manager.getBookingsForCustomer(user.getId());
        }

        for (Booking b : userBookings) {
            String canCancelStr = b.isCancellable() ? "Eligible" : "Non-Cancellable";
            tableModel.addRow(new Object[]{
                    b.getId(),
                    b.getShow().getMovie().getTitle(),
                    b.getShow().getScreen().getName(),
                    b.getShow().getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER),
                    String.join(", ", b.getSeatIds()),
                    String.format("$%.2f", b.getTotalAmount()),
                    b.getBookingStatus().getDisplayName(),
                    canCancelStr
            });
        }
    }

    private void viewSelectedTicket() {
        int row = bookingsTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a booking from the table to view the e-
ticket.", "Selection Required", JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        String bookingId = (String) tableModel.getValueAt(row, 0);
        Booking booking = manager.findBookingById(bookingId);
        PaymentTransaction txn = manager.findTransactionByBookingId(bookingId);

        if (booking != null) {
            String details = String.format(
                    "============================================================\n" +
                    "                  CINEBOOK CINEMA E-TICKET                  \n" +
                    "============================================================\n" +
                    "Booking Reference:    %s\n" +
                    "Customer Name:        %s\n" +
                    "Movie Presentation:   %s\n" +
                    "Screen / Hall:        %s\n" +
                    "Screening Showtime:   %s\n" +
                    "Seats Allocated:      %s\n" +
                    "Booking Status:       %s\n" +
                    "Total Amount Paid:    $%.2f (Incl. 18%% GST)\n" +
                    "Payment Method:       %s\n" +
                    "Gateway Auth Ref:     %s\n" +
                    "============================================================\n" +
                    "   * Please present this confirmation at the theater lobby.\n" +
                    "   * Cancellations permitted up to 2 hours prior to showtime.\n",
                    booking.getId(),
                    booking.getCustomer().getName(),
                    booking.getShow().getMovie().getTitle(),
                    booking.getShow().getScreen().getName(),
                    booking.getShow().getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER),
                    String.join(", ", booking.getSeatIds()),
                    booking.getBookingStatus().getDisplayName(),
                    booking.getTotalAmount(),
                    (txn != null ? txn.getPaymentMethodName() : "Simulated Checkout"),
                    (txn != null ? txn.getGatewayReference() : "OFFLINE-RECORD")
            );

            JTextArea area = new JTextArea(details);
```

```
            area.setFont(new Font("Consolas", Font.PLAIN, 12));
            area.setEditable(false);
            area.setBackground(new Color(248, 250, 252));
            area.setBorder(new EmptyBorder(12, 12, 12, 12));

            JScrollPane scroll = new JScrollPane(area);
            scroll.setPreferredSize(new Dimension(520, 360));
            JOptionPane.showMessageDialog(this, scroll, "CineBook Digital Admission Ticket",
JOptionPane.PLAIN_MESSAGE);
        }
    }

    private void cancelSelectedBooking() {
        int row = bookingsTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a booking to cancel.", "Notice",
JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        String bookingId = (String) tableModel.getValueAt(row, 0);

        int confirm = JOptionPane.showConfirmDialog(
                this,
                "Are you sure you want to cancel booking #" + bookingId + "?\nSeats will be immediately
released back to available status.",
                "Confirm Cancellation",
                JOptionPane.YES_NO_OPTION,
                JOptionPane.WARNING_MESSAGE
        );

        if (confirm == JOptionPane.YES_OPTION) {
            try {
                bookingService.cancelConfirmedBooking(bookingId);
                JOptionPane.showMessageDialog(this, "Booking " + bookingId + " cancelled successfully.
Seats released.",
                        "Cancellation Success", JOptionPane.INFORMATION_MESSAGE);
                refreshHistory();
            } catch (BookingNotFoundException | InvalidBookingStateException ex) {
                JOptionPane.showMessageDialog(this, "Cannot cancel booking: " + ex.getMessage(),
"Cancellation Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }
}
```

File: BookingManager.java | Package: (default)

```
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class BookingManager {
    private static volatile BookingManager instance;

    private final DatabaseManager dbManager;

    private final List<Movie> movies = new CopyOnWriteArrayList<>();
    private final List<Screen> screens = new CopyOnWriteArrayList<>();
    private final List<Show> shows = new CopyOnWriteArrayList<>();
    private final List<Booking> bookings = new CopyOnWriteArrayList<>();
    private final List<PaymentTransaction> transactions = new CopyOnWriteArrayList<>();
    private final List<Customer> customers = new CopyOnWriteArrayList<>();
    private final List<AdminUser> adminUsers = new CopyOnWriteArrayList<>();
    private final List<String> auditLogs = new CopyOnWriteArrayList<>();
```

```
    private User currentUser = null;

    private BookingManager() {
        this.dbManager = DatabaseManager.getInstance();
        loadFromDatabase();
    }

    public static BookingManager getInstance() {
        if (instance == null) {
            synchronized (BookingManager.class) {
                if (instance == null) {
                    instance = new BookingManager();
                }
            }
        }
        return instance;
    }

    public DatabaseManager getDbManager() {
        return dbManager;
    }

    public synchronized void loadFromDatabase() {
        movies.clear();
        screens.clear();
        shows.clear();
        bookings.clear();
        transactions.clear();
        customers.clear();
        adminUsers.clear();
        auditLogs.clear();

        screens.addAll(dbManager.loadAllScreens());
        movies.addAll(dbManager.loadAllMovies());
        customers.addAll(dbManager.loadAllCustomers());
        adminUsers.addAll(dbManager.loadAllAdminUsers());
        shows.addAll(dbManager.loadAllShows(movies, screens));
        bookings.addAll(dbManager.loadAllBookings(customers, shows));
        transactions.addAll(dbManager.loadAllTransactions());
        auditLogs.addAll(dbManager.loadAllAuditLogs());
    }

    public List<Movie> getMovies() { return movies; }
    public List<Screen> getScreens() { return screens; }
    public List<Show> getShows() { return shows; }
    public List<Booking> getBookings() { return bookings; }
    public List<PaymentTransaction> getTransactions() { return transactions; }
    public List<Customer> getCustomers() { return customers; }
    public List<AdminUser> getAdminUsers() { return adminUsers; }
    public List<String> getAuditLogs() { return auditLogs; }

    public User getCurrentUser() { return currentUser; }
    public void setCurrentUser(User user) { this.currentUser = user; }

    public void logAudit(String action) {
        String timestamp = LocalDateTime.now().format(UIConstants.DATE_TIME_FORMATTER);
        String logEntry = "[" + timestamp + "] " + action;
        auditLogs.add(logEntry);
        dbManager.insertAuditLog(timestamp, action);
        System.out.println("[AUDIT] " + logEntry);
    }

    public Booking findBookingById(String bookingId) {
        if (bookingId == null) return null;
        for (Booking b : bookings) {
            if (b.getId().equalsIgnoreCase(bookingId.trim())) return b;
```

```
        }
        return null;
    }

    public PaymentTransaction findTransactionByBookingId(String bookingId) {
        if (bookingId == null) return null;
        for (PaymentTransaction pt : transactions) {
            if (pt.getBookingId().equalsIgnoreCase(bookingId.trim())) return pt;
        }
        return null;
    }

    public List<Show> getShowsForMovie(String movieId) {
        List<Show> result = new ArrayList<>();
        for (Show s : shows) {
            if (s.getMovie().getId().equalsIgnoreCase(movieId)) {
                result.add(s);
            }
        }
        return result;
    }

    public List<Booking> getBookingsForCustomer(String customerId) {
        List<Booking> result = new ArrayList<>();
        for (Booking b : bookings) {
            if (b.getCustomer().getId().equalsIgnoreCase(customerId)) {
                result.add(b);
            }
        }
        return result;
    }

    public int countTotalBookedSeatsRecursive(List<Booking> list, int index) {
        if (list == null || index >= list.size()) {
            return 0;
        }
        Booking b = list.get(index);
        int currentCount = (b.getBookingStatus() == BookingStatus.CONFIRMED) ? b.getSeatIds().size() : 0;
        return currentCount + countTotalBookedSeatsRecursive(list, index + 1);
    }

    public double calculateTotalRevenue() {
        double total = 0.0;
        for (PaymentTransaction t : transactions) {
            if (t.getPaymentStatus() == PaymentStatus.SUCCESS) {
                total += t.getAmount();
            } else if (t.getPaymentStatus() == PaymentStatus.REFUNDED) {
                total -= t.getAmount();
            }
        }
        return Math.max(0.0, Math.round(total * 100.0) / 100.0);
    }
}
```

File: BookingNotFoundException.java | Package: (default)

```
public class BookingNotFoundException extends CineBookException {
    public BookingNotFoundException(String message) { super(message); }
}
```

File: BookingService.java | Package: (default)

```
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;
```

```
public class BookingService {
    private final BookingManager manager;
    private final SeatManager seatManager;

    public BookingService(BookingManager manager, SeatManager seatManager) {
        this.manager = manager;
        this.seatManager = seatManager;
    }

    public synchronized Booking createPendingReservation(Customer customer, Show show, List<String>
seatIds)
            throws SeatUnavailableException, ValidationException {
        if (customer == null) throw new ValidationException("A valid customer must be specified.");
        if (show == null) throw new ValidationException("A valid show must be selected.");
        if (seatIds == null || seatIds.isEmpty()) {
            throw new ValidationException("At least one seat must be selected to proceed.");
        }

        seatManager.expireOutdatedReservations(show);

        double total = 0.0;
        for (String seatId : seatIds) {
            Seat seat = show.findSeat(seatId);
            if (seat == null) throw new SeatUnavailableException("Seat ID " + seatId + " does not
exist.");
            if (!seat.isAvailable()) {
                throw new SeatUnavailableException("Seat " + seatId + " is no longer available.");
            }
            total += show.calculateSeatPrice(seat);
        }

        String bookingId = "BKG-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
        for (String seatId : seatIds) {
            Seat seat = show.findSeat(seatId);
            seat.reserve(bookingId, 5);
        }

        Booking booking = new Booking(bookingId, customer, show, seatIds, total, 5);
        manager.getBookings().add(booking);
        manager.logAudit("Created pending booking " + bookingId + " for " + customer.getName() + " (" +
seatIds + ")");
        return booking;
    }

    public synchronized void cancelPendingReservation(Booking booking) {
        if (booking == null) return;
        if (booking.getBookingStatus() == BookingStatus.PENDING_PAYMENT) {
            seatManager.releaseSeats(booking.getShow(), booking.getSeatIds(), booking.getId());
            try {
                booking.cancel("Checkout abandoned by user.");
            } catch (InvalidBookingStateException ignored) {}
            manager.logAudit("Pending reservation " + booking.getId() + " released and cancelled.");
        }
    }

    public synchronized void confirmBooking(Booking booking) throws InvalidBookingStateException,
SeatUnavailableException {
        if (booking == null) throw new InvalidBookingStateException("Null booking provided.");

        if (booking.isExpired()) {
            seatManager.releaseSeats(booking.getShow(), booking.getSeatIds(), booking.getId());
            throw new InvalidBookingStateException("Booking has expired and cannot be confirmed.");
        }

        seatManager.confirmSeats(booking.getShow(), booking.getSeatIds(), booking.getId());
        booking.confirm();
```

```
        try {
            manager.getDbManager().insertBooking(booking);
        } catch (SQLException e) {
            System.err.println("[JDBC WARNING] Failed to persist booking to database: " + e.getMessage());
        }

        Customer customer = booking.getCustomer();
        if (customer != null) {
            int earnedPoints = (int) Math.round(booking.getTotalAmount());
            customer.addLoyaltyPoints(earnedPoints);
            manager.getDbManager().updateCustomerLoyalty(customer.getId(), customer.getLoyaltyPoints());
        }

        manager.logAudit("Confirmed booking " + booking.getId() + " for " +
booking.getCustomer().getName());
    }

    public synchronized void cancelConfirmedBooking(String bookingId)
            throws BookingNotFoundException, InvalidBookingStateException {
        Booking booking = manager.findBookingById(bookingId);
        if (booking == null) {
            throw new BookingNotFoundException("Booking ID " + bookingId + " not found.");
        }

        if (!booking.isCancellable()) {
            throw new InvalidBookingStateException("Booking " + bookingId + " cannot be cancelled. Current
status: " + booking.getBookingStatus());
        }

        seatManager.releaseSeats(booking.getShow(), booking.getSeatIds(), booking.getId());
        booking.cancel("Cancelled by customer request");

        manager.getDbManager().updateBookingStatus(bookingId, BookingStatus.CANCELLED, "Cancelled by
customer request");

        PaymentTransaction refundTxn = new PaymentTransaction(
                "TXN-REF-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase(),
                booking.getId(),
                "Simulated Refund",
                booking.getTotalAmount(),
                LocalDateTime.now(),
                PaymentStatus.REFUNDED,
                "REFUND-PROCESSED"
        );
        manager.getTransactions().add(refundTxn);

        try {
            manager.getDbManager().insertTransaction(refundTxn);
        } catch (SQLException e) {
            System.err.println("[JDBC WARNING] Failed to persist refund transaction: " + e.getMessage());
        }

        manager.logAudit("Cancelled booking " + booking.getId() + ". Seats released and simulated refund
recorded.");
    }
}
```

File: BookingStatus.java | Package: (default)

```
public enum BookingStatus {
    PENDING_PAYMENT("Pending Payment"),
    CONFIRMED("Confirmed"),
    PAYMENT_FAILED("Payment Failed"),
    CANCELLED("Cancelled"),
    EXPIRED("Expired");
```

```
    private final String displayName;

    BookingStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

File: Cancellable.java | Package: (default)

```
public interface Cancellable {
    boolean isCancellable();
    void cancel(String reason) throws InvalidBookingStateException;
}
```

File: CheckoutPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;

public class CheckoutPanel extends JPanel {
    private final MainFrame parent;
    private final PaymentProcessor paymentProcessor;
    private final BookingService bookingService;

    private Booking currentBooking;

    private JLabel posterLabel;
    private JLabel bookingIdLabel;
    private JLabel movieDetailsLabel;
    private JLabel seatListLabel;
    private JLabel timerLabel;
    private JLabel amountPayableLabel;

    private JComboBox<String> paymentMethodCombo;
    private JComboBox<String> simulationModeCombo;
    private JPanel dynamicPaymentFieldsPanel;
    private JTextField cardNumField;
    private JTextField cardHolderField;
    private JTextField upiField;
    private JTextField walletBalanceField;

    private javax.swing.Timer countdownTimer;
    private int remainingSeconds = 300;

    public CheckoutPanel(MainFrame parent, PaymentProcessor paymentProcessor, BookingService
bookingService) {
        this.parent = parent;
        this.paymentProcessor = paymentProcessor;
        this.bookingService = bookingService;

        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel topBar = new JPanel(new BorderLayout(16, 0));
        topBar.setBackground(Color.WHITE);
```

```
        topBar.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(12, 18, 12, 18)
        ));

        JButton cancelBtn = new JButton("← Cancel & Release Seats");
        cancelBtn.setFont(UIConstants.FONT_BOLD);
        cancelBtn.setBackground(new Color(254, 242, 242));
        cancelBtn.setForeground(new Color(220, 38, 38));
        cancelBtn.setFocusPainted(false);
        cancelBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        cancelBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(254, 202, 202), 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        cancelBtn.addActionListener(e -> cancelAndAbandon());

        JLabel headerTitle = new JLabel("Checkout & Simulated Payment Gateway");
        headerTitle.setFont(UIConstants.FONT_HEADING);
        headerTitle.setForeground(UIConstants.COLOR_TEXT_DARK);

        timerLabel = new JLabel("Hold Window: 05:00");
        timerLabel.setFont(UIConstants.FONT_BOLD);
        timerLabel.setForeground(new Color(220, 38, 38));
        timerLabel.setBackground(new Color(254, 242, 242));
        timerLabel.setOpaque(true);
        timerLabel.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(254, 202, 202), 1, true),
                new EmptyBorder(4, 12, 4, 12)
        ));

        topBar.add(cancelBtn, BorderLayout.WEST);
        topBar.add(headerTitle, BorderLayout.CENTER);
        topBar.add(timerLabel, BorderLayout.EAST);
        add(topBar, BorderLayout.NORTH);

        JPanel centerGrid = new JPanel(new GridLayout(1, 2, 20, 0));
        centerGrid.setOpaque(false);

        JPanel summaryCard = new JPanel(new BorderLayout(16, 16));
        summaryCard.setBackground(Color.WHITE);
        summaryCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(20, 20, 20, 20)
        ));

        JLabel summaryTitle = new JLabel("Booking Order Summary");
        summaryTitle.setFont(UIConstants.FONT_TITLE);
        summaryTitle.setForeground(UIConstants.COLOR_PRIMARY);
        summaryCard.add(summaryTitle, BorderLayout.NORTH);

        JPanel summaryBody = new JPanel(new BorderLayout(16, 0));
        summaryBody.setOpaque(false);

        posterLabel = new JLabel();
        posterLabel.setPreferredSize(new Dimension(110, 160));
        posterLabel.setBorder(BorderFactory.createLineBorder(new Color(226, 232, 240), 1, true));
        summaryBody.add(posterLabel, BorderLayout.WEST);

        JPanel summaryDetails = new JPanel();
        summaryDetails.setLayout(new BoxLayout(summaryDetails, BoxLayout.Y_AXIS));
        summaryDetails.setOpaque(false);

        bookingIdLabel = new JLabel("Reference: -");
        bookingIdLabel.setFont(UIConstants.FONT_BOLD);
        bookingIdLabel.setForeground(UIConstants.COLOR_ACCENT);
```

```
movieDetailsLabel = new JLabel("Show: -");
movieDetailsLabel.setFont(UIConstants.FONT_REGULAR);
movieDetailsLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

seatListLabel = new JLabel("Seats: -");
seatListLabel.setFont(UIConstants.FONT_REGULAR);
seatListLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);

amountPayableLabel = new JLabel("Total Payable: $0.00");
amountPayableLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
amountPayableLabel.setForeground(UIConstants.COLOR_SUCCESS);

summaryDetails.add(bookingIdLabel);
summaryDetails.add(Box.createVerticalStrut(8));
summaryDetails.add(movieDetailsLabel);
summaryDetails.add(Box.createVerticalStrut(8));
summaryDetails.add(seatListLabel);
summaryDetails.add(Box.createVerticalStrut(14));
summaryDetails.add(new JSeparator());
summaryDetails.add(Box.createVerticalStrut(14));
summaryDetails.add(amountPayableLabel);

summaryBody.add(summaryDetails, BorderLayout.CENTER);
summaryCard.add(summaryBody, BorderLayout.CENTER);

JPanel payCard = new JPanel();
payCard.setLayout(new BoxLayout(payCard, BoxLayout.Y_AXIS));
payCard.setBackground(Color.WHITE);
payCard.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(UIConstants.COLOR_BORDER, 1, true),
        new EmptyBorder(20, 20, 20, 20)
));

JLabel payTitle = new JLabel("Payment Method & Verification");
payTitle.setFont(UIConstants.FONT_TITLE);
payTitle.setForeground(UIConstants.COLOR_PRIMARY);
payTitle.setAlignmentX(Component.LEFT_ALIGNMENT);

JLabel methodLbl = new JLabel("Select Payment Channel (Strategy Pattern):");
methodLbl.setFont(UIConstants.FONT_BOLD);
methodLbl.setForeground(UIConstants.COLOR_TEXT_DARK);
methodLbl.setAlignmentX(Component.LEFT_ALIGNMENT);

String[] methods = {
        "Credit / Debit Card (MockCardPayment)",
        "UPI Instant Pay (MockUpiPayment)",
        "Digital Wallet (MockWalletPayment)",
        "Legacy Bank DirectDebit (Adapter Pattern)"
};
paymentMethodCombo = new JComboBox<>(methods);
paymentMethodCombo.setFont(UIConstants.FONT_REGULAR);
paymentMethodCombo.setMaximumSize(new Dimension(Integer.MAX_VALUE, 34));
paymentMethodCombo.setAlignmentX(Component.LEFT_ALIGNMENT);
paymentMethodCombo.addActionListener(e -> updatePaymentFields());

dynamicPaymentFieldsPanel = new JPanel();
dynamicPaymentFieldsPanel.setLayout(new GridLayout(2, 2, 8, 8));
dynamicPaymentFieldsPanel.setOpaque(false);
dynamicPaymentFieldsPanel.setMaximumSize(new Dimension(Integer.MAX_VALUE, 80));
dynamicPaymentFieldsPanel.setAlignmentX(Component.LEFT_ALIGNMENT);

JLabel simLbl = new JLabel("Simulation Mode (Demonstration Test Cases):");
simLbl.setFont(UIConstants.FONT_BOLD);
simLbl.setForeground(UIConstants.COLOR_TEXT_DARK);
simLbl.setAlignmentX(Component.LEFT_ALIGNMENT);
```

```
        String[] simModes = {
                "SUCCESS - Authorized & Completed",
                "FORCE_FAIL - Simulate Bank Declined",
                "USER_CANCEL - Abandon & Release Seats"
        };
        simulationModeCombo = new JComboBox<>(simModes);
        simulationModeCombo.setFont(UIConstants.FONT_REGULAR);
        simulationModeCombo.setMaximumSize(new Dimension(Integer.MAX_VALUE, 34));
        simulationModeCombo.setAlignmentX(Component.LEFT_ALIGNMENT);

        JButton payButton = new JButton("Complete Payment  →");
        payButton.setFont(UIConstants.FONT_BOLD);
        payButton.setBackground(UIConstants.COLOR_SUCCESS);
        payButton.setForeground(Color.WHITE);
        payButton.setFocusPainted(false);
        payButton.setCursor(new Cursor(Cursor.HAND_CURSOR));
        payButton.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
        payButton.setAlignmentX(Component.LEFT_ALIGNMENT);
        payButton.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(5, 150, 105), 1, true),
                new EmptyBorder(10, 16, 10, 16)
        ));
        payButton.addActionListener(e -> executePayment());

        payCard.add(payTitle);
        payCard.add(Box.createVerticalStrut(14));
        payCard.add(methodLbl);
        payCard.add(Box.createVerticalStrut(6));
        payCard.add(paymentMethodCombo);
        payCard.add(Box.createVerticalStrut(14));
        payCard.add(dynamicPaymentFieldsPanel);
        payCard.add(Box.createVerticalStrut(14));
        payCard.add(simLbl);
        payCard.add(Box.createVerticalStrut(6));
        payCard.add(simulationModeCombo);
        payCard.add(Box.createVerticalStrut(20));
        payCard.add(payButton);

        centerGrid.add(summaryCard);
        centerGrid.add(payCard);
        add(centerGrid, BorderLayout.CENTER);

        updatePaymentFields();
    }

    private void updatePaymentFields() {
        dynamicPaymentFieldsPanel.removeAll();
        int idx = paymentMethodCombo.getSelectedIndex();

        if (idx == 0) {
            dynamicPaymentFieldsPanel.add(new JLabel("Card Number:"));
            cardNumField = new JTextField("4532 8812 9012 3456");
            dynamicPaymentFieldsPanel.add(cardNumField);

            dynamicPaymentFieldsPanel.add(new JLabel("Cardholder Name:"));
            cardHolderField = new JTextField("Aarav Sharma");
            dynamicPaymentFieldsPanel.add(cardHolderField);
        } else if (idx == 1) {
            dynamicPaymentFieldsPanel.add(new JLabel("Virtual Payment Address (VPA):"));
            upiField = new JTextField("user@okaxis");
            dynamicPaymentFieldsPanel.add(upiField);
            dynamicPaymentFieldsPanel.add(new JLabel(""));
            dynamicPaymentFieldsPanel.add(new JLabel(""));
        } else if (idx == 2) {
            dynamicPaymentFieldsPanel.add(new JLabel("Simulated Balance ($):"));
```

```
            walletBalanceField = new JTextField("150.00");
            dynamicPaymentFieldsPanel.add(walletBalanceField);
            dynamicPaymentFieldsPanel.add(new JLabel(""));
            dynamicPaymentFieldsPanel.add(new JLabel(""));
        } else {
            JLabel adapterNote = new JLabel("<html><font color='#4F46E5'>Adapts LegacyBankGatewayApi via
LegacyBankGatewayAdapter</font></html>");
            dynamicPaymentFieldsPanel.add(adapterNote);
            dynamicPaymentFieldsPanel.add(new JLabel(""));
        }

        dynamicPaymentFieldsPanel.revalidate();
        dynamicPaymentFieldsPanel.repaint();
    }

    public void setBooking(Booking booking) {
        this.currentBooking = booking;
        this.remainingSeconds = 300;

        Movie movie = booking.getShow().getMovie();
        posterLabel.setIcon(UIConstants.getMoviePoster(movie.getId(), 110, 160));

        bookingIdLabel.setText("Reference: " + booking.getId());
        movieDetailsLabel.setText("Show: " + movie.getTitle() + " (" +
booking.getShow().getScreen().getName() + ")");
        seatListLabel.setText("Reserved Seats: " + String.join(", ", booking.getSeatIds()));
        amountPayableLabel.setText(String.format("Total Payable: $%.2f", booking.getTotalAmount()));

        if (countdownTimer != null) countdownTimer.stop();
        countdownTimer = new javax.swing.Timer(1000, e -> {
            remainingSeconds--;
            int mins = remainingSeconds / 60;
            int secs = remainingSeconds % 60;
            timerLabel.setText(String.format("Hold Window: %02d:%02d", mins, secs));

            if (remainingSeconds <= 0) {
                countdownTimer.stop();
                JOptionPane.showMessageDialog(this,
                        "Your 5-minute checkout window has expired. Seats have been automatically
released.",
                        "Reservation Expired", JOptionPane.WARNING_MESSAGE);
                bookingService.cancelPendingReservation(currentBooking);
                parent.navigateToSeatSelection(currentBooking.getShow());
            }
        });
        countdownTimer.start();
    }

    private void cancelAndAbandon() {
        if (countdownTimer != null) countdownTimer.stop();
        if (currentBooking != null) {
            bookingService.cancelPendingReservation(currentBooking);
        }
        parent.showMoviesScreen();
    }

    private void executePayment() {
        if (currentBooking == null) return;

        String simMode = "SUCCESS";
        int simIdx = simulationModeCombo.getSelectedIndex();
        if (simIdx == 1) simMode = "FORCE_FAIL";
        if (simIdx == 2) simMode = "USER_CANCEL";

        PaymentMethod method;
        int methodIdx = paymentMethodCombo.getSelectedIndex();
```

```
        if (methodIdx == 0) {
            method = new MockCardPayment(cardNumField.getText().trim(), cardHolderField.getText().trim());
        } else if (methodIdx == 1) {
            method = new MockUpiPayment(upiField.getText().trim());
        } else if (methodIdx == 2) {
            double bal = 150.0;
            try { bal = Double.parseDouble(walletBalanceField.getText().trim()); } catch (Exception
ignored) {}
            method = new MockWalletPayment("Paytm/GPay Wallet", bal);
        } else {

            try {
                if (countdownTimer != null) countdownTimer.stop();
                LegacyBankGatewayApi legacyApi = new LegacyBankGatewayApi();
                PaymentGateway adapter = new LegacyBankGatewayAdapter(legacyApi);
                PaymentTransaction txn = adapter.processPayment(currentBooking,
currentBooking.getTotalAmount(), "DirectDebit via Adapter");
                bookingService.confirmBooking(currentBooking);
                parent.navigateToConfirmation(currentBooking, txn);
                return;
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Legacy Adapter Payment Error: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        try {
            if (countdownTimer != null) countdownTimer.stop();
            PaymentTransaction txn = paymentProcessor.processSimulatedPayment(currentBooking, method,
simMode);

            if ("USER_CANCEL".equals(simMode)) {
                JOptionPane.showMessageDialog(this,
                        "Payment was cancelled by the user. Reservation has been released.",
                        "Payment Cancelled", JOptionPane.INFORMATION_MESSAGE);
                parent.showMoviesScreen();
            } else {
                parent.navigateToConfirmation(currentBooking, txn);
            }
        } catch (PaymentProcessingException | InvalidBookingStateException | SeatUnavailableException ex)
{
            JOptionPane.showMessageDialog(this,
                    "Payment Failed: " + ex.getMessage(),
                    "Transaction Declined", JOptionPane.ERROR_MESSAGE);
            parent.showMoviesScreen();
        }
    }
}
```

File: CineBookApp.java | Package: (default)

```
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class CineBookApp {

    public static void runAutomatedCoreVerification() {
        System.out.println(">>> RUNNING CINEBOOK AUTOMATED SYSTEM VERIFICATION <<<");
        BookingManager mgr = BookingManager.getInstance();
        SeatManager seatMgr = new SeatManager();
        MovieService movieSvc = new MovieService(mgr);
        BookingService bookingSvc = new BookingService(mgr, seatMgr);
```

```
        PaymentProcessor payProc = new PaymentProcessor(mgr, bookingSvc);

        int passCount = 0;
        int testCount = 0;

        testCount++;
        if (mgr.getMovies().size() >= 6) {
            passCount++;
            System.out.println("[PASS] Test 1: Sample movie catalogue loaded (" + mgr.getMovies().size() +
" movies from SQLite).");
        } else {
            System.err.println("[FAIL] Test 1: Movies not loaded properly from database.");
        }

        testCount++;
        Show s1 = mgr.getShows().get(0);
        Show s2 = mgr.getShows().get(1);
        Seat seatA1_s1 = s1.findSeat("A3");
        Seat seatA1_s2 = s2.findSeat("A3");
        if (seatA1_s1 != null && seatA1_s2 != null && seatA1_s1 != seatA1_s2) {
            passCount++;
            System.out.println("[PASS] Test 2: Shows maintain independent seat object instances.");
        } else {
            System.err.println("[FAIL] Test 2: Shows share seat instances.");
        }

        testCount++;
        try {
            Customer cust = mgr.getCustomers().get(0);

            List<String> targetSeats = new ArrayList<>();
            for (Seat[] row : s2.getSeats()) {
                for (Seat seat : row) {
                    if (seat.isAvailable()) {
                        targetSeats.add(seat.getId());
                        if (targetSeats.size() == 2) break;
                    }
                }
                if (targetSeats.size() == 2) break;
            }

            if (targetSeats.size() < 2) {

                Show fallbackShow = mgr.getShows().get(2);
                for (Seat[] row : fallbackShow.getSeats()) {
                    for (Seat seat : row) {
                        if (seat.isAvailable()) {
                            targetSeats.add(seat.getId());
                            if (targetSeats.size() == 2) break;
                        }
                    }
                    if (targetSeats.size() == 2) break;
                }
                s2 = fallbackShow;
            }

            Booking bkg = bookingSvc.createPendingReservation(cust, s2, targetSeats);
            PaymentMethod card = new MockCardPayment("1111222233334444", "Test User");
            PaymentTransaction txn = payProc.processSimulatedPayment(bkg, card, "SUCCESS");

            if (bkg.getBookingStatus() == BookingStatus.CONFIRMED &&
                    txn.getPaymentStatus() == PaymentStatus.SUCCESS &&
                    s2.findSeat(targetSeats.get(0)).getStatus() == SeatStatus.BOOKED) {
                passCount++;
                System.out.println("[PASS] Test 3: Temporary reservation -> Payment -> Confirmed booking
completed (" + targetSeats + ").");
```

```
            } else {
                System.err.println("[FAIL] Test 3: Booking confirmation state incorrect.");
            }
        } catch (Exception e) {
            System.err.println("[FAIL] Test 3 encountered exception: " + e.getMessage());
        }

        testCount++;
        try {
            Booking bkg = mgr.getBookings().get(mgr.getBookings().size() - 1);
            bkg.confirm();
            System.err.println("[FAIL] Test 4: Double confirmation was not prevented!");
        } catch (InvalidBookingStateException expected) {
            passCount++;
            System.out.println("[PASS] Test 4: Double confirmation prevented: " + expected.getMessage());
        } catch (Exception other) {
            System.err.println("[FAIL] Test 4: Unexpected exception type: " + other.getMessage());
        }

        testCount++;
        try {
            movieSvc.removeMovie("MOV-01");
            System.err.println("[FAIL] Test 5: Deletion guard did not trigger!");
        } catch (ValidationException expected) {
            passCount++;
            System.out.println("[PASS] Test 5: Safe deletion guard prevented deleting movie with bookings:
" + expected.getMessage());
        }

        testCount++;
        int recCount = mgr.countTotalBookedSeatsRecursive(mgr.getBookings(), 0);
        if (recCount > 0) {
            passCount++;
            System.out.println("[PASS] Test 6: Recursive booked seats calculation returned valid count: "
+ recCount);
        } else {
            System.err.println("[FAIL] Test 6: Recursive counter failed.");
        }

        System.out.println(String.format(">>> VERIFICATION SUMMARY: %d/%d TESTS PASSED <<<\n", passCount,
testCount));
    }

    public static void main(String[] args) {
        runAutomatedCoreVerification();

        if (args.length > 0 && ("--test-only".equalsIgnoreCase(args[0]) || "-
t".equalsIgnoreCase(args[0]))) {
            System.out.println("Headless verification flag supplied. Exiting cleanly without launching
GUI.");
            System.exit(0);
        }

        SwingUtilities.invokeLater(() -> {
            try {
                for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                    if ("Nimbus".equals(info.getName())) {
                        UIManager.setLookAndFeel(info.getClassName());
                        break;
                    }
                }
            } catch (Exception ignored) {}

            BookingManager.getInstance().setCurrentUser(null);

            LoginDialog loginDialog = new LoginDialog(null);
```

```
            User authenticatedUser = loginDialog.showLoginDialog();

            if (authenticatedUser == null) {
                System.out.println("No active user session authenticated. Exiting CineBook.");
                System.exit(0);
            }

            MainFrame frame = new MainFrame();
            frame.onUserLoggedIn(authenticatedUser);
            frame.setVisible(true);
        });
    }
}
```

File: CineBookException.java | Package: (default)

```
public class CineBookException extends Exception {
    public CineBookException(String message) { super(message); }
    public CineBookException(String message, Throwable cause) { super(message, cause); }
}
```

File: Customer.java | Package: (default)

```
public class Customer extends User {
    private int loyaltyPoints;

    public Customer(String id, String username, String password, String name, String email, int
loyaltyPoints) {
        super(id, username, password, name, email, "Customer");
        this.loyaltyPoints = loyaltyPoints;
    }

    public Customer(String id, String name, String email) {
        super(id, name, email, "Customer");
        this.loyaltyPoints = 0;
    }

    public Customer(String id, String name, String email, int loyaltyPoints) {
        super(id, name, email, "Customer");
        this.loyaltyPoints = loyaltyPoints;
    }

    public int getLoyaltyPoints() { return loyaltyPoints; }
    public void addLoyaltyPoints(int points) { this.loyaltyPoints += points; }

    @Override
    public String getDashboardGreeting() {
        return "Welcome back, " + getName() + "! Enjoy your cinema experience.";
    }
}
```

File: DatabaseManager.java | Package: (default)

```
import java.io.File;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class DatabaseManager {
    private static final String DB_DIR = "db";
    private static final String DB_FILE = "db" + File.separator + "cinebook.db";
    private static final String DB_URL = "jdbc:sqlite:" + DB_FILE;

    private static volatile DatabaseManager instance;
```

```
    private DatabaseManager() {
        initDatabase();
    }

    public static DatabaseManager getInstance() {
        if (instance == null) {
            synchronized (DatabaseManager.class) {
                if (instance == null) {
                    instance = new DatabaseManager();
                }
            }
        }
        return instance;
    }

    public Connection getConnection() throws SQLException {
        try {
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
            System.err.println("[JDBC WARNING] SQLite JDBC Driver class not found: " + e.getMessage());
        }
        return DriverManager.getConnection(DB_URL);
    }

    private synchronized void initDatabase() {
        File dir = new File(DB_DIR);
        if (!dir.exists()) {
            dir.mkdirs();
        }

        try (Connection conn = getConnection(); Statement stmt = conn.createStatement()) {

            stmt.execute("PRAGMA foreign_keys = ON;");

            stmt.execute("CREATE TABLE IF NOT EXISTS users (" +
                    "id TEXT PRIMARY KEY, " +
                    "username TEXT UNIQUE NOT NULL, " +
                    "password TEXT NOT NULL, " +
                    "name TEXT NOT NULL, " +
                    "email TEXT, " +
                    "role TEXT NOT NULL, " +
                    "loyalty_points INTEGER DEFAULT 0, " +
                    "access_level TEXT" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS screens (" +
                    "id TEXT PRIMARY KEY, " +
                    "name TEXT NOT NULL, " +
                    "seat_rows INTEGER NOT NULL, " +
                    "seat_cols INTEGER NOT NULL" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS movies (" +
                    "id TEXT PRIMARY KEY, " +
                    "title TEXT NOT NULL, " +
                    "genre TEXT NOT NULL, " +
                    "duration INTEGER NOT NULL, " +
                    "language TEXT, " +
                    "certification TEXT, " +
                    "description TEXT" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS shows (" +
                    "id TEXT PRIMARY KEY, " +
                    "movie_id TEXT NOT NULL, " +
                    "screen_id TEXT NOT NULL, " +
```

```
                    "show_time TEXT NOT NULL, " +
                    "base_price REAL NOT NULL, " +
                    "FOREIGN KEY (movie_id) REFERENCES movies(id), " +
                    "FOREIGN KEY (screen_id) REFERENCES screens(id)" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS bookings (" +
                    "id TEXT PRIMARY KEY, " +
                    "customer_id TEXT NOT NULL, " +
                    "show_id TEXT NOT NULL, " +
                    "seat_ids TEXT NOT NULL, " +
                    "total_amount REAL NOT NULL, " +
                    "created_at TEXT NOT NULL, " +
                    "expires_at TEXT NOT NULL, " +
                    "status TEXT NOT NULL, " +
                    "cancellation_reason TEXT, " +
                    "FOREIGN KEY (customer_id) REFERENCES users(id), " +
                    "FOREIGN KEY (show_id) REFERENCES shows(id)" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS transactions (" +
                    "id TEXT PRIMARY KEY, " +
                    "booking_id TEXT NOT NULL, " +
                    "payment_method TEXT NOT NULL, " +
                    "amount REAL NOT NULL, " +
                    "timestamp TEXT NOT NULL, " +
                    "status TEXT NOT NULL, " +
                    "reference TEXT, " +
                    "FOREIGN KEY (booking_id) REFERENCES bookings(id)" +
                    ");");

            stmt.execute("CREATE TABLE IF NOT EXISTS audit_logs (" +
                    "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                    "timestamp TEXT NOT NULL, " +
                    "action TEXT NOT NULL" +
                    ");");

            seedDefaultDataIfEmpty(conn);

        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Failed to initialize database schema: " + e.getMessage());
            e.printStackTrace();
        }
    }

    private void seedDefaultDataIfEmpty(Connection conn) throws SQLException {

        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS total FROM users")) {
            if (rs.next() && rs.getInt("total") == 0) {

                String insertUserSql = "INSERT INTO users (id, username, password, name, email, role,
loyalty_points, access_level) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertUserSql)) {

                    pstmt.setString(1, "ADM-01");
                    pstmt.setString(2, "admin");
                    pstmt.setString(3, "123");
                    pstmt.setString(4, "System Administrator");
                    pstmt.setString(5, "admin@cinebook.com");
                    pstmt.setString(6, "Admin");
                    pstmt.setInt(7, 0);
                    pstmt.setString(8, "SuperAdmin");
                    pstmt.executeUpdate();

                    pstmt.setString(1, "CUST-01");
```

```
                    pstmt.setString(2, "user1");
                    pstmt.setString(3, "123");
                    pstmt.setString(4, "Aarav Sharma");
                    pstmt.setString(5, "user1@cinebook.com");
                    pstmt.setString(6, "Customer");
                    pstmt.setInt(7, 150);
                    pstmt.setString(8, null);
                    pstmt.executeUpdate();

                    pstmt.setString(1, "CUST-02");
                    pstmt.setString(2, "user2");
                    pstmt.setString(3, "123");
                    pstmt.setString(4, "Diya Patel");
                    pstmt.setString(5, "user2@cinebook.com");
                    pstmt.setString(6, "Customer");
                    pstmt.setInt(7, 220);
                    pstmt.setString(8, null);
                    pstmt.executeUpdate();
                }
            }
        }

        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS total FROM screens")) {
            if (rs.next() && rs.getInt("total") == 0) {
                String insertScreenSql = "INSERT INTO screens (id, name, seat_rows, seat_cols) VALUES (?,
?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertScreenSql)) {
                    pstmt.setString(1, "SCR-01");
                    pstmt.setString(2, "Screen 1 (IMAX Laser)");
                    pstmt.setInt(3, 5);
                    pstmt.setInt(4, 8);
                    pstmt.executeUpdate();

                    pstmt.setString(1, "SCR-02");
                    pstmt.setString(2, "Screen 2 (Dolby Atmos)");
                    pstmt.setInt(3, 5);
                    pstmt.setInt(4, 8);
                    pstmt.executeUpdate();

                    pstmt.setString(1, "SCR-03");
                    pstmt.setString(2, "Screen 3 (Gold Lounge)");
                    pstmt.setInt(3, 5);
                    pstmt.setInt(4, 6);
                    pstmt.executeUpdate();
                }
            }
        }

        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS total FROM movies")) {
            if (rs.next() && rs.getInt("total") == 0) {
                String insertMovieSql = "INSERT INTO movies (id, title, genre, duration, language,
certification, description) VALUES (?, ?, ?, ?, ?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertMovieSql)) {
                    Object[][] moviesData = {
                            {"MOV-01", "Interstellar: Beyond Time", "Sci-Fi", 169, "English", "PG-13", "A
team of explorers travel through a wormhole in space in an attempt to ensure humanity's survival."},
                            {"MOV-02", "Inception", "Sci-Fi", 148, "English", "PG-13", "A thief who steals
corporate secrets through dream-sharing technology is given an inverse task of planting an idea."},
                            {"MOV-03", "The Dark Knight", "Action", 152, "English", "PG-13", "Batman
battles the Joker who unleashes psychological chaos across Gotham City."},
                            {"MOV-04", "La La Land", "Romance", 128, "English", "PG-13", "A jazz pianist
and an aspiring actress fall in love while pursuing their passions in modern Los Angeles."},
                            {"MOV-05", "Spirited Away", "Animation", 125, "Japanese", "PG", "A young girl
enters a realm governed by gods, witches, and spirits where humans are transformed into beasts."},
```

```
                            {"MOV-06", "The Grand Budapest Hotel", "Comedy", 100, "English", "R", "The
adventures of Gustave H, a legendary concierge at a famous European hotel between the wars."}
                    };

                    for (Object[] row : moviesData) {
                        pstmt.setString(1, (String) row[0]);
                        pstmt.setString(2, (String) row[1]);
                        pstmt.setString(3, (String) row[2]);
                        pstmt.setInt(4, (Integer) row[3]);
                        pstmt.setString(5, (String) row[4]);
                        pstmt.setString(6, (String) row[5]);
                        pstmt.setString(7, (String) row[6]);
                        pstmt.executeUpdate();
                    }
                }
            }
        }

        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS total FROM shows")) {
            if (rs.next() && rs.getInt("total") == 0) {
                LocalDateTime now = LocalDateTime.now();
                String insertShowSql = "INSERT INTO shows (id, movie_id, screen_id, show_time, base_price)
VALUES (?, ?, ?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertShowSql)) {
                    Object[][] showsData = {
                            {"SHW-01", "MOV-01", "SCR-01",
now.plusHours(3).format(UIConstants.DATE_TIME_FORMATTER), 15.00},
                            {"SHW-02", "MOV-01", "SCR-01",
now.plusDays(1).withHour(18).withMinute(0).format(UIConstants.DATE_TIME_FORMATTER), 16.50},
                            {"SHW-03", "MOV-02", "SCR-02",
now.plusHours(5).format(UIConstants.DATE_TIME_FORMATTER), 14.00},
                            {"SHW-04", "MOV-03", "SCR-01",
now.plusDays(1).withHour(20).withMinute(30).format(UIConstants.DATE_TIME_FORMATTER), 18.00},
                            {"SHW-05", "MOV-04", "SCR-03",
now.plusHours(2).format(UIConstants.DATE_TIME_FORMATTER), 22.00},
                            {"SHW-06", "MOV-05", "SCR-02",
now.plusDays(2).withHour(15).withMinute(0).format(UIConstants.DATE_TIME_FORMATTER), 13.50},
                            {"SHW-07", "MOV-06", "SCR-03",
now.plusHours(4).format(UIConstants.DATE_TIME_FORMATTER), 19.00}
                    };

                    for (Object[] row : showsData) {
                        pstmt.setString(1, (String) row[0]);
                        pstmt.setString(2, (String) row[1]);
                        pstmt.setString(3, (String) row[2]);
                        pstmt.setString(4, (String) row[3]);
                        pstmt.setDouble(5, (Double) row[4]);
                        pstmt.executeUpdate();
                    }
                }

                String insertBkgSql = "INSERT INTO bookings (id, customer_id, show_id, seat_ids,
total_amount, created_at, expires_at, status, cancellation_reason) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertBkgSql)) {
                    pstmt.setString(1, "BKG-SEED-01");
                    pstmt.setString(2, "CUST-01");
                    pstmt.setString(3, "SHW-01");
                    pstmt.setString(4, "A1,A2");
                    pstmt.setDouble(5, 30.00);
                    pstmt.setString(6, now.minusMinutes(10).format(UIConstants.DATE_TIME_FORMATTER));
                    pstmt.setString(7, now.plusMinutes(50).format(UIConstants.DATE_TIME_FORMATTER));
                    pstmt.setString(8, "CONFIRMED");
                    pstmt.setString(9, null);
                    pstmt.executeUpdate();
                }
```

```
                String insertTxnSql = "INSERT INTO transactions (id, booking_id, payment_method, amount,
timestamp, status, reference) VALUES (?, ?, ?, ?, ?, ?, ?)";
                try (PreparedStatement pstmt = conn.prepareStatement(insertTxnSql)) {
                    pstmt.setString(1, "TXN-SEED-01");
                    pstmt.setString(2, "BKG-SEED-01");
                    pstmt.setString(3, "Mock Credit/Debit Card");
                    pstmt.setDouble(4, 30.00);
                    pstmt.setString(5, now.minusMinutes(10).format(UIConstants.DATE_TIME_FORMATTER));
                    pstmt.setString(6, "SUCCESS");
                    pstmt.setString(7, "AUTH-SEED-1234");
                    pstmt.executeUpdate();
                }
            }
        }
    }

    public synchronized User authenticate(String username, String password) {
        if (username == null || password == null) return null;
        String sql = "SELECT * FROM users WHERE LOWER(username) = LOWER(?) AND password = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, username.trim());
            pstmt.setString(2, password.trim());
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    String id = rs.getString("id");
                    String uName = rs.getString("username");
                    String pwd = rs.getString("password");
                    String name = rs.getString("name");
                    String email = rs.getString("email");
                    String role = rs.getString("role");
                    int loyalty = rs.getInt("loyalty_points");
                    String access = rs.getString("access_level");

                    if ("Admin".equalsIgnoreCase(role)) {
                        return new AdminUser(id, uName, pwd, name, email, access != null ? access :
"Admin");
                    } else {
                        return new Customer(id, uName, pwd, name, email, loyalty);
                    }
                }
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] User authentication failed: " + e.getMessage());
        }
        return null;
    }

    public synchronized List<Customer> loadAllCustomers() {
        List<Customer> list = new ArrayList<>();
        String sql = "SELECT * FROM users WHERE role = 'Customer'";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Customer(
                        rs.getString("id"),
                        rs.getString("username"),
                        rs.getString("password"),
                        rs.getString("name"),
                        rs.getString("email"),
                        rs.getInt("loyalty_points")
                ));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading customers: " + e.getMessage());
```

```
        }
        return list;
    }

    public synchronized List<AdminUser> loadAllAdminUsers() {
        List<AdminUser> list = new ArrayList<>();
        String sql = "SELECT * FROM users WHERE role = 'Admin'";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new AdminUser(
                        rs.getString("id"),
                        rs.getString("username"),
                        rs.getString("password"),
                        rs.getString("name"),
                        rs.getString("email"),
                        rs.getString("access_level")
                ));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading admin users: " + e.getMessage());
        }
        return list;
    }

    public synchronized void updateCustomerLoyalty(String customerId, int points) {
        String sql = "UPDATE users SET loyalty_points = ? WHERE id = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, points);
            pstmt.setString(2, customerId);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Updating loyalty points: " + e.getMessage());
        }
    }

    public synchronized List<Screen> loadAllScreens() {
        List<Screen> list = new ArrayList<>();
        String sql = "SELECT * FROM screens ORDER BY id ASC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Screen(
                        rs.getString("id"),
                        rs.getString("name"),
                        rs.getInt("seat_rows"),
                        rs.getInt("seat_cols")
                ));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading screens: " + e.getMessage());
        }
        return list;
    }

    public synchronized List<Movie> loadAllMovies() {
        List<Movie> list = new ArrayList<>();
        String sql = "SELECT * FROM movies ORDER BY id ASC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Movie(
                        rs.getString("id"),
```

```
                        rs.getString("title"),
                        rs.getString("genre"),
                        rs.getInt("duration"),
                        rs.getString("language"),
                        rs.getString("certification"),
                        rs.getString("description")
                ));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading movies: " + e.getMessage());
        }
        return list;
    }

    public synchronized void insertMovie(Movie movie) throws SQLException {
        String sql = "INSERT INTO movies (id, title, genre, duration, language, certification,
description) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, movie.getId());
            pstmt.setString(2, movie.getTitle());
            pstmt.setString(3, movie.getGenre());
            pstmt.setInt(4, movie.getDurationMinutes());
            pstmt.setString(5, movie.getLanguage());
            pstmt.setString(6, movie.getCertification());
            pstmt.setString(7, movie.getDescription());
            pstmt.executeUpdate();
        }
    }

    public synchronized void updateMovie(Movie movie) throws SQLException {
        String sql = "UPDATE movies SET title = ?, genre = ?, duration = ?, language = ?, certification =
?, description = ? WHERE id = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, movie.getTitle());
            pstmt.setString(2, movie.getGenre());
            pstmt.setInt(3, movie.getDurationMinutes());
            pstmt.setString(4, movie.getLanguage());
            pstmt.setString(5, movie.getCertification());
            pstmt.setString(6, movie.getDescription());
            pstmt.setString(7, movie.getId());
            pstmt.executeUpdate();
        }
    }

    public synchronized void deleteMovie(String movieId) throws SQLException {
        String sql = "DELETE FROM movies WHERE id = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, movieId);
            pstmt.executeUpdate();
        }
    }

    public synchronized List<Show> loadAllShows(List<Movie> movies, List<Screen> screens) {
        List<Show> list = new ArrayList<>();
        String sql = "SELECT * FROM shows ORDER BY id ASC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                String showId = rs.getString("id");
                String movieId = rs.getString("movie_id");
                String screenId = rs.getString("screen_id");
                String timeStr = rs.getString("show_time");
                double basePrice = rs.getDouble("base_price");

                Movie matchedMovie = null;
```

```
                for (Movie m : movies) {
                    if (m.getId().equalsIgnoreCase(movieId)) {
                        matchedMovie = m;
                        break;
                    }
                }

                Screen matchedScreen = null;
                for (Screen s : screens) {
                    if (s.getId().equalsIgnoreCase(screenId)) {
                        matchedScreen = s;
                        break;
                    }
                }

                if (matchedMovie != null && matchedScreen != null) {
                    LocalDateTime dt = LocalDateTime.parse(timeStr, UIConstants.DATE_TIME_FORMATTER);
                    Show show = new Show(showId, matchedMovie, matchedScreen, dt, basePrice);
                    list.add(show);
                }
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading shows: " + e.getMessage());
        }
        return list;
    }

    public synchronized void insertShow(Show show) throws SQLException {
        String sql = "INSERT INTO shows (id, movie_id, screen_id, show_time, base_price) VALUES (?, ?, ?,
?, ?)";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, show.getId());
            pstmt.setString(2, show.getMovie().getId());
            pstmt.setString(3, show.getScreen().getId());
            pstmt.setString(4, show.getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER));
            pstmt.setDouble(5, show.getBasePrice());
            pstmt.executeUpdate();
        }
    }

    public synchronized void deleteShow(String showId) throws SQLException {
        String sql = "DELETE FROM shows WHERE id = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, showId);
            pstmt.executeUpdate();
        }
    }

    public synchronized List<Booking> loadAllBookings(List<Customer> customers, List<Show> shows) {
        List<Booking> list = new ArrayList<>();
        String sql = "SELECT * FROM bookings ORDER BY created_at DESC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                String bkgId = rs.getString("id");
                String custId = rs.getString("customer_id");
                String showId = rs.getString("show_id");
                String seatCsv = rs.getString("seat_ids");
                double amount = rs.getDouble("total_amount");
                String statusStr = rs.getString("status");
                String reason = rs.getString("cancellation_reason");

                Customer cust = null;
                for (Customer c : customers) {
                    if (c.getId().equalsIgnoreCase(custId)) {
```

```
                        cust = c;
                        break;
                    }
                }

                Show show = null;
                for (Show s : shows) {
                    if (s.getId().equalsIgnoreCase(showId)) {
                        show = s;
                        break;
                    }
                }

                if (cust != null && show != null) {
                    List<String> seats = Arrays.asList(seatCsv.split(","));
                    Booking bkg = new Booking(bkgId, cust, show, seats, amount, 60);

                    if ("CONFIRMED".equalsIgnoreCase(statusStr)) {
                        try {
                            bkg.confirm();

                            for (String seatId : seats) {
                                Seat st = show.findSeat(seatId.trim());
                                if (st != null) {
                                    st.confirmBooking(bkgId);
                                }
                            }
                        } catch (Exception ignored) {}
                    } else if ("CANCELLED".equalsIgnoreCase(statusStr)) {
                        try {
                            bkg.cancel(reason != null ? reason : "Cancelled in past session");
                        } catch (Exception ignored) {}
                    } else if ("PAYMENT_FAILED".equalsIgnoreCase(statusStr)) {
                        bkg.markPaymentFailed();
                    }
                    list.add(bkg);
                }
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading bookings: " + e.getMessage());
        }
        return list;
    }

    public synchronized void insertBooking(Booking bkg) throws SQLException {
        String sql = "INSERT INTO bookings (id, customer_id, show_id, seat_ids, total_amount, created_at,
expires_at, status, cancellation_reason) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, bkg.getId());
            pstmt.setString(2, bkg.getCustomer().getId());
            pstmt.setString(3, bkg.getShow().getId());
            pstmt.setString(4, String.join(",", bkg.getSeatIds()));
            pstmt.setDouble(5, bkg.getTotalAmount());
            pstmt.setString(6, bkg.getCreatedAt().format(UIConstants.DATE_TIME_FORMATTER));
            pstmt.setString(7, bkg.getExpiresAt().format(UIConstants.DATE_TIME_FORMATTER));
            pstmt.setString(8, bkg.getBookingStatus().name());
            pstmt.setString(9, bkg.getCancellationReason());
            pstmt.executeUpdate();
        }
    }

    public synchronized void updateBookingStatus(String bookingId, BookingStatus status, String reason) {
        String sql = "UPDATE bookings SET status = ?, cancellation_reason = ? WHERE id = ?";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, status.name());
            pstmt.setString(2, reason);
```

```
            pstmt.setString(3, bookingId);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Updating booking status: " + e.getMessage());
        }
    }

    public synchronized List<PaymentTransaction> loadAllTransactions() {
        List<PaymentTransaction> list = new ArrayList<>();
        String sql = "SELECT * FROM transactions ORDER BY timestamp DESC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                String txnId = rs.getString("id");
                String bkgId = rs.getString("booking_id");
                String method = rs.getString("payment_method");
                double amt = rs.getDouble("amount");
                String timeStr = rs.getString("timestamp");
                String statusStr = rs.getString("status");
                String ref = rs.getString("reference");

                LocalDateTime dt = LocalDateTime.parse(timeStr, UIConstants.DATE_TIME_FORMATTER);
                PaymentStatus status = PaymentStatus.valueOf(statusStr.toUpperCase());
                list.add(new PaymentTransaction(txnId, bkgId, method, amt, dt, status, ref));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading transactions: " + e.getMessage());
        }
        return list;
    }

    public synchronized void insertTransaction(PaymentTransaction txn) throws SQLException {
        String sql = "INSERT INTO transactions (id, booking_id, payment_method, amount, timestamp, status,
reference) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, txn.getId());
            pstmt.setString(2, txn.getBookingId());
            pstmt.setString(3, txn.getPaymentMethodName());
            pstmt.setDouble(4, txn.getAmount());
            pstmt.setString(5, txn.getTimestamp().format(UIConstants.DATE_TIME_FORMATTER));
            pstmt.setString(6, txn.getPaymentStatus().name());
            pstmt.setString(7, txn.getGatewayReference());
            pstmt.executeUpdate();
        }
    }

    public synchronized List<String> loadAllAuditLogs() {
        List<String> list = new ArrayList<>();
        String sql = "SELECT timestamp, action FROM audit_logs ORDER BY id ASC";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add("[" + rs.getString("timestamp") + "] " + rs.getString("action"));
            }
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Loading audit logs: " + e.getMessage());
        }
        return list;
    }

    public synchronized void insertAuditLog(String timestamp, String action) {
        String sql = "INSERT INTO audit_logs (timestamp, action) VALUES (?, ?)";
        try (Connection conn = getConnection(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, timestamp);
```

```
            pstmt.setString(2, action);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("[JDBC ERROR] Inserting audit log: " + e.getMessage());
        }
    }
}
```

File: InvalidBookingStateException.java | Package: (default)

```
public class InvalidBookingStateException extends CineBookException {
    public InvalidBookingStateException(String message) { super(message); }
}
```

File: LegacyBankGatewayAdapter.java | Package: (default)

```
import java.time.LocalDateTime;
import java.util.UUID;

public class LegacyBankGatewayAdapter implements PaymentGateway {
    private final LegacyBankGatewayApi legacyApi;

    public LegacyBankGatewayAdapter(LegacyBankGatewayApi legacyApi) {
        this.legacyApi = legacyApi;
    }

    @Override
    public PaymentTransaction processPayment(Booking booking, double amount, String notes)
            throws PaymentProcessingException {
        if (booking == null) {
            throw new PaymentProcessingException("Adapter failed: Booking is null.");
        }

        String simulatedAccount = "ACCT-987654321";
        LegacyBankResult result = legacyApi.chargeAccount(simulatedAccount, amount);

        if (!result.isSuccess()) {
            throw new PaymentProcessingException("Legacy bank declined transaction: " +
result.getReference());
        }

        return new PaymentTransaction(
                "TXN-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase(),
                booking.getId(),
                "Bank DirectDebit (Adapter)",
                amount,
                LocalDateTime.now(),
                PaymentStatus.SUCCESS,
                result.getReference()
        );
    }
}
```

File: LegacyBankGatewayApi.java | Package: (default)

```
public class LegacyBankGatewayApi {
    public LegacyBankResult chargeAccount(String bankAccountNumber, double amount) {
        if (bankAccountNumber != null && !bankAccountNumber.trim().isEmpty() && amount > 0) {
            return new LegacyBankResult(true, "LEGACY-AUTH-" + System.currentTimeMillis());
        }
        return new LegacyBankResult(false, "ERR-INVALID-ACC");
    }
}
```

File: LegacyBankResult.java | Package: (default)

```
public class LegacyBankResult {
    private final boolean success;
```

```
    private final String reference;

    public LegacyBankResult(boolean success, String reference) {
        this.success = success;
        this.reference = reference;
    }

    public boolean isSuccess() { return success; }
    public String getReference() { return reference; }
}
```

File: LoginDialog.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class LoginDialog extends JDialog {
    private final JTextField usernameField;
    private final JPasswordField passwordField;
    private final JLabel errorLabel;
    private User authenticatedUser;

    public LoginDialog(Frame parent) {
        super(parent, "CineBook — Account Authentication", true);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);
        setSize(440, 500);
        setLocationRelativeTo(parent);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                authenticatedUser = null;
            }
        });

        JPanel mainPanel = new JPanel(new BorderLayout());
        mainPanel.setBackground(UIConstants.COLOR_BG_LIGHT);

        JPanel headerPanel = new JPanel(new BorderLayout());
        headerPanel.setBackground(UIConstants.COLOR_PRIMARY);
        headerPanel.setBorder(new EmptyBorder(22, 28, 22, 28));

        JLabel titleLabel = new JLabel("CINEBOOK");
        titleLabel.setFont(UIConstants.FONT_HERO);
        titleLabel.setForeground(Color.WHITE);

        JLabel subtitleLabel = new JLabel("Cinema Admission Management • System Sign-In");
        subtitleLabel.setFont(UIConstants.FONT_SMALL);
        subtitleLabel.setForeground(new Color(203, 213, 225));

        headerPanel.add(titleLabel, BorderLayout.NORTH);
        headerPanel.add(subtitleLabel, BorderLayout.SOUTH);
        mainPanel.add(headerPanel, BorderLayout.NORTH);

        JPanel formCard = new JPanel();
        formCard.setLayout(new BoxLayout(formCard, BoxLayout.Y_AXIS));
        formCard.setBackground(Color.WHITE);
        formCard.setBorder(BorderFactory.createCompoundBorder(
                new EmptyBorder(16, 24, 16, 24),
                BorderFactory.createCompoundBorder(
                        new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                        new EmptyBorder(18, 20, 18, 20)
```

```
        )
));

JLabel userLabel = new JLabel("Username:");
userLabel.setFont(UIConstants.FONT_BOLD);
userLabel.setForeground(UIConstants.COLOR_TEXT_DARK);
userLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

usernameField = new JTextField();
usernameField.setFont(UIConstants.FONT_REGULAR);
usernameField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 36));
usernameField.setAlignmentX(Component.LEFT_ALIGNMENT);

JLabel passLabel = new JLabel("Password:");
passLabel.setFont(UIConstants.FONT_BOLD);
passLabel.setForeground(UIConstants.COLOR_TEXT_DARK);
passLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

passwordField = new JPasswordField();
passwordField.setFont(UIConstants.FONT_REGULAR);
passwordField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 36));
passwordField.setAlignmentX(Component.LEFT_ALIGNMENT);

errorLabel = new JLabel(" ");
errorLabel.setFont(UIConstants.FONT_SMALL);
errorLabel.setForeground(UIConstants.COLOR_DANGER);
errorLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

JPanel hintBox = new JPanel();
hintBox.setLayout(new BoxLayout(hintBox, BoxLayout.Y_AXIS));
hintBox.setBackground(new Color(248, 250, 252));
hintBox.setBorder(BorderFactory.createCompoundBorder(
        new LineBorder(new Color(226, 232, 240), 1, true),
        new EmptyBorder(10, 12, 10, 12)
));
hintBox.setAlignmentX(Component.LEFT_ALIGNMENT);
hintBox.setMaximumSize(new Dimension(Integer.MAX_VALUE, 85));

JLabel hintTitle = new JLabel("Quick Login (Click to Fill - Password: 123):");
hintTitle.setFont(UIConstants.FONT_BADGE);
hintTitle.setForeground(UIConstants.COLOR_TEXT_MUTED);

JPanel quickFillRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
quickFillRow.setOpaque(false);

JButton fillAdminBtn = new JButton("admin");
fillAdminBtn.setFont(UIConstants.FONT_SMALL);
fillAdminBtn.setBackground(new Color(238, 242, 255));
fillAdminBtn.setForeground(UIConstants.COLOR_ACCENT);
fillAdminBtn.setFocusPainted(false);
fillAdminBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
fillAdminBtn.addActionListener(e -> {
    usernameField.setText("admin");
    passwordField.setText("123");
    errorLabel.setText(" ");
});

JButton fillUser1Btn = new JButton("user1");
fillUser1Btn.setFont(UIConstants.FONT_SMALL);
fillUser1Btn.setBackground(new Color(236, 253, 245));
fillUser1Btn.setForeground(new Color(5, 150, 105));
fillUser1Btn.setFocusPainted(false);
fillUser1Btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
fillUser1Btn.addActionListener(e -> {
    usernameField.setText("user1");
    passwordField.setText("123");
```

```
            errorLabel.setText(" ");
        });

        JButton fillUser2Btn = new JButton("user2");
        fillUser2Btn.setFont(UIConstants.FONT_SMALL);
        fillUser2Btn.setBackground(new Color(254, 243, 199));
        fillUser2Btn.setForeground(new Color(180, 83, 9));
        fillUser2Btn.setFocusPainted(false);
        fillUser2Btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        fillUser2Btn.addActionListener(e -> {
            usernameField.setText("user2");
            passwordField.setText("123");
            errorLabel.setText(" ");
        });

        quickFillRow.add(fillAdminBtn);
        quickFillRow.add(fillUser1Btn);
        quickFillRow.add(fillUser2Btn);

        hintBox.add(hintTitle);
        hintBox.add(quickFillRow);

        formCard.add(userLabel);
        formCard.add(Box.createVerticalStrut(4));
        formCard.add(usernameField);
        formCard.add(Box.createVerticalStrut(10));
        formCard.add(passLabel);
        formCard.add(Box.createVerticalStrut(4));
        formCard.add(passwordField);
        formCard.add(Box.createVerticalStrut(6));
        formCard.add(errorLabel);
        formCard.add(Box.createVerticalStrut(4));
        formCard.add(hintBox);

        mainPanel.add(formCard, BorderLayout.CENTER);

        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 14));
        buttonPanel.setBackground(UIConstants.COLOR_BG_LIGHT);

        JButton exitBtn = new JButton("Exit Application");
        exitBtn.setFont(UIConstants.FONT_REGULAR);
        exitBtn.setBackground(Color.WHITE);
        exitBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        exitBtn.setFocusPainted(false);
        exitBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        exitBtn.addActionListener(e -> {
            authenticatedUser = null;
            dispose();
        });

        JButton loginBtn = new JButton("Sign In  →");
        loginBtn.setFont(UIConstants.FONT_BOLD);
        loginBtn.setBackground(UIConstants.COLOR_ACCENT);
        loginBtn.setForeground(Color.WHITE);
        loginBtn.setFocusPainted(false);
        loginBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        loginBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(8, 20, 8, 20)
        ));
        loginBtn.addActionListener(e -> attemptLogin());

        getRootPane().setDefaultButton(loginBtn);

        buttonPanel.add(exitBtn);
        buttonPanel.add(loginBtn);
```

```
        mainPanel.add(buttonPanel, BorderLayout.SOUTH);

        setContentPane(mainPanel);
    }

    private void attemptLogin() {
        String username = usernameField.getText().trim();
        String password = new String(passwordField.getPassword()).trim();

        if (username.isEmpty() || password.isEmpty()) {
            errorLabel.setText("Please enter both username and password.");
            return;
        }

        User user = DatabaseManager.getInstance().authenticate(username, password);
        if (user != null) {
            this.authenticatedUser = user;
            BookingManager.getInstance().setCurrentUser(user);
            BookingManager.getInstance().logAudit("User logged in successfully: " + user.getName() + " ("
+ user.getRole() + ")");
            dispose();
        } else {
            errorLabel.setText("Invalid credentials. Please verify username and password.");
            passwordField.setText("");
            passwordField.requestFocus();
        }
    }

    public User showLoginDialog() {
        setVisible(true);
        return authenticatedUser;
    }
}
```

File: MainFrame.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;

public class MainFrame extends JFrame {
    private final BookingManager manager;
    private final MovieService movieService;
    private final ShowService showService;
    private final SeatManager seatManager;
    private final BookingService bookingService;
    private final PaymentProcessor paymentProcessor;

    private final CardLayout cardLayout;
    private final JPanel contentPanel;

    private MoviePanel moviePanel;
    private MovieDetailsPanel movieDetailsPanel;
    private SeatSelectionPanel seatSelectionPanel;
    private CheckoutPanel checkoutPanel;
    private BookingConfirmationPanel confirmationPanel;
    private BookingHistoryPanel historyPanel;
    private AdminPanel adminPanel;

    private JLabel userStatusLabel;
    private JButton btnMovies;
    private JButton btnHistory;
    private JButton btnAdmin;

    public MainFrame() {
        super("CineBook – Movie Ticket Booking System");
```

```
        this.manager = BookingManager.getInstance();
        this.movieService = new MovieService(manager);
        this.showService = new ShowService(manager);
        this.seatManager = new SeatManager();
        this.bookingService = new BookingService(manager, seatManager);
        this.paymentProcessor = new PaymentProcessor(manager, bookingService);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setSize(1280, 820);
        setMinimumSize(new Dimension(1120, 720));
        setLocationRelativeTo(null);

        setLayout(new BorderLayout());

        add(createTopNavBar(), BorderLayout.NORTH);

        cardLayout = new CardLayout();
        contentPanel = new JPanel(cardLayout);

        moviePanel = new MoviePanel(this, movieService);
        movieDetailsPanel = new MovieDetailsPanel(this, showService);
        seatSelectionPanel = new SeatSelectionPanel(this, bookingService);
        checkoutPanel = new CheckoutPanel(this, paymentProcessor, bookingService);
        confirmationPanel = new BookingConfirmationPanel(this);
        historyPanel = new BookingHistoryPanel(this, manager, bookingService);
        adminPanel = new AdminPanel(this, manager, movieService, showService);

        contentPanel.add(moviePanel, "MOVIES");
        contentPanel.add(movieDetailsPanel, "MOVIE_DETAILS");
        contentPanel.add(seatSelectionPanel, "SEAT_SELECTION");
        contentPanel.add(checkoutPanel, "CHECKOUT");
        contentPanel.add(confirmationPanel, "CONFIRMATION");
        contentPanel.add(historyPanel, "HISTORY");
        contentPanel.add(adminPanel, "ADMIN");

        add(contentPanel, BorderLayout.CENTER);
    }

    private JPanel createTopNavBar() {
        JPanel nav = new JPanel(new BorderLayout());
        nav.setBackground(UIConstants.COLOR_PRIMARY);
        nav.setBorder(new EmptyBorder(14, 24, 14, 24));

        JPanel leftPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 16, 0));
        leftPanel.setOpaque(false);

        JLabel logoLabel = new JLabel("CINEBOOK");
        logoLabel.setFont(new Font("Segoe UI", Font.BOLD, 22));
        logoLabel.setForeground(Color.WHITE);
        leftPanel.add(logoLabel);

        JLabel sep = new JLabel("|");
        sep.setFont(new Font("Segoe UI", Font.PLAIN, 18));
        sep.setForeground(new Color(100, 116, 139));
        leftPanel.add(sep);

        btnMovies = createNavButton("Browse Movies");
        btnMovies.addActionListener(e -> showMoviesScreen());
        leftPanel.add(btnMovies);

        btnHistory = createNavButton("My Bookings");
        btnHistory.addActionListener(e -> showHistoryScreen());
        leftPanel.add(btnHistory);

        btnAdmin = createNavButton("Admin Console");
```

```
        btnAdmin.addActionListener(e -> showAdminScreen());
        leftPanel.add(btnAdmin);

        btnAdmin.setVisible(false);

        JPanel rightPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));
        rightPanel.setOpaque(false);

        JPanel userBadge = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 4));
        userBadge.setBackground(new Color(30, 41, 59));
        userBadge.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(51, 65, 85), 1, true),
                new EmptyBorder(2, 10, 2, 10)
        ));

        userStatusLabel = new JLabel("Not logged in");
        userStatusLabel.setFont(UIConstants.FONT_BOLD);
        userStatusLabel.setForeground(new Color(241, 245, 249));
        userBadge.add(userStatusLabel);
        rightPanel.add(userBadge);

        JButton logoutBtn = new JButton("Logout");
        logoutBtn.setFont(UIConstants.FONT_BOLD);
        logoutBtn.setBackground(new Color(239, 68, 68));
        logoutBtn.setForeground(Color.WHITE);
        logoutBtn.setFocusPainted(false);
        logoutBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(220, 38, 38), 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        logoutBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        logoutBtn.addActionListener(e -> handleLogout());
        rightPanel.add(logoutBtn);

        nav.add(leftPanel, BorderLayout.WEST);
        nav.add(rightPanel, BorderLayout.EAST);
        return nav;
    }

    private JButton createNavButton(String text) {
        JButton btn = new JButton(text);
        btn.setFont(UIConstants.FONT_BOLD);
        btn.setForeground(new Color(226, 232, 240));
        btn.setBackground(new Color(30, 41, 59));
        btn.setFocusPainted(false);
        btn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(71, 85, 105), 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        return btn;
    }

    public void handleLogout() {
        int confirm = JOptionPane.showConfirmDialog(
                this,
                "Are you sure you want to log out of CineBook?",
                "Confirm Logout",
                JOptionPane.YES_NO_OPTION,
                JOptionPane.QUESTION_MESSAGE
        );

        if (confirm == JOptionPane.YES_OPTION) {
            if (manager.getCurrentUser() != null) {
                manager.logAudit("User logged out: " + manager.getCurrentUser().getName());
            }
```

```
            manager.setCurrentUser(null);
            setVisible(false);

            LoginDialog loginDialog = new LoginDialog(null);
            User user = loginDialog.showLoginDialog();
            if (user != null) {
                onUserLoggedIn(user);
                setVisible(true);
            } else {
                System.exit(0);
            }
        }
    }

    public void onUserLoggedIn(User user) {
        manager.setCurrentUser(user);

        if (userStatusLabel != null) {
            String roleTag = user.getRole().equalsIgnoreCase("Admin") ? " [Admin]" : " [Customer]";
            userStatusLabel.setText(user.getName() + roleTag);
        }

        boolean isAdmin = "Admin".equalsIgnoreCase(user.getRole());

        if (isAdmin) {

            btnMovies.setVisible(false);
            btnHistory.setVisible(false);
            btnAdmin.setVisible(true);
            showAdminScreen();
        } else {

            btnMovies.setVisible(true);
            btnHistory.setVisible(true);
            btnAdmin.setVisible(false);
            showMoviesScreen();
        }

        if (historyPanel != null) historyPanel.refreshHistory();
        if (moviePanel != null) moviePanel.refreshList();
        if (adminPanel != null) adminPanel.refreshData();
    }

    public void showMoviesScreen() {
        moviePanel.refreshList();
        cardLayout.show(contentPanel, "MOVIES");
    }

    public void navigateToMovieDetails(Movie movie) {
        movieDetailsPanel.setMovie(movie);
        cardLayout.show(contentPanel, "MOVIE_DETAILS");
    }

    public void navigateToSeatSelection(Show show) {
        seatSelectionPanel.setShow(show);
        cardLayout.show(contentPanel, "SEAT_SELECTION");
    }

    public void navigateToCheckout(Booking booking) {
        checkoutPanel.setBooking(booking);
        cardLayout.show(contentPanel, "CHECKOUT");
    }

    public void navigateToConfirmation(Booking booking, PaymentTransaction txn) {
        confirmationPanel.displayConfirmation(booking, txn);
        cardLayout.show(contentPanel, "CONFIRMATION");
```

```
    }

    public void showHistoryScreen() {
        historyPanel.refreshHistory();
        cardLayout.show(contentPanel, "HISTORY");
    }

    public void showAdminScreen() {
        adminPanel.refreshData();
        cardLayout.show(contentPanel, "ADMIN");
    }
}
```

File: MockCardPayment.java | Package: (default)

```
public class MockCardPayment extends PaymentMethod {
    private final String cardNumber;
    private final String cardHolder;

    public MockCardPayment(String cardNumber, String cardHolder) {
        super("Mock Credit/Debit Card");
        this.cardNumber = cardNumber;
        this.cardHolder = cardHolder;
    }

    public String getCardNumber() { return cardNumber; }
    public String getCardHolder() { return cardHolder; }

    @Override
    public boolean executeTransaction(double amount) throws PaymentProcessingException {
        if (cardNumber == null || cardNumber.trim().length() < 4) {
            throw new PaymentProcessingException("Card payment failed: Invalid card details provided.");
        }
        return true;
    }
}
```

File: MockUpiPayment.java | Package: (default)

```
public class MockUpiPayment extends PaymentMethod {
    private final String upiId;

    public MockUpiPayment(String upiId) {
        super("Mock UPI Instant Pay");
        this.upiId = upiId;
    }

    public String getUpiId() { return upiId; }

    @Override
    public boolean executeTransaction(double amount) throws PaymentProcessingException {
        if (upiId == null || !upiId.contains("@")) {
            throw new PaymentProcessingException("UPI payment failed: Invalid virtual payment address
(VPA).");
        }
        return true;
    }
}
```

File: MockWalletPayment.java | Package: (default)

```
public class MockWalletPayment extends PaymentMethod {
    private final String walletName;
    private final double walletBalance;

    public MockWalletPayment(String walletName, double walletBalance) {
        super("Mock Digital Wallet (" + walletName + ")");
```

```
        this.walletName = walletName;
        this.walletBalance = walletBalance;
    }

    public String getWalletName() { return walletName; }
    public double getWalletBalance() { return walletBalance; }

    @Override
    public boolean executeTransaction(double amount) throws PaymentProcessingException {
        if (walletBalance < amount) {
            throw new PaymentProcessingException(
                    String.format("Wallet payment failed: Insufficient balance ($%.2f available, $%.2f
required).",
                            walletBalance, amount));
        }
        return true;
    }
}
```

File: Movie.java | Package: (default)

```
public class Movie {
    private final String id;
    private String title;
    private String genre;
    private int durationMinutes;
    private String language;
    private String certification;
    private String description;

    public Movie(String id, String title, String genre, int durationMinutes,
                 String language, String certification, String description) {
        this.id = id;
        this.title = title;
        this.genre = genre;
        this.durationMinutes = durationMinutes;
        this.language = language;
        this.certification = certification;
        this.description = description;
    }

    public Movie(String id, String title, String genre, int durationMinutes) {
        this(id, title, genre, durationMinutes, "English", "PG-13", "Feature film presentation.");
    }

    public String getId() { return id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getGenre() { return genre; }
    public void setGenre(String genre) { this.genre = genre; }
    public int getDurationMinutes() { return durationMinutes; }
    public void setDurationMinutes(int durationMinutes) { this.durationMinutes = durationMinutes; }
    public String getLanguage() { return language; }
    public void setLanguage(String language) { this.language = language; }
    public String getCertification() { return certification; }
    public void setCertification(String certification) { this.certification = certification; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public boolean matches(String keyword) {
        if (keyword == null || keyword.trim().isEmpty()) return true;
        String lower = keyword.trim().toLowerCase();
        return title.toLowerCase().contains(lower) || genre.toLowerCase().contains(lower);
    }

    public boolean matches(String titleQuery, String genreFilter) {
        boolean matchesTitle = (titleQuery == null || titleQuery.trim().isEmpty())
```

```
                || title.toLowerCase().contains(titleQuery.trim().toLowerCase());
        boolean matchesGenre = (genreFilter == null || genreFilter.equalsIgnoreCase("All") ||
genreFilter.trim().isEmpty())
                || genre.equalsIgnoreCase(genreFilter.trim());
        return matchesTitle && matchesGenre;
    }

    @Override
    public String toString() {
        return title + " (" + certification + ", " + durationMinutes + " mins)";
    }
}
```

File: MovieDetailsPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.util.List;

public class MovieDetailsPanel extends JPanel {
    private final MainFrame parent;
    private final ShowService showService;

    private Movie currentMovie;
    private JLabel posterLabel;
    private JLabel titleLabel;
    private JPanel badgesPanel;
    private JLabel durationLabel;
    private JTextArea synopsisArea;
    private JPanel showsContainer;

    public MovieDetailsPanel(MainFrame parent, ShowService showService) {
        this.parent = parent;
        this.showService = showService;
        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel topNav = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
        topNav.setOpaque(false);

        JButton backBtn = new JButton("← Back to Movie Catalogue");
        backBtn.setFont(UIConstants.FONT_BOLD);
        backBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        backBtn.setBackground(Color.WHITE);
        backBtn.setFocusPainted(false);
        backBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        backBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(8, 16, 8, 16)
        ));
        backBtn.addActionListener(e -> parent.showMoviesScreen());
        topNav.add(backBtn);
        add(topNav, BorderLayout.NORTH);

        JPanel mainContent = new JPanel();
        mainContent.setLayout(new BoxLayout(mainContent, BoxLayout.Y_AXIS));
        mainContent.setOpaque(false);

        JPanel heroCard = new JPanel(new BorderLayout(24, 0));
```

```
        heroCard.setBackground(Color.WHITE);
        heroCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(20, 24, 20, 24)
        ));

        posterLabel = new JLabel();
        posterLabel.setPreferredSize(new Dimension(200, 290));
        posterLabel.setBorder(BorderFactory.createLineBorder(new Color(226, 232, 240), 1, true));
        heroCard.add(posterLabel, BorderLayout.WEST);

        JPanel detailsPanel = new JPanel();
        detailsPanel.setLayout(new BoxLayout(detailsPanel, BoxLayout.Y_AXIS));
        detailsPanel.setOpaque(false);

        titleLabel = new JLabel("Movie Title");
        titleLabel.setFont(UIConstants.FONT_HERO);
        titleLabel.setForeground(UIConstants.COLOR_TEXT_DARK);
        titleLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

        badgesPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        badgesPanel.setOpaque(false);
        badgesPanel.setAlignmentX(Component.LEFT_ALIGNMENT);

        durationLabel = new JLabel();
        durationLabel.setFont(UIConstants.FONT_REGULAR);
        durationLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);
        durationLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

        JLabel synopsisHeading = new JLabel("SYNOPSIS");
        synopsisHeading.setFont(UIConstants.FONT_SMALL);
        synopsisHeading.setForeground(UIConstants.COLOR_ACCENT);
        synopsisHeading.setAlignmentX(Component.LEFT_ALIGNMENT);

        synopsisArea = new JTextArea();
        synopsisArea.setFont(UIConstants.FONT_REGULAR);
        synopsisArea.setForeground(new Color(51, 65, 85));
        synopsisArea.setLineWrap(true);
        synopsisArea.setWrapStyleWord(true);
        synopsisArea.setEditable(false);
        synopsisArea.setOpaque(false);
        synopsisArea.setAlignmentX(Component.LEFT_ALIGNMENT);

        detailsPanel.add(titleLabel);
        detailsPanel.add(Box.createVerticalStrut(8));
        detailsPanel.add(badgesPanel);
        detailsPanel.add(Box.createVerticalStrut(10));
        detailsPanel.add(durationLabel);
        detailsPanel.add(Box.createVerticalStrut(14));
        detailsPanel.add(new JSeparator());
        detailsPanel.add(Box.createVerticalStrut(12));
        detailsPanel.add(synopsisHeading);
        detailsPanel.add(Box.createVerticalStrut(4));
        detailsPanel.add(synopsisArea);

        heroCard.add(detailsPanel, BorderLayout.CENTER);
        mainContent.add(heroCard);
        mainContent.add(Box.createVerticalStrut(18));

        JPanel showtimesCard = new JPanel(new BorderLayout(0, 14));
        showtimesCard.setBackground(Color.WHITE);
        showtimesCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(20, 24, 20, 24)
        ));
```

```
        JPanel showHeader = new JPanel();
        showHeader.setLayout(new BoxLayout(showHeader, BoxLayout.Y_AXIS));
        showHeader.setOpaque(false);

        JLabel showTitle = new JLabel("Available Showtimes & Screens");
        showTitle.setFont(UIConstants.FONT_TITLE);
        showTitle.setForeground(UIConstants.COLOR_TEXT_DARK);

        JLabel showSubtitle = new JLabel("Select your preferred screening session to choose seats in the
auditorium");
        showSubtitle.setFont(UIConstants.FONT_SMALL);
        showSubtitle.setForeground(UIConstants.COLOR_TEXT_MUTED);

        showHeader.add(showTitle);
        showHeader.add(Box.createVerticalStrut(2));
        showHeader.add(showSubtitle);
        showtimesCard.add(showHeader, BorderLayout.NORTH);

        showsContainer = new JPanel(new FlowLayout(FlowLayout.LEFT, 16, 16));
        showsContainer.setOpaque(false);
        showtimesCard.add(showsContainer, BorderLayout.CENTER);

        mainContent.add(showtimesCard);

        JScrollPane scrollPane = new JScrollPane(mainContent);
        scrollPane.setBorder(null);
        scrollPane.setHorizontalScrollBarPolicy(ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);
        scrollPane.getVerticalScrollBar().setUnitIncrement(16);
        add(scrollPane, BorderLayout.CENTER);
    }

    public void setMovie(Movie movie) {
        this.currentMovie = movie;

        posterLabel.setIcon(UIConstants.getMoviePoster(movie.getId(), 200, 290));

        titleLabel.setText(movie.getTitle());

        badgesPanel.removeAll();
        badgesPanel.add(UIConstants.createBadge(movie.getGenre(), new Color(238, 242, 255),
UIConstants.COLOR_ACCENT));
        badgesPanel.add(UIConstants.createBadge(movie.getCertification(), new Color(254, 243, 199), new
Color(180, 83, 9)));
        badgesPanel.add(UIConstants.createBadge(movie.getLanguage(), new Color(241, 245, 249),
UIConstants.COLOR_TEXT_DARK));

        durationLabel.setText("⏱ Runtime: " + movie.getDurationMinutes() + " minutes (" +
(movie.getDurationMinutes() / 60) + "h " + (movie.getDurationMinutes() % 60) + "m)");
        synopsisArea.setText(movie.getDescription());

        showsContainer.removeAll();
        List<Show> movieShows = showService.getShowsForMovie(movie.getId());

        if (movieShows.isEmpty()) {
            JLabel noShows = new JLabel("No scheduled screenings currently available for this title.");
            noShows.setFont(UIConstants.FONT_REGULAR);
            noShows.setForeground(UIConstants.COLOR_TEXT_MUTED);
            showsContainer.add(noShows);
        } else {
            for (Show show : movieShows) {
                showsContainer.add(createShowtimeCard(show));
            }
        }

        revalidate();
        repaint();
```

```
    }

    private JPanel createShowtimeCard(Show show) {
        JPanel card = new JPanel(new BorderLayout(10, 10));
        card.setPreferredSize(new Dimension(280, 140));
        card.setBackground(new Color(248, 250, 252));
        card.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(203, 213, 225), 1, true),
                new EmptyBorder(12, 14, 12, 14)
        ));

        JPanel top = new JPanel();
        top.setLayout(new BoxLayout(top, BoxLayout.Y_AXIS));
        top.setOpaque(false);

        JLabel screenLabel = new JLabel(show.getScreen().getName());
        screenLabel.setFont(UIConstants.FONT_BOLD);
        screenLabel.setForeground(UIConstants.COLOR_PRIMARY);

        JLabel timeLabel = new JLabel("🖩 " +
show.getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER));
        timeLabel.setFont(UIConstants.FONT_REGULAR);
        timeLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        JLabel priceLabel = new JLabel(String.format("Base Ticket: $%.2f", show.getBasePrice()));
        priceLabel.setFont(UIConstants.FONT_SMALL);
        priceLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);

        int avail = show.getAvailableSeatsCount();
        int total = show.getTotalSeatsCount();
        Color availColor = (avail > 5) ? UIConstants.COLOR_SUCCESS : UIConstants.COLOR_WARNING;
        JLabel seatLabel = new JLabel("● " + avail + " / " + total + " seats available");
        seatLabel.setFont(UIConstants.FONT_SMALL);
        seatLabel.setForeground(availColor);

        top.add(screenLabel);
        top.add(Box.createVerticalStrut(3));
        top.add(timeLabel);
        top.add(Box.createVerticalStrut(3));
        top.add(priceLabel);
        top.add(Box.createVerticalStrut(3));
        top.add(seatLabel);

        card.add(top, BorderLayout.CENTER);

        JButton selectBtn = new JButton("Select Seats →");
        selectBtn.setFont(UIConstants.FONT_BOLD);
        selectBtn.setBackground(UIConstants.COLOR_ACCENT);
        selectBtn.setForeground(Color.WHITE);
        selectBtn.setFocusPainted(false);
        selectBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        selectBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(6, 12, 6, 12)
        ));
        selectBtn.addActionListener(e -> parent.navigateToSeatSelection(show));
        card.add(selectBtn, BorderLayout.SOUTH);

        return card;
    }
}
```

File: MoviePanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
```

```
import java.awt.*;
import java.util.List;

public class MoviePanel extends JPanel {
    private final MainFrame parent;
    private final MovieService movieService;

    private JTextField searchField;
    private JComboBox<String> genreFilter;
    private JPanel moviesGridPanel;
    private JLabel resultsCountLabel;

    public MoviePanel(MainFrame parent, MovieService movieService) {
        this.parent = parent;
        this.movieService = movieService;
        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel filterCard = new JPanel(new BorderLayout(16, 12));
        filterCard.setBackground(Color.WHITE);
        filterCard.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(16, 20, 16, 20)
        ));

        JPanel titlePanel = new JPanel();
        titlePanel.setLayout(new BoxLayout(titlePanel, BoxLayout.Y_AXIS));
        titlePanel.setOpaque(false);

        JLabel sectionTitle = new JLabel("Now Showing in Theaters");
        sectionTitle.setFont(UIConstants.FONT_TITLE);
        sectionTitle.setForeground(UIConstants.COLOR_TEXT_DARK);

        JLabel sectionSubtitle = new JLabel("Browse active movies, select showtimes, and book seats in
real time");
        sectionSubtitle.setFont(UIConstants.FONT_SMALL);
        sectionSubtitle.setForeground(UIConstants.COLOR_TEXT_MUTED);

        titlePanel.add(sectionTitle);
        titlePanel.add(Box.createVerticalStrut(3));
        titlePanel.add(sectionSubtitle);

        JPanel controlsPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 0));
        controlsPanel.setOpaque(false);

        JLabel searchLabel = new JLabel("Search:");
        searchLabel.setFont(UIConstants.FONT_BOLD);
        searchLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        searchField = new JTextField(15);
        searchField.setFont(UIConstants.FONT_REGULAR);
        searchField.setToolTipText("Filter by title or keywords");

        JLabel genreLabel = new JLabel("Genre:");
        genreLabel.setFont(UIConstants.FONT_BOLD);
        genreLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        genreFilter = new JComboBox<>(new String[]{"All", "Sci-Fi", "Action", "Comedy", "Thriller",
"Drama", "Animation", "Romance"});
        genreFilter.setFont(UIConstants.FONT_REGULAR);
```

```
        JButton searchBtn = new JButton("Filter");
        searchBtn.setFont(UIConstants.FONT_BOLD);
        searchBtn.setBackground(UIConstants.COLOR_ACCENT);
        searchBtn.setForeground(Color.WHITE);
        searchBtn.setFocusPainted(false);
        searchBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        searchBtn.addActionListener(e -> filterMovies());

        JButton resetBtn = new JButton("Reset");
        resetBtn.setFont(UIConstants.FONT_REGULAR);
        resetBtn.setBackground(new Color(241, 245, 249));
        resetBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        resetBtn.setFocusPainted(false);
        resetBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        resetBtn.addActionListener(e -> {
            searchField.setText("");
            genreFilter.setSelectedIndex(0);
            filterMovies();
        });

        searchField.addActionListener(e -> filterMovies());

        controlsPanel.add(searchLabel);
        controlsPanel.add(searchField);
        controlsPanel.add(genreLabel);
        controlsPanel.add(genreFilter);
        controlsPanel.add(searchBtn);
        controlsPanel.add(resetBtn);

        filterCard.add(titlePanel, BorderLayout.WEST);
        filterCard.add(controlsPanel, BorderLayout.EAST);
        add(filterCard, BorderLayout.NORTH);

        moviesGridPanel = new JPanel();
        moviesGridPanel.setLayout(new GridLayout(0, 3, 18, 18));
        moviesGridPanel.setBackground(UIConstants.COLOR_BG_LIGHT);

        JPanel wrapper = new JPanel(new BorderLayout());
        wrapper.setBackground(UIConstants.COLOR_BG_LIGHT);
        wrapper.add(moviesGridPanel, BorderLayout.NORTH);

        JScrollPane scrollPane = new JScrollPane(wrapper);
        scrollPane.setBorder(null);
        scrollPane.setHorizontalScrollBarPolicy(ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);
        scrollPane.getVerticalScrollBar().setUnitIncrement(16);
        add(scrollPane, BorderLayout.CENTER);

        resultsCountLabel = new JLabel("Loading movies...");
        resultsCountLabel.setFont(UIConstants.FONT_SMALL);
        resultsCountLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);
        resultsCountLabel.setBorder(new EmptyBorder(6, 4, 0, 0));
        add(resultsCountLabel, BorderLayout.SOUTH);

        refreshList();
    }

    public void refreshList() {
        filterMovies();
    }

    private void filterMovies() {
        String query = searchField.getText().trim();
        String genre = (String) genreFilter.getSelectedItem();

        List<Movie> matched = movieService.search(query, genre);
```

```
        moviesGridPanel.removeAll();

        if (matched.isEmpty()) {
            moviesGridPanel.setLayout(new BorderLayout());
            JLabel noData = new JLabel("No movies found matching your search criteria.",
SwingConstants.CENTER);
            noData.setFont(UIConstants.FONT_HEADING);
            noData.setForeground(UIConstants.COLOR_TEXT_MUTED);
            noData.setBorder(new EmptyBorder(80, 0, 80, 0));
            moviesGridPanel.add(noData, BorderLayout.CENTER);
        } else {
            moviesGridPanel.setLayout(new GridLayout(0, 3, 18, 18));
            for (Movie m : matched) {
                moviesGridPanel.add(createMovieCard(m));
            }
        }

        resultsCountLabel.setText("Showing " + matched.size() + " featured title(s)");
        moviesGridPanel.revalidate();
        moviesGridPanel.repaint();
    }

    private JPanel createMovieCard(Movie movie) {
        JPanel card = new JPanel(new BorderLayout(12, 10));
        card.setBackground(Color.WHITE);
        card.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(14, 14, 14, 14)
        ));

        ImageIcon posterIcon = UIConstants.getMoviePoster(movie.getId(), 110, 160);
        JLabel posterLabel = new JLabel(posterIcon);
        posterLabel.setBorder(BorderFactory.createLineBorder(new Color(226, 232, 240), 1, true));
        card.add(posterLabel, BorderLayout.WEST);

        JPanel detailsPanel = new JPanel();
        detailsPanel.setLayout(new BoxLayout(detailsPanel, BoxLayout.Y_AXIS));
        detailsPanel.setOpaque(false);

        JLabel titleLabel = new JLabel("<html><b>" + movie.getTitle() + "</b></html>");
        titleLabel.setFont(UIConstants.FONT_HEADING);
        titleLabel.setForeground(UIConstants.COLOR_TEXT_DARK);
        titleLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

        JPanel badgesPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 0));
        badgesPanel.setOpaque(false);
        badgesPanel.setAlignmentX(Component.LEFT_ALIGNMENT);

        JLabel genreBadge = UIConstants.createBadge(movie.getGenre(), new Color(238, 242, 255),
UIConstants.COLOR_ACCENT);
        JLabel certBadge = UIConstants.createBadge(movie.getCertification(), new Color(254, 243, 199), new
Color(180, 83, 9));
        badgesPanel.add(genreBadge);
        badgesPanel.add(certBadge);

        JLabel metaLabel = new JLabel("⏱ " + movie.getDurationMinutes() + " mins  •  " +
movie.getLanguage());
        metaLabel.setFont(UIConstants.FONT_SMALL);
        metaLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);
        metaLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

        String desc = movie.getDescription();
        if (desc.length() > 85) desc = desc.substring(0, 82) + "...";
        JLabel descLabel = new JLabel("<html><body style='width: 170px; color: #64748B; font-size:
10px;'>" + desc + "</body></html>");
        descLabel.setAlignmentX(Component.LEFT_ALIGNMENT);
```

```
        detailsPanel.add(titleLabel);
        detailsPanel.add(Box.createVerticalStrut(6));
        detailsPanel.add(badgesPanel);
        detailsPanel.add(Box.createVerticalStrut(6));
        detailsPanel.add(metaLabel);
        detailsPanel.add(Box.createVerticalStrut(6));
        detailsPanel.add(descLabel);

        card.add(detailsPanel, BorderLayout.CENTER);

        JButton actionBtn = new JButton("Book Tickets  →");
        actionBtn.setFont(UIConstants.FONT_BOLD);
        actionBtn.setBackground(UIConstants.COLOR_ACCENT);
        actionBtn.setForeground(Color.WHITE);
        actionBtn.setFocusPainted(false);
        actionBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        actionBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(new Color(67, 56, 202), 1, true),
                new EmptyBorder(8, 12, 8, 12)
        ));
        actionBtn.addActionListener(e -> parent.navigateToMovieDetails(movie));

        card.add(actionBtn, BorderLayout.SOUTH);
        return card;
    }
}
```

File: MovieService.java | Package: (default)

```
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class MovieService implements Searchable<Movie> {
    private final BookingManager manager;

    public MovieService(BookingManager manager) {
        this.manager = manager;
    }

    @Override
    public List<Movie> search(String query) {
        List<Movie> results = new ArrayList<>();
        for (Movie movie : manager.getMovies()) {
            if (movie.matches(query)) results.add(movie);
        }
        return results;
    }

    public List<Movie> search(String title, String genre) {
        List<Movie> results = new ArrayList<>();
        for (Movie movie : manager.getMovies()) {
            if (movie.matches(title, genre)) results.add(movie);
        }
        return results;
    }

    public synchronized void addMovie(Movie movie) throws ValidationException {
        if (movie == null) throw new ValidationException("Movie cannot be null.");
        if (movie.getTitle() == null || movie.getTitle().trim().isEmpty()) {
            throw new ValidationException("Movie title cannot be empty.");
        }
        if (movie.getDurationMinutes() <= 0) {
            throw new ValidationException("Duration must be a positive integer.");
        }
```

```
        try {
            manager.getDbManager().insertMovie(movie);
            manager.getMovies().add(movie);
            manager.logAudit("Added movie: " + movie.getTitle() + " (" + movie.getId() + ")");
        } catch (SQLException e) {
            throw new ValidationException("Database error while adding movie: " + e.getMessage());
        }
    }

    public synchronized void updateMovie(String id, String title, String genre,
                                int duration, String language, String cert, String desc)
            throws ValidationException {
        Movie movie = null;
        for (Movie m : manager.getMovies()) {
            if (m.getId().equalsIgnoreCase(id)) {
                movie = m;
                break;
            }
        }
        if (movie == null) throw new ValidationException("Movie with ID " + id + " not found.");
        if (title == null || title.trim().isEmpty()) throw new ValidationException("Title cannot be
empty.");
        if (duration <= 0) throw new ValidationException("Duration must be positive.");

        movie.setTitle(title.trim());
        movie.setGenre(genre.trim());
        movie.setDurationMinutes(duration);
        movie.setLanguage(language.trim());
        movie.setCertification(cert.trim());
        movie.setDescription(desc.trim());

        try {
            manager.getDbManager().updateMovie(movie);
            manager.logAudit("Updated movie: " + movie.getTitle() + " (" + id + ")");
        } catch (SQLException e) {
            throw new ValidationException("Database error while updating movie: " + e.getMessage());
        }
    }

    public synchronized void removeMovie(String movieId) throws ValidationException {
        for (Show show : manager.getShowsForMovie(movieId)) {
            for (Booking booking : manager.getBookings()) {
                if (booking.getShow().getId().equals(show.getId())) {
                    throw new ValidationException("Cannot remove movie: Shows associated with this movie
have active or past booking records.");
                }
            }
        }

        try {
            manager.getDbManager().deleteMovie(movieId);
            manager.getShows().removeIf(s -> s.getMovie().getId().equals(movieId));
            boolean removed = manager.getMovies().removeIf(m -> m.getId().equals(movieId));
            if (!removed) {
                throw new ValidationException("Movie ID not found: " + movieId);
            }
            manager.logAudit("Safely removed movie ID: " + movieId);
        } catch (SQLException e) {
            throw new ValidationException("Database error while removing movie: " + e.getMessage());
        }
    }
}
```

File: PaymentGateway.java | Package: (default)

```
public interface PaymentGateway {
    PaymentTransaction processPayment(Booking booking, double amount, String notes)
```

```
            throws PaymentProcessingException;
}
```

File: PaymentMethod.java | Package: (default)

```
public abstract class PaymentMethod {
    private final String methodName;

    public PaymentMethod(String methodName) {
        this.methodName = methodName;
    }

    public String getMethodName() {
        return methodName;
    }

    public abstract boolean executeTransaction(double amount) throws PaymentProcessingException;

    @Override
    public String toString() {
        return methodName;
    }
}
```

File: PaymentProcessingException.java | Package: (default)

```
public class PaymentProcessingException extends CineBookException {
    public PaymentProcessingException(String message) { super(message); }
}
```

File: PaymentProcessor.java | Package: (default)

```
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.util.UUID;

public class PaymentProcessor {
    private final BookingManager manager;
    private final BookingService bookingService;

    public PaymentProcessor(BookingManager manager, BookingService bookingService) {
        this.manager = manager;
        this.bookingService = bookingService;
    }

    public synchronized PaymentTransaction processSimulatedPayment(
            Booking booking,
            PaymentMethod paymentMethod,
            String simulationMode
    ) throws PaymentProcessingException, InvalidBookingStateException, SeatUnavailableException {

        if (booking == null) {
            throw new PaymentProcessingException("No booking associated with payment.");
        }

        if (booking.getBookingStatus() == BookingStatus.CONFIRMED) {
            throw new InvalidBookingStateException("Booking " + booking.getId() + " is already confirmed.
Duplicate payment prevented.");
        }

        if (booking.isExpired()) {
            bookingService.cancelPendingReservation(booking);
            throw new PaymentProcessingException("Checkout failed: Your 5-minute reservation window has
expired. Please select seats again.");
        }

        String txnId = "TXN-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
```

```
        if ("USER_CANCEL".equalsIgnoreCase(simulationMode)) {
            bookingService.cancelPendingReservation(booking);
            PaymentTransaction txn = new PaymentTransaction(
                    txnId, booking.getId(), paymentMethod.getMethodName(),
                    booking.getTotalAmount(), LocalDateTime.now(),
                    PaymentStatus.CANCELLED, "USER-ABORT"
            );
            manager.getTransactions().add(txn);
            try {
                manager.getDbManager().insertTransaction(txn);
            } catch (SQLException ignored) {}
            manager.logAudit("Payment cancelled by user for booking " + booking.getId());
            return txn;
        }

        if ("FORCE_FAIL".equalsIgnoreCase(simulationMode)) {
            booking.markPaymentFailed();
            bookingService.cancelPendingReservation(booking);
            PaymentTransaction txn = new PaymentTransaction(
                    txnId, booking.getId(), paymentMethod.getMethodName(),
                    booking.getTotalAmount(), LocalDateTime.now(),
                    PaymentStatus.FAILED, "ERR-DECLINED"
            );
            manager.getTransactions().add(txn);
            try {
                manager.getDbManager().insertTransaction(txn);
            } catch (SQLException ignored) {}
            manager.logAudit("Payment forcefully failed for booking " + booking.getId());
            throw new PaymentProcessingException("Simulated payment was declined by the issuer.");
        }

        boolean authorized = paymentMethod.executeTransaction(booking.getTotalAmount());
        if (!authorized) {
            booking.markPaymentFailed();
            bookingService.cancelPendingReservation(booking);
            throw new PaymentProcessingException("Payment authorization failed.");
        }

        bookingService.confirmBooking(booking);

        PaymentTransaction successTxn = new PaymentTransaction(
                txnId, booking.getId(), paymentMethod.getMethodName(),
                booking.getTotalAmount(), LocalDateTime.now(),
                PaymentStatus.SUCCESS, "AUTH-" + UUID.randomUUID().toString().substring(0,
6).toUpperCase()
        );
        manager.getTransactions().add(successTxn);
        try {
            manager.getDbManager().insertTransaction(successTxn);
        } catch (SQLException e) {
            System.err.println("[JDBC WARNING] Failed to persist transaction: " + e.getMessage());
        }

        manager.logAudit("Payment successful (" + successTxn.getId() + ") for booking " +
booking.getId());
        return successTxn;
    }
}
```

File: PaymentStatus.java | Package: (default)

```
public enum PaymentStatus {
    NOT_STARTED("Not Started"),
    PENDING("Pending"),
    SUCCESS("Success"),
    FAILED("Failed"),
```

```
    CANCELLED("Cancelled"),
    REFUNDED("Refunded");

    private final String displayName;

    PaymentStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

File: PaymentTransaction.java | Package: (default)

```
import java.time.LocalDateTime;

public class PaymentTransaction {
    private final String id;
    private final String bookingId;
    private final String paymentMethodName;
    private final double amount;
    private final LocalDateTime timestamp;
    private final PaymentStatus paymentStatus;
    private final String gatewayReference;

    public PaymentTransaction(String id, String bookingId, String paymentMethodName,
                              double amount, LocalDateTime timestamp,
                              PaymentStatus paymentStatus, String gatewayReference) {
        this.id = id;
        this.bookingId = bookingId;
        this.paymentMethodName = paymentMethodName;
        this.amount = Math.round(amount * 100.0) / 100.0;
        this.timestamp = timestamp;
        this.paymentStatus = paymentStatus;
        this.gatewayReference = gatewayReference;
    }

    public String getId() { return id; }
    public String getBookingId() { return bookingId; }
    public String getPaymentMethodName() { return paymentMethodName; }
    public double getAmount() { return amount; }
    public LocalDateTime getTimestamp() { return timestamp; }
    public PaymentStatus getPaymentStatus() { return paymentStatus; }
    public String getGatewayReference() { return gatewayReference; }

    @Override
    public String toString() {
        return "Txn #" + id + " (" + paymentStatus.getDisplayName() + ") $" +
                String.format("%.2f", amount) + " via " + paymentMethodName + " [" + gatewayReference +
"]";
    }
}
```

File: Screen.java | Package: (default)

```
public class Screen {
    private final String id;
    private final String name;
    private final int rows;
    private final int columns;

    public Screen(String id, String name, int rows, int columns) {
        this.id = id;
        this.name = name;
        this.rows = rows;
```

```
        this.columns = columns;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public int getRows() { return rows; }
    public int getColumns() { return columns; }
    public int getTotalCapacity() { return rows * columns; }

    @Override
    public String toString() {
        return name + " (" + getTotalCapacity() + " seats)";
    }
}
```

File: Searchable.java | Package: (default)

```
import java.util.List;

public interface Searchable<T> {
    List<T> search(String query);
}
```

File: Seat.java | Package: (default)

```
import java.time.LocalDateTime;

public class Seat {
    private final String id;
    private final int rowNumber;
    private final int colNumber;
    private final SeatCategory category;
    private SeatStatus status;
    private LocalDateTime reservedUntil;
    private String reservationBookingId;

    public Seat(String id, int rowNumber, int colNumber, SeatCategory category) {
        this.id = id;
        this.rowNumber = rowNumber;
        this.colNumber = colNumber;
        this.category = category;
        this.status = SeatStatus.AVAILABLE;
    }

    public String getId() { return id; }
    public int getRowNumber() { return rowNumber; }
    public int getColNumber() { return colNumber; }
    public SeatCategory getCategory() { return category; }
    public SeatStatus getStatus() { return status; }
    public void setStatus(SeatStatus status) { this.status = status; }
    public LocalDateTime getReservedUntil() { return reservedUntil; }
    public String getReservationBookingId() { return reservationBookingId; }

    public synchronized boolean isAvailable() {
        if (status == SeatStatus.RESERVED && reservedUntil != null &&
LocalDateTime.now().isAfter(reservedUntil)) {
            this.status = SeatStatus.AVAILABLE;
            this.reservedUntil = null;
            this.reservationBookingId = null;
            return true;
        }
        return status == SeatStatus.AVAILABLE;
    }

    public synchronized void reserve(String bookingId, int durationMinutes) throws
SeatUnavailableException {
        if (!isAvailable()) {
```

```
            throw new SeatUnavailableException("Seat " + id + " is not available for reservation.");
        }
        this.status = SeatStatus.RESERVED;
        this.reservedUntil = LocalDateTime.now().plusMinutes(durationMinutes);
        this.reservationBookingId = bookingId;
    }

    public synchronized void confirmBooking(String bookingId) throws SeatUnavailableException {
        if (status == SeatStatus.BOOKED) {
            throw new SeatUnavailableException("Seat " + id + " is already booked.");
        }
        this.status = SeatStatus.BOOKED;
        this.reservationBookingId = bookingId;
        this.reservedUntil = null;
    }

    public synchronized void release(String bookingId) {
        if (this.reservationBookingId == null || this.reservationBookingId.equals(bookingId) ||
this.status != SeatStatus.BOOKED) {
            this.status = SeatStatus.AVAILABLE;
            this.reservedUntil = null;
            this.reservationBookingId = null;
        }
    }

    @Override
    public String toString() {
        return id + " (" + category.getLabel() + " - " + status.getDisplayName() + ")";
    }
}
```

File: SeatCategory.java | Package: (default)

```
public enum SeatCategory {
    STANDARD("Standard", 1.0),
    PREMIUM("Premium", 1.3),
    VIP("VIP", 1.6);

    private final String label;
    private final double priceMultiplier;

    SeatCategory(String label, double priceMultiplier) {
        this.label = label;
        this.priceMultiplier = priceMultiplier;
    }

    public String getLabel() {
        return label;
    }

    public double getPriceMultiplier() {
        return priceMultiplier;
    }
}
```

File: SeatManager.java | Package: (default)

```
import java.time.LocalDateTime;
import java.util.List;

public class SeatManager {
    public synchronized void expireOutdatedReservations(Show show) {
        if (show == null) return;
        for (Seat[] row : show.getSeats()) {
            for (Seat seat : row) {
                if (seat.getStatus() == SeatStatus.RESERVED &&
                        seat.getReservedUntil() != null &&
```

```
                    LocalDateTime.now().isAfter(seat.getReservedUntil())) {
                seat.release(seat.getReservationBookingId());
            }
        }
    }
}

public synchronized void releaseSeats(Show show, List<String> seatIds, String bookingId) {
    if (show == null || seatIds == null) return;
    for (String seatId : seatIds) {
        Seat seat = show.findSeat(seatId);
        if (seat != null) {
            seat.release(bookingId);
        }
    }
}

public synchronized void confirmSeats(Show show, List<String> seatIds, String bookingId)
        throws SeatUnavailableException {
    for (String seatId : seatIds) {
        Seat seat = show.findSeat(seatId);
        if (seat == null) throw new SeatUnavailableException("Seat " + seatId + " not found.");
        seat.confirmBooking(bookingId);
    }
}
}
```

File: SeatSelectionPanel.java | Package: (default)

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.util.*;
import java.util.List;

public class SeatSelectionPanel extends JPanel {
    private final MainFrame parent;
    private final BookingService bookingService;

    private Show currentShow;
    private final Set<String> selectedSeatIds = new HashSet<>();
    private final Map<String, JButton> seatButtons = new HashMap<>();

    private JLabel showInfoLabel;
    private JLabel selectedSeatsLabel;
    private JLabel priceBreakdownLabel;
    private JButton checkoutButton;
    private JPanel gridPanel;

    public SeatSelectionPanel(MainFrame parent, BookingService bookingService) {
        this.parent = parent;
        this.bookingService = bookingService;
        setLayout(new BorderLayout(0, 16));
        setBackground(UIConstants.COLOR_BG_LIGHT);
        setBorder(new EmptyBorder(20, 28, 20, 28));

        initializeUI();
    }

    private void initializeUI() {

        JPanel topBar = new JPanel(new BorderLayout(16, 0));
        topBar.setBackground(Color.WHITE);
        topBar.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(12, 18, 12, 18)
```

```
        ));

        JButton backBtn = new JButton("← Back to Details");
        backBtn.setFont(UIConstants.FONT_BOLD);
        backBtn.setBackground(new Color(241, 245, 249));
        backBtn.setForeground(UIConstants.COLOR_TEXT_DARK);
        backBtn.setFocusPainted(false);
        backBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        backBtn.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(6, 14, 6, 14)
        ));
        backBtn.addActionListener(e -> {
            if (currentShow != null) parent.navigateToMovieDetails(currentShow.getMovie());
            else parent.showMoviesScreen();
        });

        showInfoLabel = new JLabel("Select your seats");
        showInfoLabel.setFont(UIConstants.FONT_HEADING);
        showInfoLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        topBar.add(backBtn, BorderLayout.WEST);
        topBar.add(showInfoLabel, BorderLayout.CENTER);
        add(topBar, BorderLayout.NORTH);

        JPanel centerPanel = new JPanel(new BorderLayout(0, 16));
        centerPanel.setBackground(Color.WHITE);
        centerPanel.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(20, 24, 20, 24)
        ));

        JPanel screenIndicator = new JPanel() {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2 = (Graphics2D) g;
                g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

                GradientPaint gp = new GradientPaint(0, 0, new Color(79, 70, 229, 60), 0, getHeight(), new
Color(241, 245, 249, 10));
                g2.setPaint(gp);
                g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);

                g2.setColor(UIConstants.COLOR_ACCENT);
                g2.setStroke(new BasicStroke(3));
                g2.drawLine(20, 6, getWidth() - 20, 6);
            }
        };
        screenIndicator.setOpaque(false);
        screenIndicator.setPreferredSize(new Dimension(650, 40));

        JLabel screenText = new JLabel("CINEMA SCREEN (ALL EYES THIS DIRECTION)");
        screenText.setFont(UIConstants.FONT_BADGE);
        screenText.setForeground(UIConstants.COLOR_ACCENT);
        screenIndicator.add(screenText);

        JPanel screenWrapper = new JPanel(new FlowLayout(FlowLayout.CENTER));
        screenWrapper.setOpaque(false);
        screenWrapper.add(screenIndicator);
        centerPanel.add(screenWrapper, BorderLayout.NORTH);

        gridPanel = new JPanel();
        gridPanel.setOpaque(false);

        JPanel gridWrapper = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 10));
```

```
        gridWrapper.setOpaque(false);
        gridWrapper.add(gridPanel);

        JScrollPane scrollGrid = new JScrollPane(gridWrapper);
        scrollGrid.setBorder(null);
        scrollGrid.setOpaque(false);
        scrollGrid.getViewport().setOpaque(false);
        centerPanel.add(scrollGrid, BorderLayout.CENTER);

        JPanel legendPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 24, 6));
        legendPanel.setOpaque(false);
        legendPanel.add(createLegendItem("Available", UIConstants.COLOR_SEAT_AVAILABLE,
UIConstants.COLOR_SEAT_AVAILABLE_BORDER));
        legendPanel.add(createLegendItem("Selected", UIConstants.COLOR_SEAT_SELECTED, new Color(5, 150,
105)));
        legendPanel.add(createLegendItem("In Checkout", UIConstants.COLOR_SEAT_RESERVED, new Color(250,
204, 21)));
        legendPanel.add(createLegendItem("Booked", UIConstants.COLOR_SEAT_BOOKED, new Color(148, 163,
184)));
        centerPanel.add(legendPanel, BorderLayout.SOUTH);

        add(centerPanel, BorderLayout.CENTER);

        JPanel bottomBar = new JPanel(new BorderLayout(16, 0));
        bottomBar.setBackground(Color.WHITE);
        bottomBar.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_BORDER, 1, true),
                new EmptyBorder(16, 20, 16, 20)
        ));

        JPanel summaryInfo = new JPanel();
        summaryInfo.setLayout(new BoxLayout(summaryInfo, BoxLayout.Y_AXIS));
        summaryInfo.setOpaque(false);

        selectedSeatsLabel = new JLabel("Selected Seats: None");
        selectedSeatsLabel.setFont(UIConstants.FONT_HEADING);
        selectedSeatsLabel.setForeground(UIConstants.COLOR_TEXT_DARK);

        priceBreakdownLabel = new JLabel("Subtotal: $0.00  |  Tax (18% GST): $0.00  |  Total Payable:
$0.00");
        priceBreakdownLabel.setFont(UIConstants.FONT_REGULAR);
        priceBreakdownLabel.setForeground(UIConstants.COLOR_TEXT_MUTED);

        summaryInfo.add(selectedSeatsLabel);
        summaryInfo.add(Box.createVerticalStrut(3));
        summaryInfo.add(priceBreakdownLabel);

        checkoutButton = new JButton("Proceed to Checkout  →");
        checkoutButton.setFont(UIConstants.FONT_BOLD);
        checkoutButton.setBackground(UIConstants.COLOR_ACCENT);
        checkoutButton.setForeground(Color.WHITE);
        checkoutButton.setFocusPainted(false);
        checkoutButton.setEnabled(false);
        checkoutButton.setCursor(new Cursor(Cursor.HAND_CURSOR));
        checkoutButton.setBorder(BorderFactory.createCompoundBorder(
                new LineBorder(UIConstants.COLOR_ACCENT_HOVER, 1, true),
                new EmptyBorder(10, 20, 10, 20)
        ));
        checkoutButton.addActionListener(e -> proceedToCheckout());

        bottomBar.add(summaryInfo, BorderLayout.CENTER);
        bottomBar.add(checkoutButton, BorderLayout.EAST);
        add(bottomBar, BorderLayout.SOUTH);
    }

    private JPanel createLegendItem(String label, Color color, Color border) {
```

```
        JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        p.setOpaque(false);
        JPanel box = new JPanel();
        box.setPreferredSize(new Dimension(20, 20));
        box.setBackground(color);
        box.setBorder(new LineBorder(border, 1, true));
        JLabel lbl = new JLabel(label);
        lbl.setFont(UIConstants.FONT_SMALL);
        lbl.setForeground(UIConstants.COLOR_TEXT_DARK);
        p.add(box);
        p.add(lbl);
        return p;
    }

    public void setShow(Show show) {
        this.currentShow = show;
        this.selectedSeatIds.clear();
        this.seatButtons.clear();

        showInfoLabel.setText(show.getMovie().getTitle() + "  •  " +
                show.getScreen().getName() + "  •  " +
show.getShowDateTime().format(UIConstants.DATE_TIME_FORMATTER));

        buildSeatGrid();
        updateSummary();
    }

    private void buildSeatGrid() {
        gridPanel.removeAll();
        if (currentShow == null) return;

        Seat[][] seats = currentShow.getSeats();
        int rows = currentShow.getScreen().getRows();
        int cols = currentShow.getScreen().getColumns();

        gridPanel.setLayout(new GridLayout(rows, cols + 1, 10, 10));

        for (int r = 0; r < rows; r++) {
            char rowChar = (char) ('A' + r);
            JLabel rowLbl = new JLabel(String.valueOf(rowChar), SwingConstants.CENTER);
            rowLbl.setFont(UIConstants.FONT_BOLD);
            rowLbl.setForeground(UIConstants.COLOR_TEXT_MUTED);
            rowLbl.setPreferredSize(new Dimension(26, 42));
            gridPanel.add(rowLbl);

            for (int c = 0; c < cols; c++) {
                Seat seat = seats[r][c];
                JButton seatBtn = new JButton(seat.getId());
                seatBtn.setFont(UIConstants.FONT_BOLD);
                seatBtn.setFocusPainted(false);
                seatBtn.setMargin(new Insets(2, 2, 2, 2));
                seatBtn.setPreferredSize(new Dimension(54, 42));
                seatBtn.setCursor(new Cursor(Cursor.HAND_CURSOR));

                updateSeatButtonAppearance(seatBtn, seat);

                seatBtn.addActionListener(e -> toggleSeatSelection(seat, seatBtn));

                seatBtns.put(seat.getId(), seatBtn);
                gridPanel.add(seatBtn);
            }
        }

        gridPanel.revalidate();
        gridPanel.repaint();
    }
```

```
    private void updateSeatButtonAppearance(JButton btn, Seat seat) {
        if (seat.getStatus() == SeatStatus.BOOKED) {
            btn.setBackground(UIConstants.COLOR_SEAT_BOOKED);
            btn.setForeground(new Color(148, 163, 184));
            btn.setBorder(new LineBorder(new Color(203, 213, 225), 1, true));
            btn.setEnabled(false);
            btn.setToolTipText(seat.getId() + " - Already Booked");
        } else if (seat.getStatus() == SeatStatus.RESERVED && !seat.isAvailable()) {
            btn.setBackground(UIConstants.COLOR_SEAT_RESERVED);
            btn.setForeground(new Color(133, 77, 14));
            btn.setBorder(new LineBorder(new Color(250, 204, 21), 1, true));
            btn.setEnabled(false);
            btn.setToolTipText(seat.getId() + " - In Checkout Hold by Another Customer");
        } else if (selectedSeatIds.contains(seat.getId())) {
            btn.setBackground(UIConstants.COLOR_SEAT_SELECTED);
            btn.setForeground(Color.WHITE);
            btn.setBorder(new LineBorder(new Color(5, 150, 105), 2, true));
            btn.setEnabled(true);
            btn.setToolTipText(seat.getId() + " - Selected ($" + String.format("%.2f",
currentShow.calculateSeatPrice(seat)) + ")");
        } else {
            btn.setBackground(UIConstants.COLOR_SEAT_AVAILABLE);
            btn.setForeground(new Color(22, 101, 52));
            btn.setBorder(new LineBorder(UIConstants.COLOR_SEAT_AVAILABLE_BORDER, 1, true));
            btn.setEnabled(true);
            btn.setToolTipText(seat.getId() + " (" + seat.getCategory().getLabel() + ") - $" +
                    String.format("%.2f", currentShow.calculateSeatPrice(seat)));
        }
    }

    private void toggleSeatSelection(Seat seat, JButton btn) {
        if (selectedSeatIds.contains(seat.getId())) {
            selectedSeatIds.remove(seat.getId());
        } else {
            selectedSeatIds.add(seat.getId());
        }
        updateSeatButtonAppearance(btn, seat);
        updateSummary();
    }

    private void updateSummary() {
        if (selectedSeatIds.isEmpty()) {
            selectedSeatsLabel.setText("Selected Seats: None");
            priceBreakdownLabel.setText("Subtotal: $0.00  |  Tax (18% GST): $0.00  |  Total Payable:
$0.00");
            checkoutButton.setEnabled(false);
            checkoutButton.setBackground(new Color(203, 213, 225));
            return;
        }

        double subtotal = 0.0;
        List<String> sortedList = new ArrayList<>(selectedSeatIds);
        Collections.sort(sortedList);

        for (String sId : sortedList) {
            Seat seat = currentShow.findSeat(sId);
            if (seat != null) subtotal += currentShow.calculateSeatPrice(seat);
        }

        double tax = Math.round(subtotal * 0.18 * 100.0) / 100.0;
        double grandTotal = Math.round((subtotal + tax) * 100.0) / 100.0;

        selectedSeatsLabel.setText("Selected Seats: " + String.join(", ", sortedList) + " (" +
sortedList.size() + " seat(s))");
        priceBreakdownLabel.setText(String.format("Subtotal: $%.2f  |  Tax (18%% GST): $%.2f  |  Total
```

```
Payable: $%.2f",
                subtotal, tax, grandTotal));
        checkoutButton.setEnabled(true);
        checkoutButton.setBackground(UIConstants.COLOR_ACCENT);
    }

    private void proceedToCheckout() {
        if (selectedSeatIds.isEmpty() || currentShow == null) return;

        User currentUser = BookingManager.getInstance().getCurrentUser();
        if (!(currentUser instanceof Customer)) {
            JOptionPane.showMessageDialog(this,
                    "Only Customers can book tickets. Please log in with a Customer account (e.g. username
'user').",
                    "Customer Role Required", JOptionPane.WARNING_MESSAGE);
            return;
        }

        try {
            Customer customer = (Customer) currentUser;
            List<String> seatsToBook = new ArrayList<>(selectedSeatIds);
            Booking booking = bookingService.createPendingReservation(customer, currentShow, seatsToBook);
            parent.navigateToCheckout(booking);
        } catch (SeatUnavailableException | ValidationException ex) {
            JOptionPane.showMessageDialog(this,
                    "Could not reserve seats: " + ex.getMessage(),
                    "Reservation Conflict", JOptionPane.ERROR_MESSAGE);
            buildSeatGrid();
            updateSummary();
        }
    }
}
```

File: SeatStatus.java | Package: (default)

```
public enum SeatStatus {
    AVAILABLE("Available"),
    SELECTED("Selected"),
    RESERVED("Reserved"),
    BOOKED("Booked");

    private final String displayName;

    SeatStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

File: SeatUnavailableException.java | Package: (default)

```
public class SeatUnavailableException extends CineBookException {
    public SeatUnavailableException(String message) { super(message); }
}
```

File: Show.java | Package: (default)

```
import java.time.LocalDateTime;

public class Show {
    private final String id;
    private final Movie movie;
    private final Screen screen;
    private final LocalDateTime showDateTime;
```

```
    private final double basePrice;
    private final Seat[][] seats;

    public Show(String id, Movie movie, Screen screen, LocalDateTime showDateTime, double basePrice) {
        this.id = id;
        this.movie = movie;
        this.screen = screen;
        this.showDateTime = showDateTime;
        this.basePrice = basePrice;
        this.seats = new Seat[screen.getRows()][screen.getColumns()];
        initializeSeats();
    }

    private void initializeSeats() {
        for (int r = 0; r < screen.getRows(); r++) {
            char rowLetter = (char) ('A' + r);
            SeatCategory cat;
            if (r < 2) {
                cat = SeatCategory.STANDARD;
            } else if (r < 4) {
                cat = SeatCategory.PREMIUM;
            } else {
                cat = SeatCategory.VIP;
            }

            for (int c = 0; c < screen.getColumns(); c++) {
                String seatId = "" + rowLetter + (c + 1);
                seats[r][c] = new Seat(seatId, r + 1, c + 1, cat);
            }
        }
    }

    public String getId() { return id; }
    public Movie getMovie() { return movie; }
    public Screen getScreen() { return screen; }
    public LocalDateTime getShowDateTime() { return showDateTime; }
    public double getBasePrice() { return basePrice; }
    public Seat[][] getSeats() { return seats; }

    public Seat findSeat(String seatId) {
        if (seatId == null) return null;
        for (Seat[] row : seats) {
            for (Seat seat : row) {
                if (seat.getId().equalsIgnoreCase(seatId.trim())) {
                    return seat;
                }
            }
        }
        return null;
    }

    public double calculateSeatPrice(Seat seat) {
        if (seat == null) return basePrice;
        return Math.round(basePrice * seat.getCategory().getPriceMultiplier() * 100.0) / 100.0;
    }

    public int getAvailableSeatsCount() {
        int count = 0;
        for (Seat[] row : seats) {
            for (Seat seat : row) {
                if (seat.isAvailable()) count++;
            }
        }
        return count;
    }
```

```
    public int getTotalSeatsCount() {
        return screen.getTotalCapacity();
    }

    @Override
    public String toString() {
        return movie.getTitle() + " @ " + screen.getName() + " (" +
showDateTime.format(UIConstants.DATE_TIME_FORMATTER) + ")";
    }
}
```

File: ShowService.java | Package: (default)

```
import java.sql.SQLException;
import java.util.List;

public class ShowService {
    private final BookingManager manager;

    public ShowService(BookingManager manager) {
        this.manager = manager;
    }

    public List<Show> getAllShows() {
        return manager.getShows();
    }

    public List<Show> getShowsForMovie(String movieId) {
        return manager.getShowsForMovie(movieId);
    }

    public synchronized void addShow(Show show) throws ValidationException {
        if (show == null) throw new ValidationException("Show cannot be null.");
        if (show.getBasePrice() <= 0) throw new ValidationException("Base price must be greater than
zero.");

        try {
            manager.getDbManager().insertShow(show);
            manager.getShows().add(show);
            manager.logAudit("Added show " + show.getId() + " for movie " + show.getMovie().getTitle());
        } catch (SQLException e) {
            throw new ValidationException("Database error while adding show: " + e.getMessage());
        }
    }

    public synchronized void removeShow(String showId) throws ValidationException {
        for (Booking booking : manager.getBookings()) {
            if (booking.getShow().getId().equals(showId)) {
                throw new ValidationException("Cannot remove show: Booking records exist for Show ID " +
showId);
            }
        }

        try {
            manager.getDbManager().deleteShow(showId);
            boolean removed = manager.getShows().removeIf(s -> s.getId().equals(showId));
            if (!removed) throw new ValidationException("Show ID not found: " + showId);
            manager.logAudit("Safely removed show ID: " + showId);
        } catch (SQLException e) {
            throw new ValidationException("Database error while removing show: " + e.getMessage());
        }
    }
}
```

File: UIConstants.java | Package: (default)

```
import javax.swing.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;
import java.time.format.DateTimeFormatter;
import java.util.HashMap;
import java.util.Map;
import javax.imageio.ImageIO;

public final class UIConstants {
    private UIConstants() {}

    public static final DateTimeFormatter DATE_TIME_FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm");
    public static final DateTimeFormatter TIME_FORMATTER = DateTimeFormatter.ofPattern("hh:mm a");

    public static final Color COLOR_PRIMARY = new Color(15, 23, 42);
    public static final Color COLOR_PRIMARY_DARK = new Color(2, 6, 23);
    public static final Color COLOR_ACCENT = new Color(79, 70, 229);
    public static final Color COLOR_ACCENT_HOVER = new Color(67, 56, 202);
    public static final Color COLOR_SUCCESS = new Color(16, 185, 129);
    public static final Color COLOR_WARNING = new Color(245, 158, 11);
    public static final Color COLOR_DANGER = new Color(239, 68, 68);
    public static final Color COLOR_BG_LIGHT = new Color(248, 250, 252);
    public static final Color COLOR_CARD_BG = Color.WHITE;
    public static final Color COLOR_TEXT_DARK = new Color(15, 23, 42);
    public static final Color COLOR_TEXT_MUTED = new Color(100, 116, 139);
    public static final Color COLOR_BORDER = new Color(226, 232, 240);

    public static final Color COLOR_SEAT_AVAILABLE = new Color(220, 252, 231);
    public static final Color COLOR_SEAT_AVAILABLE_BORDER = new Color(134, 239, 172);
    public static final Color COLOR_SEAT_SELECTED = new Color(16, 185, 129);
    public static final Color COLOR_SEAT_RESERVED = new Color(254, 240, 138);
    public static final Color COLOR_SEAT_BOOKED = new Color(203, 213, 225);

    public static final Font FONT_HEADER_LOGO = new Font("Segoe UI", Font.BOLD, 22);
    public static final Font FONT_HERO = new Font("Segoe UI", Font.BOLD, 24);
    public static final Font FONT_TITLE = new Font("Segoe UI", Font.BOLD, 18);
    public static final Font FONT_HEADING = new Font("Segoe UI", Font.BOLD, 15);
    public static final Font FONT_REGULAR = new Font("Segoe UI", Font.PLAIN, 13);
    public static final Font FONT_BOLD = new Font("Segoe UI", Font.BOLD, 13);
    public static final Font FONT_SMALL = new Font("Segoe UI", Font.PLAIN, 11);
    public static final Font FONT_BADGE = new Font("Segoe UI", Font.BOLD, 11);

    private static final Map<String, ImageIcon> posterCache = new HashMap<>();

    public static ImageIcon getMoviePoster(String movieId, int width, int height) {
        String cacheKey = movieId + "_" + width + "x" + height;
        if (posterCache.containsKey(cacheKey)) {
            return posterCache.get(cacheKey);
        }

        File imgFile = new File("images" + File.separator + movieId + ".jpg");
        if (imgFile.exists()) {
            try {
                BufferedImage originalImage = ImageIO.read(imgFile);
                if (originalImage != null) {
                    Image scaled = originalImage.getScaledInstance(width, height, Image.SCALE_SMOOTH);
                    ImageIcon icon = new ImageIcon(scaled);
                    posterCache.put(cacheKey, icon);
                    return icon;
                }
            } catch (Exception e) {
                System.err.println("[POSTER ERROR] Could not load " + imgFile.getAbsolutePath() + ": " +
e.getMessage());
            }
```

```
        }

        BufferedImage placeholder = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);
        Graphics2D g2 = placeholder.createGraphics();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

        GradientPaint gp = new GradientPaint(0, 0, new Color(30, 41, 59), width, height, new Color(15, 23,
42));
        g2.setPaint(gp);
        g2.fillRect(0, 0, width, height);

        g2.setColor(COLOR_ACCENT);
        g2.drawRect(0, 0, width - 1, height - 1);

        g2.setColor(new Color(255, 255, 255, 180));
        g2.setFont(new Font("Segoe UI", Font.BOLD, Math.max(14, width / 6)));
        FontMetrics fm = g2.getFontMetrics();
        String label = (movieId != null ? movieId : "FILM");
        int textX = (width - fm.stringWidth(label)) / 2;
        int textY = (height / 2) + (fm.getAscent() / 4);
        g2.drawString(label, textX, textY);

        g2.dispose();
        ImageIcon fallbackIcon = new ImageIcon(placeholder);
        posterCache.put(cacheKey, fallbackIcon);
        return fallbackIcon;
    }

    public static JLabel createBadge(String text, Color bg, Color fg) {
        JLabel badge = new JLabel(text);
        badge.setFont(FONT_BADGE);
        badge.setForeground(fg);
        badge.setBackground(bg);
        badge.setOpaque(true);
        badge.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(fg.getRed(), fg.getGreen(), fg.getBlue(), 60), 1,
true),
                BorderFactory.createEmptyBorder(3, 8, 3, 8)
        ));
        return badge;
    }
}
```

File: User.java | Package: (default)

```
public abstract class User {
    private final String id;
    private String username;
    private String password;
    private String name;
    private String email;
    private final String role;

    public User(String id, String username, String password, String name, String email, String role) {
        this.id = id;
        this.username = username;
        this.password = password;
        this.name = name;
        this.email = email;
        this.role = role;
    }

    public User(String id, String name, String email, String role) {
        this(id, id != null ? id.toLowerCase() : "user", "123", name, email, role);
    }
```

```
    public String getId() { return id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getRole() { return role; }

    public abstract String getDashboardGreeting();

    @Override
    public String toString() { return name + " (" + role + ")"; }
}
```

File: ValidationException.java | Package: (default)

```
public class ValidationException extends CineBookException {
    public ValidationException(String message) { super(message); }
}
```

Chapter 8
CONCLUSION

The Movie Ticket Booking System (CineBook) successfully demonstrates the practical and production-grade application of Object-Oriented Programming (OOP) concepts in Java SE. By coupling a contemporary Java Swing graphical user interface with relational SQLite database connectivity through JDBC, the application provides an efficient, reliable, and automated platform for managing the operational lifecycle of a modern cinema theatre. The software drastically reduces manual paperwork, eliminates box-office ticketing queues, guarantees data persistence across sessions, and ensures strict referential integrity between shows, seat allocations, and payment records.

The modular architecture of the codebase adheres strictly to industry software engineering best practices. The application implements clear separation of concerns, data encapsulation through private variables and accessors, class inheritance for user roles and payment channels, dynamic runtime polymorphism via method overriding, and interface segregation for search and cancellation capabilities. Advanced design patterns—including the double-checked locking Singleton pattern, Strategy payment pattern, and structural Adapter pattern—demonstrate the real-world utility of object-oriented design. Furthermore, robust custom exception handling protects the system from runtime state inconsistencies and duplicate bookings.

Future enhancements to the system could include integrating network socket broadcasting for multi-client concurrent bookings, SMS and email notification gateways for automated booking confirmations, QR-code scanner validation at theater entrance gates, and integration of external payment gateway APIs. Overall, the Movie Ticket Booking System effectively bridges theoretical concepts from the KTU PBCST304 syllabus with practical software craftsmanship, resulting in an extensible, reliable, and academically distinguished microproject.